

StahlTrace · The Architecture · v6

A self-scoring trace graph for persistent entities, with the cold-start operational layer made first-class.

Authored for: Ronald Moesker · CEO/CTO · COO (open) · Founding Technical Architect (open) **Scope:** Self-scoring trace graph, agent architecture, data substrate, epistemic discipline, the 14-step flow, the cold-start canonical-chapter seeding path, canonical chapters over the fourteen dragnet chapters, multi-layer enforcement, launch path. **Supersedes:** v3.1, v4.0, v5.0, and v5.1 in full. v5.0 collapsed the parallel delta and belief-update ledgers into a single primary object: the trace. v5.1 added the canonical chapter as a co-equal substrate object. **v6 folds the operational learnings of the v4 / v4.1 reference implementation into the two layers v4.1 actually exercised — the canonical-chapter layer and the deep-dive agentic layer — and leaves the core architecture as v5.1's end-state design.** v4.1 was a single-shot, ephemeral trial that ran the dragnet end-to-end on a live ticker; it yields insight into how those two layers should work, but it is not the end state, and v6 does not let it rewrite core machinery. The longitudinal calibration loop, the trace primary, scoring, orchestration, Threadweave, the twelve-table Postgres substrate, the nine enforcement rules, and the parent-updater function are all untouched. What v6 adds, confined to where it belongs: the deep-dive agentic decomposition (§08), the supervisor steer (§14), the cold-start phase as the canonical-chapter seeding path (§17a), the deep-dive-brief ↔ structured-outline binding (§06d), four-layer enforcement (§06.10), honest non-scoring as a deep-dive-layer convention (§40a), and the PM-facing per-run deliverable (§06e). The base agent layer is the ten company-facing chapter agents over the dragnet, with the four meta-chapters in the priors/validation tier (§07) — the Deep Insight decomposition (D/07) is committed. The customer-facing chapter count and whether honest non-scoring reaches core trace state remain **flagged as open** (§25). **Status:** Confidential · draft · pre-operating.

Notice

N/01 · Confidentiality. For the named recipients only. This document is confidential and is shared with the named recipients on the cover for internal review and early-stage diligence. It is not to be forwarded, quoted, excerpted, or published in whole or in part without the prior written consent of Ronald Moesker. Recipients are asked to treat the contents as they would any non-public technical architecture under an implied non-disclosure obligation.

N/02 · Pre-operating status. StahlTrace is not yet a full operating company. StahlTrace Lda is incorporated in Portugal but is not yet a full operating company. It has no commercial customers, no public product, and no contractual engagements with third parties as of the date of this document. The architecture described here is a design in active development. The v4 / v4.1 reference implementation cited throughout is a working code artifact — a sub-agent orchestration with a live trial run on disk — but it is a research prototype, not a product, and the v6 substrate it informs is not yet built.

N/03 · Intellectual property. © Ronald Moesker / StahlTrace Lda. All architectural concepts, naming (including StahlTrace, Threadweave, the six-layer stack terminology, the trace primary, the trace journey, the nine enforcement rules, the canonical chapter object class, the structured outline, the chapter projection model, the Stahltrace Dragnet framework, and the cold-start input phase), diagrams, schemas, and written material in this document are the intellectual property of Ronald Moesker and StahlTrace Lda. No licence, express or implied, is

granted to recipients by receipt of this document. Reproduction, adaptation, or derivative work requires prior written consent.

N/04 · Forward-looking content. Designs are not commitments. Sections describing the launch path, the sealed client codification layer, the commercial tier structure, the v6 phase plan, and any named capabilities of a future system are forward-looking. They describe the design the company intends to build. They are not representations, warranties, or contractual commitments. Any recipient making a decision that depends on those plans should seek direct confirmation from Ronald Moesker rather than relying on this document.

N/05 · No offer, no solicitation. This document is a technical architecture, not an offering memorandum. It does not constitute an offer to sell, or a solicitation of an offer to buy, any security or interest in StahlTrace Lda, and it contains none of the disclosures that would be required for such purposes. It is shared for technical and strategic review only.

N/06 · Draft status and what v6 changes. This is the canonical v6 of the architecture document, the authoritative reference for internal design conversations, and it is nonetheless a draft. v6 amends v5.1 by folding the operational learnings of the v4 / v4.1 reference implementation into the two layers v4.1 actually exercised — the **canonical-chapter layer** and the **deep-dive agentic layer** — while leaving the core architecture (the trace primary, scoring, orchestration, Threadweave, the calibration loop, the nine enforcement rules, and the twelve-table substrate) as the end-state design v5.1 specified. v4.1 is a trail that yields insight, not the end state. The substantive additions are confined to where they belong: the deep-dive agentic decomposition (§08), the supervisor steer (§14), the cold-start phase as the canonical-chapter *seeding* path (§17a), the deep-dive-brief ↔ structured-outline binding and Check/06 (§06d), honest non-scoring as a deep-dive-layer convention (§40a), the four-layer enforcement discipline (§06.10), and the PM-facing per-run deliverable (§06e). The substrate is unchanged at twelve tables. The Deep Insight decomposition is now committed (D/07): the base layer is the ten company-facing chapter agents, the four meta-chapters sit in the priors/validation tier (§07, §08). How a chapter-level score is derived from trace posteriors (D/08) remains **flagged as open** (§25). Future revisions will be labelled v6.1 and so on, or v7.0 if the substrate changes materially.

Questions, corrections, or requests for additional context should be directed to Ronald Moesker. Comments on specific sections are welcome and will be incorporated into the next revision.

Contents *a reading path from premise to shipping plan*

PART I · THE PREMISE

01 · The product, in one page	AMENDED	§01
02 · First principles, the four non-negotiables		§02
03 · The epistemic core, fog of war not narrative fallacy		§03
04 · The five epistemic rules		§04
05 · The trace journey, the operational and epistemic spine		§05
06 · The nine enforcement rules, and how enforcement actually lives	AMENDED	§06
06a · The canonical chapter		§06a
06b · The two authoring modes, seed and re-author		§06b
06c · The chapter type registry, the fourteen dragnet chapters	AMENDED	§06c
06d · The structured outline and Check/06		§06d
06e · The PM-facing deliverable per cold-start	NEW	§06e

PART II · THE AGENT LAYER

07 · The six-layer learning stack		§07
08 · The chapter agents, and the chapter-keyed decomposition	AMENDED	§08
09 · The numerical surface, the stats block and the two-surface PIE	AMENDED	§09
10 · The forward chapters, reachable states not just TAM		§10
11 · CrossValidator, formal contradiction detection		§11
12 · Dot Connector / Insight Engine, cross-trace synthesis		§12
13 · Threadweave, projecting traces into chapters	AMENDED	§13
14 · Support agents, and the per-chapter supervisor steer	AMENDED	§14
15 · Scoring methodology, what “scored by reality” actually means		§15
16 · Identity resolution, anchoring claims to canonical entities		§16
17 · The runtime, the 14-step flow		§17
17a · The cold-start input phase	NEW	§17a
17b · Degrade-safe input handling	NEW	§17b
18 · Security model, authentication authorization encryption response		§18

PART III · THE DATA SUBSTRATE

19 · Data acquisition, what feeds the substrate		§19
20 · Data substrate, twelve source-of-truth tables	AMENDED	§20
21 · Schemas, the tables that carry the moat		§21
22 · Access matrix, who reads and writes what		§22

PART IV · THE OPERATION

23 · The loop, a worked example end to end		§23
24 · Launch scope, what to build, with the cold-start budget	AMENDED	§24
25 · Open questions, what still needs decisions	AMENDED	§25

26 · Operational concerns deliberately out of scope	§26
---	-----

PART V · THE SEALED CLIENT LAYER

27 · The sealed client workbench, additive tenant-isolated	§27
28 · The workbench, what customers can build	§28
29 · The seal, what never crosses the boundary	§29
30 · Delivery and commercial shape	§30
31 · Metering, billing, and customer accounts	§31
32 · API and access surface	§32
33 · The Trace Feed	§33

PART VI · R&D ROADMAP

34 · Framing, what this part is and is not	§34
35 · The corpus, trace lifecycles paired with reality	§35
36 · Prompt evolution, calibration applied to the prompts not just the outputs	§36
37 · Coverage economics	§37
38 · Public Actor Trace, the Worldview & Game Layer	§38
39 · Preconditions	§39

PART VII · THE TRACE ALGEBRA

40 · Magnitude, persistence, and the parent updater	§40
40a · Honest non-scoring in the deep-dive layer	§40a
41 · The trace lifecycle, eight states	§41
42 · Phase plan, migration, and the autonomy commitment	§42
43 · Decision memo	§43

APPENDIX

A1 · What the v6 amendment did, and did not do	§A1
A2 · The deep-dive-brief working artifact	§A2
A3 · What v4.1 confirms about the architecture, no change needed	§A3
A4 · Open questions deferred to v6.1 or later	§A4
A5 · The v4.1 → v6 traceability matrix	§A5

The premise

What we are actually building, and why its shape is the shape it is. The product, the principles, the epistemic core, the five rules, the trace journey, and the nine enforcement rules — and the canonical chapter over the dragnet chapters. Read these before anything else. The rest of the document is consequence.

§01 The product, in one page AMENDED

StahlTrace is a self-scoring trace graph for persistent entities. Every belief in the system is a posterior, Bayesian-updated against an explicit counter-hypothesis set and scored against reality over time — the trace graph is a Bayesian engine first, and everything else (the agents, the chapters, the projections) exists to feed and read it. For each tracked entity the system maintains a graph of traces — living hypotheses about the entity's structure, capital, demand, supply, moat, regulation, fragility, and reachable futures. The graph is hierarchical: child traces track mechanisms (data migration friction, workflow lock-in, audit risk); parent traces track investment-relevant conclusions (switching-cost moat strength, moat durability); chapter projections render the canonical parents into the company narrative. Every piece of evidence updates the most specific relevant traces first; child posteriors propagate upward through defined parent updaters; chapter projections refresh from canonical parent traces. Old traces sleep rather than disappear.

A trace is the primary object. Each trace has a hypothesis, a counter-hypothesis set, a posterior probability, a magnitude, a persistence, a materiality grade, an activation state, a scoring status, and a history of every snapshot the system has ever held about it. Each trace is anchored to one chapter of the company model. Each trace carries its own predictions and falsifiers. Each trace is independently scored against reality. Formal company-chapter beliefs and free-form emerging signals are not separate systems; both are traces, distinguished only by maturity and role. A speculative ad-hoc trace and a canonical moat trace use the same Bayesian engine, the same scoring, the same lifecycle.

The work is organised as a stack. A flat base layer of fourteen chapter agents builds the model, each owning one dragnet chapter: the qualitative chapters read the company, the numerical chapters quantify the economics, the forward chapters map the space of reachable states. A CrossValidator tests them against each other. A Dot Connector identifies cross-trace patterns. Threadweave, the top layer, projects the trace graph forward through opportunity space. The agents do not write to deltas or beliefs, and they do not write posteriors directly; they *propose* trace creations and updates with explicit prior-and-likelihood-ratio arithmetic, and the substrate performs the Bayesian update and commits the snapshot. Agents author the reasoning; the Bayesian engine does the scoring. The substrate enforces the nine rules of §06.

Belief flows up; attention flows down. Bottom-up belief propagation is how the system learns: evidence Bayesian-updates child traces; child posteriors feed parent updaters, which aggregate them into parent posteriors through typed, weighted edges (a precision-weighted Bayesian combination, not a heuristic roll-up); parent posteriors

update chapter projections. Top-down attention propagation is how the system focuses: a parent whose risk has risen tells the system to monitor its children more aggressively. The two flows are kept separate. A parent's concern does not become evidence for a child unless independent evidence appears — belief only ever moves on evidence, never on a parent's prior. This separation is a substrate-level guarantee, not a runtime hope.

The structural / numerical separation, made explicit in v6. The v4.1 reference implementation shipped a clean separation that v6 now commits to architecturally: the chapter-scoring layer scores structurally (a posterior, a magnitude band, a counter-hypothesis distribution — never a fabricated point estimate), while the numerical layer (a price-implied-expectations surface, reverse-DCF cells across WACC $\times$ terminal-growth $\times$ fade-years) is *context for the deep-dive brief, not a score*. The numerical surface informs the prose and the analyst; it never short-circuits the Bayesian scoring of the trace. A trace's posterior is earned through evidence and counter-hypotheses, not read off a discounted-cashflow grid. §09 specifies the two-surface pattern in full.

The loop is self-improving by construction. Every trace update writes a snapshot to an append-only history. Every material trace emits a prediction with a horizon and a falsifier. Reality resolves predictions and scores the trace's posterior, magnitude, and persistence trajectories. Per-entity, per-trace, per-source calibration compounds over time. True signals sharpen the trace graph. False positives narrow the source weights. Missed signals expose blind spots. **This loop is the moat, and v6 does not defer it.** The v4 / v4.1 reference implementation deliberately lacked it — it was single-shot and ephemeral, scoring a ticker once with no longitudinal record — and that is precisely why v4.1 is a reference for the cold-start mechanics and not a substitute for the architecture. v6 takes the working cold-start machinery from v4.1 and bolts it onto the front of the longitudinal loop, rather than letting the cold-start become the whole system.

The initial vertical is finance, where markets provide a fast, unforgiving ground truth. The architecture is domain-general; finance is where calibration is cheapest to come by.

What this delivers in practice has two layers. On day one, the system is a signal/noise filter tied to entities the customer cares about, plus a per-entity cold-start deep-dive that reads like a thoughtful analyst's first full pass. Over time, it becomes a calibrated, self-scoring model whose chapter projections are trustworthy because every posterior in them has been Bayesian-updated against counter-hypotheses, graded against reality, and corrected through compounding source-weight calibration. The cold-start gives the customer something to read on day one; the Bayesian loop is what makes the reading worth trusting by month three.

§02 First principles, the four non-negotiables

Before schemas or agents, four commitments. Every subsequent design decision can be traced back to these. Where a decision conflicts with a principle, the principle wins.

P/01 • Identity-first. One spine, one ID space. Every signal, observation, trace, prediction, and outcome attaches to a stable, auditable identity. The identity graph is the spine. Everything else joins against it.

P/02 • Structured before fluent. Typed schemas, not chat. Agents read and write typed schemas. Free-text reasoning exists only as explanation attached to structured trace updates and counter-hypothesis sets the substrate can verify. If the system cannot verify a claim's shape, it cannot score it.

P/03 • Outcomes close every loop. No prediction is complete until scored. Interpretations only compound once they survive contact with reality. A prediction without an outcome is not yet knowledge, it is a hypothesis held open. A trace update without a calibration target is not yet a trace update, it is a model adjustment.

P/04 • Model-agnostic, moat-bearing. The record is the asset. Claude is the reasoning engine today. The defensible layer is the longitudinal record of reasoning itself, not the model producing it. Tomorrow's model inherits the trace graph; the trace graph does not inherit the model.

These four carry the entire design. The six-layer stack described later is a consequence of P/02 (structured exchange between specialists) and P/03 (every layer's output must be scorable). The substrate described in Part III is a consequence of P/01 (join everything by `entity_id`) and P/04 (append-only, so the record outlives any single model). The trace model of v5 is a consequence of P/02 sharpened (named, addressable hypotheses, not implicit fields) and P/03 sharpened (every trace update must have a calibration target, not just an event).

§03 The epistemic core, fog of war not narrative fallacy

There is a temptation to think of the baseline as the thing of value, the “rich dossier” on each entity. It is not. Baselines can be reconstructed by anyone with enough compute and good prompts. What cannot be reconstructed is what the system knew when it didn't yet know what followed.

Consider two StahlTraces. Both arrive at the same current trace graph today. System A builds it by running the current model over all historical evidence in one pass. System B has been running since entity creation, writing a snapshot every time a trace's posterior, magnitude, or persistence changed, recording the counter-hypothesis the evidence shifted probability away from, emitting predictions against open horizons, and recording outcomes as they landed. Same endpoint. Different objects entirely.

System A produces a trace graph contaminated by hindsight. Every trace it surfaces as material is unconsciously selected for its relevance to outcomes the system already knows happened. System A has no track record, because every “prediction” it could retroactively generate was written after the outcome was known. System A cannot detect regime changes, because regime changes are shifts in interpretation, and A has no record of interpretation over time. System A cannot attribute error, because there is no wrong answer in its history, only the current one. System A also has no posterior trajectories: even if it could reconstruct what it would have asserted about a company at a given date, it cannot reconstruct which counter-hypothesis it would have entertained at that date, because counter-hypothesis weighting is a function of the evidence available at that moment plus the prior the system held going in, and the system did not have a prior going in.

System B is the lived record. Every trace snapshot written at wall-clock time. Every prediction timestamped before resolution. Every calibration snapshot versioned as it shifted. Every counter-hypothesis explicit before the outcome resolved. System B can prove its forecasts weren't fitted to outcomes. System B can show when it changed its mind and why. System B has a legally defensible audit trail.

This also settles the business question. Every serious customer or regulator conversation eventually asks a version of “prove you weren't fitted to outcomes.” System A cannot answer. System B answers trivially with a join. The difference between those two answers is the difference between a subscription product with pricing power and a wrapper over an API with a half-life measured in months.

Given B, you can always derive A as a retrospective view. Given A, you cannot ever reconstruct B. The information that the system didn't know yet is destroyed permanently the moment you stop recording it. This asymmetry, not taste, not aesthetics, is what forces every major architectural decision downstream. The moat is not what you know. The moat is what you knew, when you didn't yet know what followed.

The trace corollary A system that records what it believed before the answer was known cannot do so honestly unless what it believed is a named, addressable hypothesis with a posterior probability, a counter-hypothesis set, magnitude, persistence, and a calibration target. Otherwise the record is of model state, not reasoning.

v5.0 makes the trace the unit of account. A trace is a living hypothesis: it has a probability that the hypothesis holds, a counter-hypothesis set against which the probability is computed, a magnitude (how big the effect is if the hypothesis holds), a persistence (how long the effect lasts), and a snapshot history of every move on those four numbers. Every snapshot is appended; every snapshot is immutable; the trajectory compounds. The previous architecture's separation of operational deltas from epistemic beliefs collapses into a single object because the separation was a workaround for the substrate, not a real distinction. What moved operationally and what changed epistemically were always the same event seen from two angles, and v5 records it once.

§04 The five epistemic rules

An append-only data model is necessary but not sufficient. Five rules, enforced in code and reviewed in every pull request, are what make the system's history epistemically meaningful rather than merely tidy. v5 carries forward four of the v3.1/v4 rules and rewrites the fifth around traces.

Rule 1 · Every trace snapshot is written at the moment it happens, never retroactively. The `created_at` field on a `trace_snapshot` is the wall-clock timestamp of when the system wrote the row. It is distinct from `evidence_time`, which is when the underlying observation happened in the world. If these two fields ever become equal on a `trace_snapshot`, you are writing retrospective, and the row has no epistemic status.

Enforcement: only the live-mode orchestrator writes to `trace_snapshot` with `created_at = now()`. Any retrospective-mode write lands in a separate `retrospective_view` table, never in the canonical `trace_snapshot` history.

Rule 2 · Predictions and trace snapshots are immutable after commit. Outcomes attach, never overwrite. A prediction committed at T holds its probability, mechanism, and falsifier forever. A trace snapshot committed at T holds its posterior, magnitude, persistence, and counter-hypothesis distribution forever. If new evidence arrives, a new snapshot is appended and (when forecastable) a new prediction is emitted. The old ones still stand, possibly destined to be wrong. That is the track record.

This is what protects Brier scores from leakage, posterior calibration from leakage, and the record from being legally defensible.

Rule 3 · Calibration snapshots are versioned, because weights change over time. When the orchestrator applies a source-reliability weight of 0.65 to a Bloomberg headline in May 2026, the calibration state at that moment must be recoverable. A trace snapshot built with May-2026 calibration is a different object from one built with today's calibration, even if the underlying evidence is identical.

Without versioned calibration, you cannot fairly score past trace snapshots. You would be grading 2026 work by 2027 standards, which either flatters or punishes unfairly.

Rule 4 · Retrospective passes are additive, labelled, never authoritative. When a new agent is added, a new evidence type becomes available, or an old reading looks wrong in context, running a retrospective pass is legitimate. Store the output in a parallel `retrospective_view` table, labelled with pass ID and date. Lived and retrospective views can be queried side by side. The authoritative `trace_snapshot` record and the predictions that underwrite Brier and posterior calibration scores remain the lived ones, immutably.

Retrospective views are useful for analysis, for back-filling historical context on newly added entities, and for migration validation (see §42). They are not a substitute for the track record, and they must never be queryable as if they were.

Rule 5 · Every probability is computed against an explicit counter-hypothesis set.

A trace's posterior probability is meaningless without the alternatives it was scored against. A `trace_snapshot` that records `posterior_probability` = 0.78 without an accompanying `counter_hypotheses` distribution is recorded with `posterior_probability` = NULL and excluded from calibration aggregates. No probability without counter-hypotheses is rule six of the nine enforcement rules in §06; rule 5 here is the epistemic statement of that constraint.

Enforcement is hard-gated in code. The Orchestrator's commit step rejects any `trace_snapshot` whose `counter_hypotheses` field is empty when `posterior_probability` is non-null. A rejected snapshot is re-routed to counter-hypothesis generation, which either attaches to an existing counter-hypothesis set or proposes a new one autonomously. There is no override path. The architecture commits to either naming the alternatives a probability was scored against, or declaring the score absent.

Rule 5 is what makes v5.0 a Bayesian trace engine rather than v4.0 with renamed tables. Without it, posteriors become opinions; with it, every posterior carries the alternative space against which it was computed, and the calibration record carries the alternative space too.

§05 The trace journey, the operational and epistemic spine

The architecture is a self-scoring trace graph. Every piece of evidence flows through a 14-step pipeline that produces, updates, scores, projects, and eventually retires traces. This is the operational spine and the epistemic spine at once: there is no separate delta or belief mechanism running underneath. The flow is what every agent in Part II is doing, and the trace graph is what every table in Part III stores.

The 14-step flow is the architecture in motion. Each step has an owner, a substrate write, and a set of conditions for entry and exit. Steps 1 through 8 are inline (typically under 14 seconds for a single observation against a known entity). Steps 9 through 12 are scheduled (sweep cadence varies by entity activity). Steps 13 and 14 run continuously in the background.

Atomic observations are matched, scored, and Bayesian-updated against the trace graph; one `trace_snapshot` per affected trace per `evidence_event` commits in step 08, atomically with the underlying observation.

Material `trace_snapshots` cross the prediction threshold, generate predictions, get scored against later reality, and promote into `chapter_projections` when materiality clears the chapter floor (rule 5 of §06).

Trends are derived from posterior history, never authored (rule 1 of §06). Dormant traces remain available for reactivation, gated against keyword-only matches (rule 4 of §06). Resolved and falsified traces persist as the track record.

Inline: steps 1–8 commit one `trace_snapshot` per affected trace per `evidence_event` Scheduled: steps 9–12 emit predictions, ingest outcomes, recalibrate, promote chapters Continuous: steps 13–14 derive trends and manage trace activation lifecycle

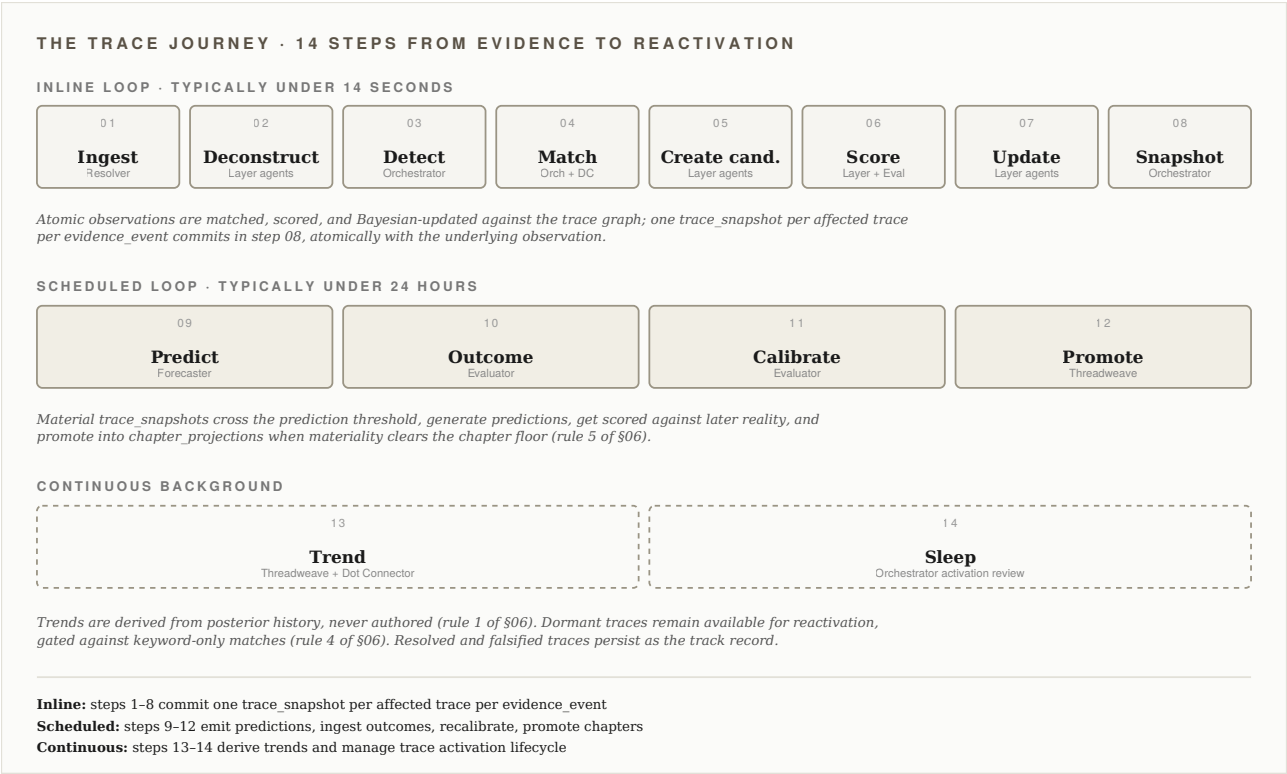


FIGURE 1 · THE TRACE JOURNEY AT A GLANCE. FOURTEEN STEPS FROM EVIDENCE INGEST THROUGH SCORED OUTCOME, CHAPTER PROMOTION, AND LIFECYCLE MANAGEMENT. EACH STEP HAS A SINGLE OWNER AND A TYPED SUBSTRATE WRITE. THE NINE ENFORCEMENT RULES OF §06 GATE THE TRANSITIONS BETWEEN THEM.

The 14-step flow

#	Step	Owner	What happens
01 · Ingest	Information enters the system	Resolver	A filing lands. A call is transcribed. An expert conversation is logged. A market move prints. The Resolver attaches the inbound item to the right entity (§16) and writes one row to <code>evidence_events</code> .
02 · Deconstruct	Decompose into atomic observations	Layer agents	The base layer agents read the evidence event and decompose it into one or more <code>atomic_observations</code> : structurally typed claims, each addressable (an earnings beat, a guidance revision, a capital allocation announcement, a regulatory finding, a competitor move). Each has a unique id and is the unit of work for everything that follows.
03 · Detect	Identify affected entities and dimensions	Orchestrator	For each atomic observation, the Orchestrator identifies which entities are affected (often more than one) and which business dimensions could be involved (moat, demand, supply, capital, regulation, fragility). The dimensions narrow the candidate trace search in step 04.

04 · Match	Match to existing active, dormant, or canonical traces	Orchestrator + Dot Connector	The substrate is queried for traces whose entity_id , chapter_anchor , and scope overlap with the observation. Active traces are passed forward; dormant traces are surfaced for possible reactivation (matching cannot be keyword-only); canonical traces are checked for refinement candidates.
05 · Create candidates	Author new candidate traces if no match	Layer agents	When no existing trace matches and the dimension is material, a layer agent proposes a new candidate carrying hypothesis, counter_hypotheses (non-empty, per rule 5 of §04), chapter_anchor , and scope. Candidates that fail the counter-hypothesis gate are rejected.
06 · Score	Score reliability, novelty, independence, diagnosticity, materiality	Layer agents + Evaluator	Each affected trace receives an evidence packet with five scores: source reliability (calibrated, §15), novelty, independence, diagnosticity, materiality. The materiality grade gates promotion in step 12.
07 · Update	Bayesian-update probability, magnitude, persistence; children first then parents	Layer agents + parent updaters	The update walks affected traces bottom-up. For each leaf, posterior odds = prior odds × likelihood ratios over the counter-hypothesis set. Once children commit, each parent whose children moved is enqueued for the parent updater, which applies per-edge weights and substitute/complement/bottleneck semantics. Parents never read evidence directly; children never write to parents.
08 · Snapshot	Store posterior snapshot	Orchestrator	The Orchestrator writes the new trace_snapshot in one transaction with the linked evidence_event_id and atomic_observation_id . Prior posterior, magnitude, persistence, and counter-hypothesis distribution are preserved. The snapshot is immutable; the trace's current values update; the history grows by one row.
09 · Predict	Generate prediction and falsifier automatically	Forecaster	If the snapshot crosses the prediction threshold, the Forecaster emits a prediction: direction, probability, horizon, mechanism, falsifier, mechanism evidence requirements. Predictions are immutable from emission (rule 2 of §04).
10 · Outcome	Score future outcomes against predictions	Evaluator	When reality resolves a prediction, the Evaluator writes a prediction_outcome with realised direction, magnitude, and mechanism verdict. It reads only the prediction text, the pre-stated mechanism evidence requirements, and the resolution-window evidence — not the originating reasoning chain.
11 · Calibrate	Update source and diagnosticity weights	Evaluator	The Evaluator updates calibration weights for the contributing source and the diagnosticity claim. Calibration is per-source, per-trace-type, per-horizon-tier; rows are append-only with prev_calibration_id chains.
12 · Promote	Promote material traces into chapter projections	Threadweave	Traces whose materiality grade clears the chapter threshold are promoted into the entity's chapter_projections . Promotion is gated by materiality (rule 5 of §06). The projection is the primary surface customers read.
13 · Trend	Derive trends from posterior history	Threadweave + Dot Connector	Trends are derived, never authored. The system reads the posterior snapshot history and computes the trajectory shape (rising, falling, stable, oscillating, regime-shifting). No agent writes a trend value (rule 1 of §06).
14 · Sleep	Keep dormant, resolved, falsified traces available	Orchestrator	Traces that lose materiality go dormant, not deleted. Resolved predictions resolve the trace; a dominant counter-hypothesis falsifies it. All three remain matchable in step 04; the reactivation rule of §41 protects against keyword-only reawakening.

Why this is one journey, not two In v3.1 and v4 the architecture distinguished an operational journey (deltas: what moved in the model) from an epistemic journey (belief updates: what changed in the system's reasoning). Two ledgers, joined transactionally, capturing the same event at different abstraction levels. v5 collapses them. A trace snapshot is both: it records what moved (the prior posterior, magnitude, and persistence are preserved on the new row) and what was reasoned (the counter-hypothesis distribution at the moment of update, the diagnosticity claim, the source reliability applied). One write per material event, one history per trace, one substrate of truth.

The collapse is what permits the nine enforcement rules of §06 to be expressible as substrate-level guarantees. With two ledgers, "no double counting" required a transactional join across them; with one ledger, double counting is a constraint on `evidence_event_id` uniqueness within a trace's snapshot history. With two ledgers, "no probability without counter-hypotheses" required propagating the constraint from belief updates back to deltas; with one ledger, it is a NOT NULL on `counter_hypotheses` when `posterior_probability` is non-null.

A third cadence, canonical chapter authoring Canonical chapter authoring (§06a–§06d) runs on a third cadence, separate from the inline loop and the scheduled sweep. Chapters are authored on threshold accumulation, schedule, falsifying outcome, or explicit request. The 14-step flow continues to describe the trace graph's life; canonical chapters are an authoring surface over the substrate the trace graph populates. Step 15 of §17 specifies the runtime placement: triggered, not sequential, on the four trigger conditions defined in §06b.

The strongest version, in one sentence

StahlTrace is a self-scoring trace graph: every piece of evidence updates one or more living hypotheses; important hypotheses reshape the company chapters; the system tracks its own predictions; old traces sleep rather than disappear.

Every architectural choice in Parts II and III exists to make this sentence enforceable. The nine rules of §06 are what enforcement looks like in practice.

§06 The nine enforcement rules, and how enforcement actually lives AMENDED

The 14-step flow describes what the architecture does. The nine enforcement rules describe what the architecture refuses to do. They are checked in code, in every commit path, in every read view, and in every retrospective pass. A violation is a bug, not a judgment call. The nine rules are unchanged from v5.1; v6 adds one meta-rule (§06.10) describing how enforcement actually lives in the codebase, sourced directly from what the v4.1 reference implementation demonstrated works.

Rule 1 • No manual trend field. Trends are derived from posterior history. No agent writes a trend column. Asking "is this trace's posterior rising" is a query against the snapshot history, not a read of a stored value. Without this rule, agents would store opinions about trajectory that drift from the underlying data.

Rule 2 • No hand-scored Bayesian journey. Posterior, magnitude, and persistence on a `trace_snapshot` are computed by the layer agents from explicit prior × likelihood-ratio arithmetic over the counter-hypothesis set. No agent writes a posterior directly without showing the prior and the likelihood ratios. The substrate retains the LR vector on every snapshot for replay. Without this rule, posteriors become opinions and calibration becomes uninterpretable.

Rule 3 · No double counting. A given `evidence_event_id` may update a trace's snapshot at most once. Subsequent observations that reference the same evidence event are matched to the original event and discarded as duplicate. Independence scoring (step 06) further down-weights evidence that correlates with prior evidence already counted on the trace. Without this rule, a single event ratchets traces upward through repetition.

Rule 4 · No keyword-only reactivation. A dormant trace cannot be reactivated by an observation that merely shares keywords with its hypothesis. Reactivation requires that the new observation materially addresses the dormant hypothesis or its counter-hypothesis set, scored by an independent reactivation gate (§41). Without this rule, dormant traces would wake at every passing mention of their topic and pollute the Trace Feed.

Rule 5 · No promotion without materiality. A trace cannot be promoted into chapter projections (step 12) unless its materiality grade clears the chapter threshold. Materiality is computed at scoring time (step 06) and re-checked at promotion time. Below-threshold traces remain visible in the Trace Feed but do not enter the Deep Dive document. Without this rule, chapters would fill with noise and lose their function as the customer's primary read.

Rule 6 · No probability without counter-hypotheses. A `trace_snapshot` whose `posterior_probability` is non-null must carry a non-empty `counter_hypotheses` distribution that sums to 1.0 within tolerance. The substrate enforces this as a CHECK constraint. A snapshot that violates the constraint is rejected at commit and the originating observation is re-routed to counter-hypothesis generation (step 05). Without this rule, posteriors are opinions in disguise. *Note (v6): whether a trace_snapshot should be allowed to carry a null posterior under an explicit "insufficient evidence" status — rather than only the non-null, counter-hypothesis-bearing form this rule describes — is flagged as an open question (D/11, §40a, §25). Rule 6 itself is unchanged.*

Rule 7 · No belief update from absence unless a prediction failed. The mere passage of time, or the absence of confirming evidence, does not by itself update a trace's posterior. Decay happens only when a prediction the trace emitted failed to be confirmed within its horizon — and then the update is mediated by `prediction_outcomes`, not by a calendar trigger. Without this rule, traces would drift downward through silence and the system would lose information by default.

Rule 8 · No formal chapter pollution from weak traces. Threadweave chapter projections (step 12) are the customer-visible primary surface. They carry only traces whose materiality and posterior strength clear configured floors. Weak traces remain in the trace graph for context, query, and possible reactivation, but they do not appear in the projection document. Canonical chapters (§06a) are governed separately by Check/06 (§06d) and are not subject to this rule; they may include weak traces with explicit `confidence_band='low'` provided the citation is honest.

Rule 9 · No parent overwriting child beliefs. Parent traces never write to child trace posteriors directly. Belief flows up through the parent updater, never down. When a parent's posterior shifts, it triggers attention propagation (§40) — children get higher monitoring priority — but their posteriors only move when independent evidence arrives. The attention path and the belief path are kept on separate substrate writes. Without this rule, parent concerns leak into child posteriors as confirmation bias and the calibration record becomes circular.

§06.10 · How enforcement actually lives in the codebase (new in v6)

The nine rules above describe *what* the architecture refuses to do. This meta-rule describes *where* the refusal is enforced, and it exists because the v4.1 reference implementation taught the project a specific lesson: **a substrate CHECK constraint is necessary but not sufficient.** An agent prompt or a runner code-path can still form invalid *intent* before the substrate ever sees it, and a substrate-only defence catches the violation late, at commit, after compute has been spent and sometimes after a partial write has begun.

Every load-bearing invariant in v6 is therefore enforced at four layers, not one. The canonical example from v4.1 is the forbidden "out of scope" linkage status — the exact failure mode that motivated the whole framework, where a chapter agent under context pressure marked sibling-chapter linkages as out of scope rather than evaluating them. v4.1 made that value unrepresentable at four layers simultaneously:

1. **Human-facing rule.** The obligation is written in prose in the agent's system prompt and the operator documentation — the layer a careful human or a well-behaved agent reads and follows.
2. **Typed schema.** The forbidden value is absent from the JSON-Schema (or Pydantic) enum, so a document carrying it literally cannot validate. An agent that forgets the prose obligation hits the schema.
3. **Code-path gate.** A runtime sentinel in the orchestration code raises on the forbidden value before any write. An agent that bypasses the schema — or a developer who hand-constructs a payload — hits the gate.
4. **Regression test.** A test asserts that the forbidden value raises. A developer who tries to soften the sentinel "just for now" hits the test in CI.

Each layer catches a failure the others miss, and the friction of removing all four is what makes the invariant durable rather than aspirational. In v6 this pattern is mandatory for every one of the nine rules: the substrate CHECK constraint is layer three; layers one, two, and four must also exist. Rule 6 (counter-hypotheses sum to 1.0) is enforced as agent-prompt obligation + outline schema + substrate CHECK + a test that commits a violating snapshot and asserts rejection. Rule 3 (no double counting) is enforced as prompt obligation + an idempotency key on the snapshot write + the substrate uniqueness constraint + a test that submits the same `evidence_event_id` twice. The discipline generalises: *the substrate is the last line of defence, never the only one.*

v4.1 demonstrated the failure of substrate-only thinking concretely. The original NVIDIA TAM failure — a coherent-sounding analysis with an internal contradiction the scoring machinery validated against the truncated state rather than the source-of-truth state — was not a missing constraint. It was a missing layer: the prose obligation existed, but nothing structurally prevented an agent under context pressure from routing around it. Four-layer enforcement is the architectural answer to that class of failure.

Why these nine, why no others

Earlier drafts considered additional rules (no agent self-citation, no probability above 0.99, no posterior shift greater than 0.5 per snapshot). Each was either redundant with one of the nine or better expressed as a calibration constraint than as a hard rule. The nine are the minimum sufficient set: each prevents a specific failure mode that, if unaddressed, would invalidate the trace graph as an audit-grade record; each is enforceable as a substrate constraint or a code-path gate, not a review-time judgment; each is testable. A tenth rule remains under consideration for v6.1: *no chapter projection that does not name a falsifying observation*. It is not yet ruled, because the operational definition of "falsifying observation for a chapter" needs work. The flag is here so the nine do not become ten quietly.

§06a The canonical chapter

The architecture distinguishes two surfaces that share the word "chapter" and must not be conflated.

`chapter_projections` (existing). Threadweave's customer-facing read surface, machine-assembled per Threadweave run, populated only with traces that clear the materiality floor (rule 5 of §06). One row per `(entity_id, intent_assumption, projection_version)`. Single writer: Threadweave. Read surface, not authoring surface. **Unchanged from v5.1.** The customer-facing chapter count for this surface is left open in v6 (D/06, §25); v6 does not alter Threadweave or impose a collapse rule.

`canonical_chapters` (introduced in v5.1). The specialist-authored long-form prose deep-dive per `(entity_id, chapter_type)`. Versioned, append-only, retrievable as a coherent document. Authored by the specialist that owns the `chapter_type` (per §06c). The substrate's primary record of *what the system has written* about an entity, as distinct from *what the system has typed* about an entity, which lives in `atomic_observations` and `trace_snapshots`.

Canonical chapters and the trace graph are mutually constraining. The relationship is bidirectional and mode-dependent (§06b). Both are first-class. Neither is derivative. The Fuchs SE deep-dive in the corpus is the model of a fully populated canonical chapter set: original prose on top, trace status block underneath each chapter, evidence flowing into traces, traces constraining the next prose revision. **v6 grounds this object class in a working precedent:** the v4.1 reference implementation produced exactly this artifact for a live ticker — ten chapter-keyed deep-dive briefs, each with cited findings mapping to substrate, totalling roughly 290 KB of cited content, all schema-valid, with zero hallucinated facts in post-hoc review. The canonical chapter is not a hoped-for object; v4.1 shipped its single-shot ancestor.

What a canonical chapter contains

A canonical chapter row holds the prose body in markdown; a `structured_outline` (the typed claim list that maps prose passages back to substrate, load-bearing per §06d); a `built_from` payload (IDs of the `trace_snapshots`, `atomic_observations`, and prior canonical chapter version this draft was generated from); an `author_specialist_id`, `prompt_version`, `model`, and authoring timestamp; a `status` enum (`draft`, `active`, `superseded`, `withdrawn`); and optional `reasoning_notes`, the specialist's diff narrative explaining what changed from the prior version and why. A canonical chapter is never edited in place. Each revision is a new row, marked `active`; the prior `active` row is moved to `superseded` in the same transaction. Old versions remain queryable as the historical record of the system's narrative understanding.

What a canonical chapter is for

The canonical chapter is the customer's primary read for narrative understanding of an entity. It is the artifact a fund manager opens when asking "what does the system think about Company X's moat." *Trace_snapshots* are *what* the system thinks; canonical chapters are *how* the system explains what it thinks. A canonical chapter without a structured outline is prose without grounding. A trace graph without canonical chapters is grounding without explanation. The architecture commits to both, with the structured outline as the bridge.

§06b The two authoring modes, seed and re-author

A canonical chapter is authored in one of two modes. The mode is determined by the substrate state at invocation time, not by the agent's discretion. The harness selects the mode before dispatching the specialist.

Seed mode

The (entity_id, chapter_type) pair has no active traces in the chapter's scope. The agent reads source material — atomic_observations attached to the entity, the source pointers behind them, peer comparisons where the chapter_type's contract specifies — and authors prose freely. Alongside the prose, the agent emits a structured_outline whose entries declare proposed traces. At commit, the harness materialises the outline: each outline entry becomes a candidate trace with a first trace_snapshot, walking steps 05 through 08 of the 14-step flow. Traces that pass become active; traces that fail are rejected with a typed reason recorded in reasoning_notes. The chapter row and the surviving trace rows commit in a single transaction. The chapter is the seed; the traces are the harvest. Both commit together or neither commits.

Seed mode is gated on the absence of active traces matching the chapter's scope. It is the cold-start path: used when an entity first enters the universe, when an existing entity adds a chapter_type not previously covered, or when an analyst explicitly bootstraps a chapter. **In v6, seed mode is the bridge to the cold-start input phase (§17a):** the per-chapter deep-dive briefs produced during cold-start are the natural seed-mode inputs, and the v4.1 P16 Ch.11 checklist (§06c) is the structured seed template for the chapters that have one.

Re-author mode

The (entity_id, chapter_type) pair has existing active traces in scope. The agent receives those traces' current state — the most recent trace_snapshots with their typed fields — as part of the evidence packet, marked binding. The agent authors new prose freely in form (section structure, narrative arc, emphasis, framing, length) and emits a new structured_outline whose entries cite existing trace_snapshot_id values. At commit, the harness runs Check/06 (§06d), the deterministic consistency check between outline claims and cited snapshots. On pass, the new chapter row is written and the prior version superseded in the same transaction. On fail, the commit is rejected and the typed mismatch list returned for retry with a regenerated prompt; if retries exhaust, the re-author attempt is logged as failed and the prior version remains active.

Re-author is triggered by the harness, not the agent, on one of four conditions: **material accumulation** (snapshots committed since the last version, weighted by magnitude and posterior shift, exceed a chapter_type-specific threshold); **explicit user request**; **scheduled refresh** (every N months per chapter_type, to catch drift the threshold rule misses); and **falsifying outcome** (a prediction tied to a cited trace has resolved incorrect or partial). The mode-selection logic is a deterministic harness function, written as typed code. It is not an LLM judgment.

Why two modes and not one

The chapter ↔ trace relationship is generative at cold start (chapter creates traces) and referential in steady state (traces constrain chapter). A single-mode design that always treats traces as primary cannot bootstrap from prose; a single-mode design that always treats prose as primary cannot accumulate corrections through evidence. Two modes, with mode-selection driven by substrate state, is the minimum sufficient design. The same structured_outline schema serves both modes; the harness disambiguates by checking the substrate for existing active traces before invocation.

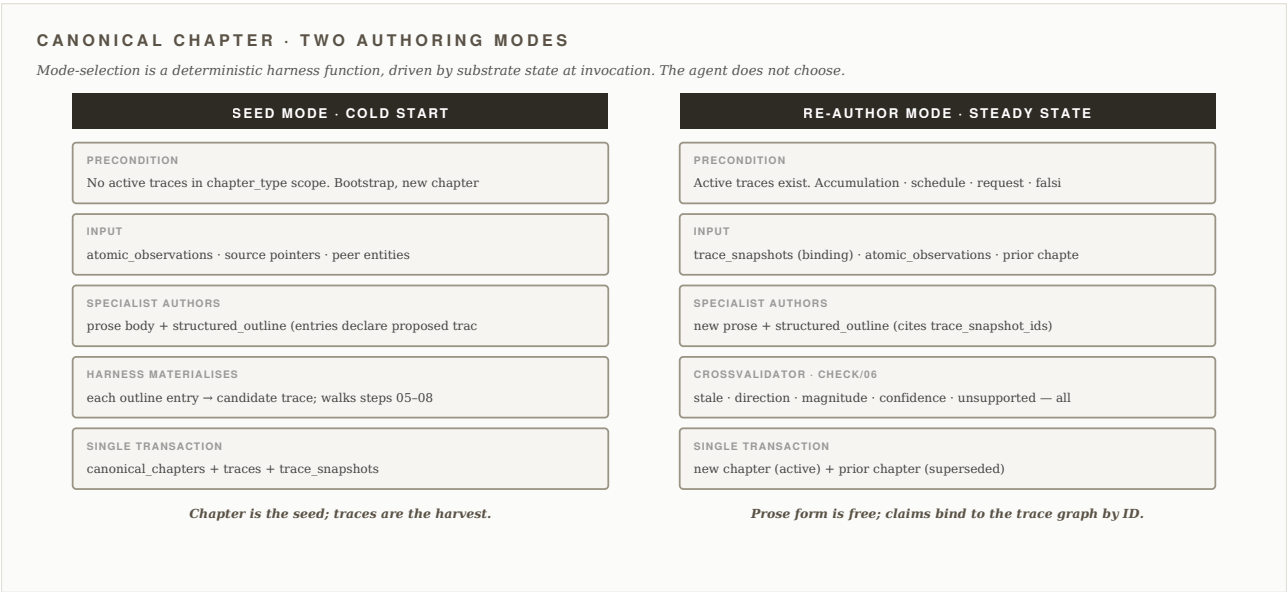


FIGURE 13 · THE TWO AUTHORING MODES FOR CANONICAL CHAPTERS: SEED (GENERATIVE) AND RE-AUTHOR (REFERENTIAL, BOUND BY CHECK/06).

§06c The chapter type registry, the fourteen dragnet chapters AMENDED

The registry is a typed catalogue of canonical chapter types. Each row pairs a `chapter_type` to its author specialist, in-scope trace types, accumulation thresholds, refresh interval, model tier, and peer-comparison contract. The registry is curated, not LLM-authored: new `chapter_types` are proposed in writing, reviewed, and added by migration. It is read-only to specialists and to the harness.

What v6 settles, and what it leaves open. v5.1 left the canonical chapter set partly unspecified — §06a referenced an "eight-chapter document" and §06c listed ten illustrative chapter types, neither reconciled with the framework the system actually scores. The v4.1 trial operationalised the full **StahlTrace Dragnet v1.2 — fourteen chapters, 996 nodes** — and authored canonical deep-dives against it on a live ticker. v6 takes the narrow, well-supported step: **the canonical-chapter authoring layer uses the fourteen dragnet chapters as its chapter types**, because that is the granularity at which v4.1 actually produced specialist deep-dives and the granularity the framework checks. This is a canonical-chapter-layer decision, not a change to the customer-facing `chapter_projections` surface, which is core (§13) and unchanged from v5.1.

One larger question the v4.1 analysis raised against this section is **deliberately left open**, because it is a core read-surface decision rather than a canonical-chapter-layer mechanic. A second — how the Deep Insight layer decomposes — is now **resolved**:

- **The customer-facing chapter count** (does the projection surface stay at eight, expand to fourteen, or collapse many-to-one from the authoring chapters). v6 does not decide this; it is flagged as D/06 in §25. The fourteen-chapter authoring set does not force a fourteen-chapter read surface, and v6 declines to import a collapse rule from the trial.
- **The Deep Insight decomposition** (whether the six DI specialists are realised as chapter-keyed agents over the fourteen chapters). **Resolved (D/07): committed.** The base layer is the ten company-facing chapter agents; the mapping below is the committed ownership, not a candidate. See §07 and §08.

The fourteen dragnet chapters and a candidate specialist mapping

The table below shows the fourteen chapter types the canonical-chapter layer authors against, with the specialist ownership v4.1 carried implicitly in its prompt files. **The ownership column is committed (D/07 resolved)** — it is the chapter-to-specialist assignment v6 adopts, the shape v4.1 ran on a live ticker. The ten company-facing chapters are the base-layer deep-dive authors; the four meta-chapters (ST-1, ST-2, ST-11, ST-13) score the run and are owned by the priors band, CrossValidator, and Evaluator rather than by base-layer agents (§07).

ST CHAPTER	CANONICAL CHAPTER_TYPE	OWNING SPECIALIST (COMMITTED)	COMPANY-FACING?
ST-1	population_priors	Orchestrator (meta) + DI/06 Sentiment	meta
ST-2	knowability	ST-13 / Evaluator (meta)	meta
ST-3	fragility / no-ruin	DI/05 Fragility	yes
ST-4	switchability	DI/04 Moat Mechanisms	yes
ST-5	demand_durability	DI/02 Effective Utility	yes
ST-6	economic_engine	FM/01 Capital	yes
ST-7	moat	DI/04 Moat Mechanisms	yes
ST-8	leadership / capital_allocation	DI/01 Supply	yes
ST-9	opportunity_space	SF/01 Demand (strategy)	yes
ST-10	dynamic_change	SF/02 Dynamic Exploration + DI/03 Anomalies	yes
ST-11	system_coherence	CrossValidator (meta)	yes
ST-12	price_implied_expectations	FM/02 Forecasting	yes
ST-13	thinking_quality	Evaluator (self-supervision)	meta
ST-14	endgame	SF/03 Endgame Paths	yes

Ten of the fourteen are company-facing and map cleanly to a P16 Investment Framework v1.47 Chapter 11 checklist (ST-3, ST-5, ST-6, ST-7, ST-8, ST-9, ST-10, ST-11, ST-12, ST-14) — these are the ten v4.1 actually authored deep-dives for, and they inherit the P16 Ch.11 checklist as their seed-mode template (§06d). Four are meta-chapters scoring properties of the run rather than the company (ST-1 Population Priors, ST-2 Knowability, ST-4 Switchability where it overlaps moat, ST-13 Thinking Quality); these need their own structured-outline patterns, which v4.1 did not provide and which v6 flags as deferred (§A4).

Two notes on the mapping. First, DI/06 Sentiment has no direct ST-chapter ancestor in v4.1 — the closest is the recent-dynamics signal/noise tagging in the top-level node. v6 sites Sentiment in ST-1 Population Priors as an extension, pending a decision on whether it warrants its own chapter (§A4). Second, each `chapter_type` row still carries the v5.1 fields: `trace_types_in_scope`, a three-component `material_accumulation_threshold`, a `scheduled_refresh_interval` (6 months for slow-moving types like moat and structural_future, 3 for sentiment and demand_durability, 1 for regulatory_exposure under an active alert), `tier_assignment` (Sonnet for most, Opus for synthesis-tier types), and `peer_comparison_required` (true for moat and endgame).

§06d The structured outline and Check/06

The `structured_outline` is the load-bearing surface that mediates the canonical chapter ↔ trace graph relationship. The prose is free-form. The outline is typed. Every substantive claim in the prose has a corresponding entry in the outline; the outline is what the harness validates.

Schema of the structured outline

A `structured_outline` is an ordered list of `outline_entry` rows. Each entry has a `section_path` (which part of the prose this corresponds to; free-text but stable across versions); a `claim_text` (a paraphrased complete sentence of the prose's assertion; empty rejected); a `claim_direction` (one of `strengthening`, `weakening`, `stable`, `bidirectional_uncertain`); a `magnitude_band` (`low`, `moderate`, `high`, `structural`, aligned with the cited snapshot's magnitude within tolerance); and a `confidence_band` (`low` 0.30–0.55, `medium` 0.55–0.75, `high` 0.75–0.92, `near_certain` >0.92, aligned with the cited snapshot's posterior within tolerance).

Check/06, the consistency gate

Check/06 is the deterministic validator CrossValidator runs at every canonical-chapter commit. It is the prose-to-substrate analog of the numerical checks Check/01–05 run on trace updates. It blocks the commit on any of five mismatches: **stale_citation** (the cited `trace_snapshot_id` is no longer the latest snapshot for its trace), **direction** (the outline's `claim_direction` contradicts the snapshot's posterior movement), **magnitude** (the `magnitude_band` disagrees with the snapshot's magnitude beyond tolerance), **confidence** (the `confidence_band` disagrees with the snapshot's posterior beyond tolerance), and **unsupported_citation** (a prose claim with no outline entry, or an outline entry citing no snapshot). All five are blocking. The retry budget is three; on exhaustion, the prior chapter version remains active and the failed attempt is logged.

v6's connection to the v4.1 brief format. Check/06 is not new machinery invented for the architecture; it formalises a discipline v4.1 already exercised. The v4.1 deep-dive brief carried `checklist_items[].finding` (the structured-outline claim), `checklist_items[].evidence_citations[]` (the substrate evidence link), `checklist_items[].load_bearing_for_st_nodes[]` (the outline→trace mapping), and an `explicit_handoff_to_chapter_agent` (the typed-claim handoff into trace updates). These map one-to-one onto the structured outline. v6's instruction is explicit: *do not let the P16 Ch.11 checklist mapping get lost*. It is the most concrete operationalisation of "specialist-authored canonical chapter" the project has produced, and it is the seed-mode authoring template for the ten company-facing chapters.

§06e The PM-facing deliverable per cold-start NEW

v5.1 specified the canonical chapter as a data product but was silent on the human-facing artifact a customer actually opens. The sealed-client sections discuss the workbench and the Trace Feed, but not the per-run branded read. v4.1 shipped exactly this and proved customers ask for it first: an eight-part branded PDF — Cover, Contents, Sections I–IV (Identity → Delta loop → Scoring loop → Synthesis), Appendix — that mirrors all run content, enforced by a minimum-content test so it can never silently degrade to a stub.

v6 commits to the PM-facing deliverable as the minimum read artifact for any new-entity cold-start. The structure is locked: a cover with entity identity and run metadata; a contents page; an **Identity** section (the stats block and origins); a **chapter** section rendering the canonical chapters with, beneath each, an evidence / interpretation / judgment split and the trace status block; a **synthesis** section (the endgame distribution and the cross-chapter convergence flags); and an appendix carrying the full citation chain. The deliverable is generated from substrate, not hand-assembled: it is a render of the `canonical_chapters` rows plus the `top_level_node` plus the promoted traces, so it cannot drift from what the system actually holds. When a customer asks "what's the actual artifact I read," the answer is this PDF, not the `chapter_projections` table.

PART II

The agent layer

Six layers that make reasoning measurable: a flat base layer of fourteen chapter agents (no Deep Insight / Financial / Synthetic Futures cluster tier), a cross-validator, a hypothesis engine, a path engine on top, plus four support agents that close the loop with reality. v6 leaves the core agents as v5.1's end-state design and amends only the chapter-keyed decomposition (§08), the numerical surface — stats block and two-surface PIE (§09), and the Orchestrator's supervisor steer (§14); it adds the cold-start seeding path (§17a) and a degrade-safe input note (§17b).

§07 The six-layer learning stack AMENDED

Read top-down for ambition: Threadweave is where the architecture is most ambitious; the base layers do the bulk of the work. Read bottom-up for information flow: observations land in L1-L3, propagate to L4 for validation, are committed at L5 cadence, get woven into paths at L6. Each layer has a distinct epistemic job and a distinct output shape.

v5: every layer writes `trace_snapshots` through the Orchestrator. The substrate is twelve source-of-truth tables. Nine enforcement rules: §06.

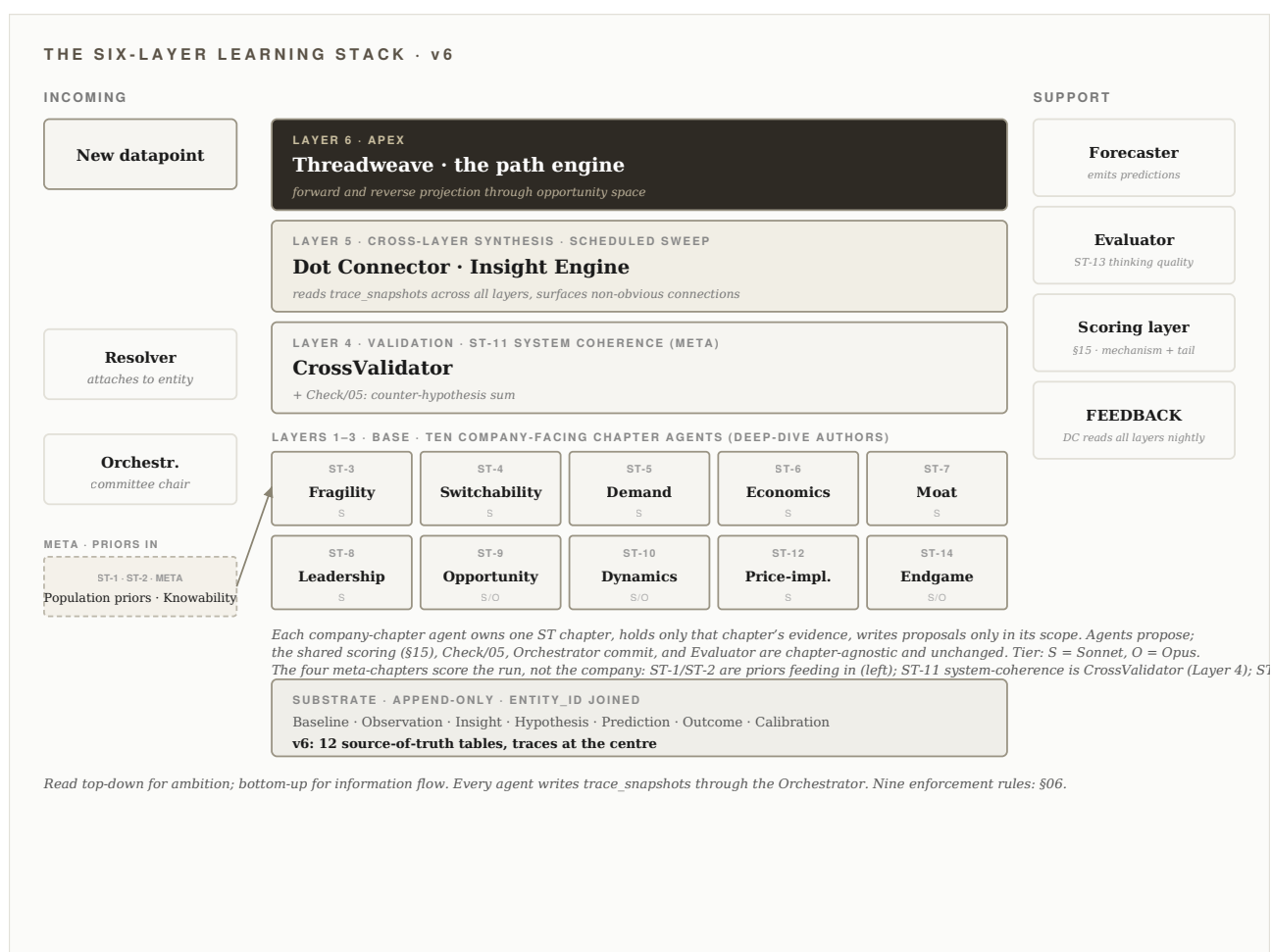


FIGURE 2 · THE SIX-LAYER LEARNING STACK, V6. THE BASE LAYER (LAYERS 1-3) IS THE TEN COMPANY-FACING CHAPTER AGENTS (THE DEEP-DIVE AUTHORS), ONE PER COMPANY-FACING DRAGNET CHAPTER, REPLACING THE V5.1 THREE GENERIC CLUSTERS. THE FOUR META-CHAPTERS ARE NOT PEERS IN THE BASE GRID: ST-1 POPULATION PRIORS AND ST-2 KNOWABILITY FEED IN AS PRIORS (LEFT); ST-11 SYSTEM COHERENCE IS THE CROSSVALIDATOR (LAYER 4); ST-13 THINKING QUALITY IS THE EVALUATOR. AGENTS PROPOSE; THE SHARED SCORING (\$15), CHECK/05, ORCHESTRATOR COMMIT, AND EVALUATOR ARE CHAPTER-AGNOSTIC AND UNCHANGED.

The agent layer is organised as a stack, read top-down in the order ambition increases and bottom-up in the order information flows. Each layer has a distinct epistemic job, a distinct output shape, and a distinct place in the trace journey.

Layer 6 · apex · Threadweave. Weaves the probable path of the entity through opportunity space over time. Reads the insight graph primarily, with confirmed hypotheses sharpening the path and unresolved ones widening uncertainty. Falls back to the base models when an insight does not cover a specific question. The core investment question lives here: can this entity turn opportunity space into economic reality faster and more efficiently than competitors?

Layer 5 · cross-layer synthesis · Dot Connector / Insight Engine. Reads across every layer below (all recent observations, all recent `trace_snapshots`, base-model states, the hypothesis ledger, CrossValidator's disagreements) on a scheduled sweep, and synthesises insights that no single layer could produce alone. Insights are the primary output. Hypotheses are emitted as children only when an insight is forecast-worthy. Writes to the Insight table, and to Hypothesis when forecastable.

Layer 4 · validation · CrossValidator. Formal contradiction detection across the base chapter agents, plus probability conservation across counter-hypothesis sets. Five typed checks (coherence, dependence, adjacency,

preservation, probability conservation). Disagreements are preserved as unresolved disagreements rather than averaged into a number.

Base · the forward chapters (ST-9, ST-10, ST-14). Probable futures, reachable states, path dependencies, key uncertainty nodes. Not “how big is the market today” but how many new reachable states can this entity assemble over time? Assembly-theoretic framing (§10).

Base · the numerical chapters (ST-6, ST-12). Revenue drivers, margins, capital efficiency, unit economics, reinvestment runway, balance sheet risk, dilution, operating leverage. Every update shows exactly what changed (§09).

Base · the qualitative chapters (ST-3, ST-4, ST-5, ST-7, ST-8). Qualitative understanding: strategy, product, culture, moat, customer love, regulation, management, competition, incentives. Outputs living models with evidence, confidence, falsifiers, and explicit observations (§08).

Layers 1 to 3 run in parallel on a given observation, with lateral read-only consultation allowed. Layer 4 validates every observation inline before it commits, with the trace journey (§06) running inside its window. Layer 5, the Dot Connector, runs on a different cadence entirely: it reads the full persisted substrate on a scheduled sweep (typically nightly) and looks for patterns across layers and across `trace_snapshots` that no single observation would reveal. Layer 6 consumes Layer 5’s insight graph as its primary input and falls back to the base models only when an insight does not cover a specific path question.

Each layer’s specialists also author canonical chapters in their `chapter_types`-of-record per §06a–§06d. Chapter authoring runs on a separate cadence from inline trace updates; the same specialist may operate at different model tiers across the two surfaces, per the chapter type registry (§06c).

Four support agents (Resolver, Orchestrator, Forecaster, Evaluator) are described in §14. They are not layers in the stack; they wrap it. The Orchestrator is the committee chair for the whole stack. The Resolver decides what entity an incoming claim belongs to. The Forecaster emits the prediction when the Insight Engine flags an insight as forecastable. The Evaluator closes the loop when reality resolves.

How the base layers decompose in v6: the fourteen chapter-keyed agents

The figure above shows the v5.1 form of the base layers as three generic clusters — Deep Insight (six unspecified specialists), Financial Model (two), Synthetic Futures (four). v6 commits to the more concrete shape that the v4.1 reference implementation actually ran on a live ticker: **the base-layer specialists are realised as fourteen chapter-keyed agents, one per dragnet chapter (ST-1 ... ST-14), not as undifferentiated clusters.** Each agent owns exactly one chapter of the company model (the ownership map is in §06c), holds only that chapter's evidence in its context window, and may write proposed trace updates only within its chapter's scope. The old cluster names survive only as an informal reading lens, never as a structural tier: one can still speak of the *qualitative* chapters (ST-3 fragility, ST-4/ST-7 switchability and moat, ST-5 demand, ST-8 leadership), the *numerical* chapters (ST-6 economic engine, ST-12 price-implied expectations), and the *forward* chapters (ST-9 opportunity space, ST-10 dynamic change, ST-14 endgame) — but there is no Deep Insight / Financial Model / Synthetic Futures agent that authors as a unit. The unit of work is the chapter agent. §08, §09, and §10 document the qualitative, numerical, and forward chapters respectively.

The chapter-keyed decomposition matters for the two reasons v4.1 demonstrated. First, **context budget**: no single agent holds all fourteen chapters and all 996 dragnet nodes at once, which is the exact failure mode — silent dossier truncation, narrative-from-memory replacing fact-from-evidence — that motivated the framework.

Second, **contradiction detection**: when an agent can write only into its own chapter's paths, two agents touching the same path with incompatible proposals is a *syntactic* flag at CrossValidator, not a semantic one. In the v4.1 live trial, all ten company-facing chapter agents dispatched in parallel on Sonnet in roughly four-to-seven minutes each, producing 6–11 cited findings apiece.

How the fourteen agents combine with scoring and evaluation

This is the load-bearing reconciliation, and it is deliberately conservative: **the fourteen chapter agents change who authors, not how anything is scored**. They sit exactly where the generic clusters sat — as proposal sources feeding the unchanged scoring loop beneath them. The division of labour is "agents propose, the shared scoring layer disposes":

- A chapter agent reads its chapter's evidence and emits *proposed* trace creations and trace updates, each carrying a hypothesis, a non-empty counter-hypothesis set, and the explicit prior $\times$ likelihood-ratio arithmetic (rule 2 and rule 6 of §06). It does not write a posterior to the substrate; it proposes one with its work shown.
- **Scoring (§15) is chapter-agnostic and shared**. The same source-reliability weights, novelty, independence, diagnosticity, and materiality scoring apply to every proposal regardless of which chapter agent produced it. A fragility-chapter proposal and a moat-chapter proposal are scored by the identical machinery.
- **CrossValidator (§11), the Orchestrator commit (step 08), and the parent updater (§40) are unchanged**. Check/05 enforces counter-hypothesis sum-to-one on every proposed snapshot; the Orchestrator alone commits trace_snapshots; the parent updater rolls child posteriors into canonical parents — all exactly as the core specifies, with the chapter agents simply being the upstream authors.
- **The Forecaster and Evaluator (§14, §15) never see chapter identity**. The Forecaster emits a prediction from any material snapshot; the Evaluator scores the outcome reading only the prediction text and pre-stated mechanism requirements (Evaluator independence). The longitudinal calibration loop compounds per-source and per-trace-type, not per-chapter-agent — so adding the fourteen-agent decomposition does not fragment or weaken calibration.

The net effect is that the fourteen chapter agents are a *better-organised front end* on the identical scoring, validation, and calibration substrate. Nothing downstream of the proposal changes. This is why the decomposition can be adopted without touching the core loop, and why it is sited in the agent-authoring tier rather than treated as a new scoring mechanism. The four meta-chapters (ST-1, ST-2, ST-11, ST-13) score properties of the run rather than the company and are owned by the support agents and CrossValidator rather than by base-layer chapter agents (§06c); they propose into the same scoring layer where they touch company traces and are otherwise diagnostic-only.

One scope note, carried from §08 and §25: committing the fourteen-agent decomposition here resolves what earlier drafts flagged as open question D/07. The decomposition is now the committed base-layer shape. What remains open is finer — sub-chapter granularity within an ST chapter and where DI/06 Sentiment sits in the dragnet (§25) — not whether the chapter-keyed shape is adopted.

§08 The chapter agents, and the chapter-keyed decomposition AMENDED

The base layer of the stack (§07) is the set of chapter agents that automate the work of a thoughtful analyst: reading everything a company says and does, and building a mental model that survives more than a quarter. Their

output is a living model of structured claims — about strategy, moat, culture, regulation, management, competition, incentives, economics, and reachable futures — each attached to evidence, confidence, a falsifier, a history of observations, and the traces each observation moves. There is no Deep Insight / Financial Model / Synthetic Futures cluster tier between the agents and the trace graph; v6 commits to the flat fourteen-chapter decomposition the v4.1 reference implementation ran on a live ticker. Each agent owns exactly one dragnet chapter and writes proposed trace updates only within that chapter's scope.

Ten of the fourteen chapters are company-facing — the deep-dive authors — and they divide the analyst's work by subject rather than by a separate agent tier:

- **ST-3 fragility** — what unusually smooth results may hide; the no-ruin question.
- **ST-4 switchability** and **ST-7 moat** — whether the advantage reinforces itself; the derivative, not the width.
- **ST-5 demand** — the real function the company serves and the truth of its demand, including where the company is structurally different (durability over novelty).
- **ST-6 economics** — what the financials say now: ROE/ROIC stability, leverage, cashflow quality, unit economics, capital allocation.
- **ST-8 leadership** — talent density, incentive alignment, capital-allocation track record.
- **ST-9 opportunity**, **ST-10 dynamics**, and **ST-14 endgame** — the forward chapters: the strategic direction of travel, paradigm shifts and inflection points, and where the industry resolves.
- **ST-12 price-implied expectations** — what the current price requires the business to do, scored structurally (§09).

Each agent is a Claude instance with a narrow system prompt, a typed tool set, and strict read/write scope confined to its chapter. They do not chat; they exchange structured proposed trace updates through the substrate. (The fact-sentiment gap that earlier drafts assigned to a standalone "Sentiment" specialist has no committed home in the company-facing set; it is sited provisionally in the ST-1 priors meta-chapter pending the §A4 decision, D/09.)

The operational shape, learned from v4.1

v5.1 described the base agents as a handful of general processes grouped by discipline. v6 commits to the more concrete and more scalable shape the v4.1 reference implementation ran on a live ticker: **the base agents are realised as chapter-keyed agents over the fourteen-chapter dragnet, not as discipline clusters.** The mechanics and the reconciliation with scoring are specified in §07; the per-chapter ownership map is in §06c. v4.1 ran fourteen chapter-scoring sub-agents (one per ST chapter, each in its own context) plus ten chapter-keyed deep-dive sub-agents (one per company-facing chapter). Each deep-dive produced 6–11 cited findings against its chapter's P16 Ch.11 questions, and the live trial dispatched all ten in parallel via Sonnet in roughly four-to-seven minutes per agent. The decomposition is attractive for two reasons. First, context budget: no single agent has to hold all fourteen chapters and all 996 nodes at once, which is the exact failure mode — silent dossier truncation, narrative-from-memory replacing fact-from-evidence — that motivated the framework. Second, contradiction detection: when an agent can only write into its own chapter's paths, two agents touching the same path with incompatible proposals is a *syntactic* flag at CrossValidator, not a semantic one. Committing this resolves what earlier drafts flagged as open question D/07; the chapter agents author, the shared scoring and evaluation layer (§15) disposes, and the core loop is untouched.

The four remaining chapters are meta-chapters that score the run rather than the company and do not sit in the base grid (§07): ST-1 population priors and ST-2 knowability feed in as priors; ST-11 system coherence is the

CrossValidator; ST-13 thinking quality is the Evaluator. Each company-facing agent produces, as part of its observation, the `trace_snapshot` rows that attach to the affected traces.

§09 The numerical surface, the stats block and the two-surface PIE AMENDED

Two of the company-facing chapter agents do the numerical work that forces every qualitative read into numbers — **ST-6 economics** (what the financials say now: ROE/ROIC stability, leverage, cashflow quality, unit economics, capital allocation) and **ST-12 price-implied expectations** (what the financials will have to say: FCF paths, reinvestment runway, growth limits, the variables that must work). They are chapter agents like any other (§08), not a separate financial tier. The discipline they enforce is cross-chapter: every qualitative observation that implies a financial consequence must produce a corresponding numerical observation within the same orchestration window, or the Orchestrator flags the insight as unsupported. An insight that cannot find a home in the financials is a narrative, not an investment view. This section specifies the two numerical primitives those agents rely on — the stats block and the two-surface price-implied-expectations separation.

The stats block, a typed numerical anchor (new in v6)

v4.1 shipped a single typed primitive that every numerical agent reads first — the **stats block** — and v6 carries it forward as a *deep-dive-layer working artifact*, not a new source-of-truth table (§20). The stats block carries `current_price`, `market_cap`, `shares_outstanding`; `valuation_multiples` (trailing/forward P/E, PEG, EV/Revenue, EV/EBITDA, P/B, P/S); `return_ratios` (ROE and ROIC at TTM / 5-year / 10-year, an assumed WACC, and the ROIC–WACC spread); `cashflow` (FCF TTM, OCF TTM, capex); `chart_assets` (paths to a one-year and a max-period log-scale 200-week-moving-average chart); and `data_provenance` (source, fetched-at timestamp, a manual-cross-check flag, and a `missing_fields[]` list). In the end-state it is an `atomic_observation` of the entity's market state that the numerical chapter agents read as their anchor — pre-computed once per run for within-run determinism, not given a privileged table.

The block's critical property is that it is **null-tolerant**: every numeric field may be null, with the absence recorded in `data_provenance.missing_fields[]` rather than guessed. This makes the schema robust to data-source failures — an unavailable market-data feed, sparse coverage, a recent IPO — and it is the natural home of the degrade-safe discipline of §17b. Pre-computing the stats block once per run is cheaper than asking every numerical agent to fetch its own market data, and it guarantees determinism within a run: every numerical agent reads the same numbers, so a re-spawn during convergence does not get confounded by source drift.

The two-surface PIE separation (new in v6)

The most important discipline v4.1 proved, and the one v6 commits to architecturally, is the separation of *structural* from *numerical* price-implied expectations. At the chapter-scoring layer, ST-12 (price-implied expectations) is scored **structurally**: a posterior, a magnitude band, a counter-hypothesis distribution — and, under an anti-fake-precision rule, a point estimate only when the evidence genuinely supports one. At the deep-dive layer, a separate **numerical** surface is produced — a reverse-DCF sensitivity grid of price-implied growth across $WACC \times \text{terminal-growth} \times \text{fade-years}$ — and it is consumed by the ST-12 deep-dive specialist *as context for the brief, not as a score*.

This separation is the architectural answer to the original failure that named the project. A reverse-DCF grid produces a precise-looking number, and the temptation is to let that number *be* the valuation judgment. It cannot be: a discounted-cashflow output is a function of assumptions, and treating it as evidence launders the assumptions into a posterior they did not earn. v6's rule: **the numerical surface informs the prose and the analyst; the structural score is earned through evidence and counter-hypotheses**. The surface is a derived deep-dive-layer artifact the ST-12 specialist reads as context; the structural score lives on the ST-12 trace. They never collapse into one number, and the surface never gets a privileged place in the scoring path.

The numerical layer's most important output is not the model itself; it is the list of variables that must work for the thesis to hold. Those variables become watch-items for the Forecaster, scoring items for the Evaluator, and the `expected_evidence` field on the financial-quality traces these agents write.

§10 The forward chapters, reachable states not just TAM

Three of the company-facing chapter agents ask a different question than the rest. The qualitative chapters (§08) ask *what is this company*; the numerical chapters (§09) ask *what will it earn*. The forward chapters — **ST-9 opportunity**, **ST-10 dynamics**, and **ST-14 endgame** — ask *what states can this company reach, with what probability, and through what path*. They are chapter agents like any other (§08), not a separate Synthetic Futures tier; what unites them is the forward-looking question, not a cluster boundary.

The right frame is assembly-theoretic rather than addressable-market-theoretic. Instead of estimating a static TAM and multiplying by share, the forward chapters enumerate the adjacent reachable states (new products, new markets, new business models, competitive responses, constraint shifts) and attach probability weights and path dependencies. The output is a tree, not a number.

- **ST-9 opportunity (strategic direction of travel)**. Expanding addressable market, defending position, or competing in shrinking space. Tested through case comparisons and direct checks: change in demand drivers, direction-of-travel.
- **ST-10 dynamics (paradigm shifts and inflection points)**. Expanding opportunity spaces, fat-tailed outcomes, adoption inflection, ROE turning points, escape-velocity readiness: change in inflection, scale readiness.
- **ST-14 endgame (where the industry resolves)**. Winner-takes-all, winner-takes-most, scope-widening, bounded, average, displaced — which path, why, and what probability weight this company carries on each: endgame class, path probability.

Price-path evidence (what the price path already implies — log-growth path intact, broken, accelerating or reverting; drawdowns, volatility, relative strength, volume, recoveries) is not a chapter of its own. It is routed into the forward chapters and into Threadweave as one source type among many; reliability and diagnosticity calibration apply to it the same as for any other source.

Why the forward question is kept distinct

The forward chapters are about the company as it *could become*; the qualitative chapters are about the company as it *stands*. The two are deliberately decoupled so that a qualitative observation (e.g. "product culture has sharpened") can be evaluated against futures independently of whether the forward chapters agree the opportunity

space is widening. Collapsing the two questions into one conflates two different epistemic operations and makes contradictions invisible — which is exactly the kind of contradiction the CrossValidator exists to surface.

Output shape

The futures sub-model is a weighted tree of reachable states, each with a path description, a probability weight, the key uncertainty nodes along the path, the constraint shifts required, and the expected competitive response. Observations are updates to this tree (new branches added, probabilities shifted, paths pruned). Each tree restructuring writes one `trace_snapshot` per affected trace, with `trace_type='structural_future'` for the reachable-state traces.

Two scopes: entity-bound reachable states, and structural futures

The forward chapters produce output at two distinct scopes, and the distinction is architecturally meaningful. **Entity-bound reachable states** describe what configurations a specific company could be in, one to five years out, given its current structure, resources, and trajectory; they are written as observations in the entity's substrate. **Structural futures** describe states of the world that will exist regardless of which entity occupies them. The two scopes are complementary: entity-bound reachable states feed Threadweave's forward direction (where can this entity go); structural futures feed Threadweave's reverse direction (what would have to be true for this entity to occupy a future position the structural future predicts will exist). The forward chapters are the only agents that produce both, and the architectural commitment is that structural-future output is first-class, not a byproduct of entity-bound analysis.

Why structural claims are often more confident than identity claims: "vertical-AI radiology will have a dominant player by 2032 with approximately \$20B revenue" is a structural claim grounded in demand curves, capability requirements, and historical patterns of market formation. "Company A will be that player" is an identity claim that depends on Company A's specific decisions, competitor responses, regulatory accidents, and timing. Most quant systems conflate these or skip the structural layer entirely. The forward chapters separate them by design.

§11 CrossValidator, formal contradiction detection

CrossValidator is a rules engine, not an agent. It does not propose trace updates of its own. It sees the proposed trace updates and `trace_snapshots` from all base chapter agents for a given orchestration window and runs a narrow set of typed checks. Most of these checks are deterministic schema-level invariants (coherence, dependence, adjacency, preservation, probability conservation) expressible as SQL or typed code rather than as LLM judgment. Genuinely ambiguous coherence cases escalate to the Orchestrator's critique pass (§14), where LLM reasoning legitimately belongs. CrossValidator's value is being fast, cheap, and consistent on the 95% of contradictions that are mechanically detectable; the remaining 5% deserves an agent, not a rule.

Check/01 · Coherence Does the insight model support the financial? If Deep Insight updates a moat trace upward, the Financial model should show (or be on the path to show) trace updates consistent with improving unit economics. If it does not, the trace updates are flagged as incoherent, not averaged.

Check/02 · Dependence Does the future require the present? If a Synthetic Futures trace update requires a capability or position the Deep Insight layer has not asserted in any active trace, the future trace is flagged as unsupported. An unsupported future trace is not rejected; it is logged with status='candidate', not promoted.

Check/03 · Adjacency Do untouched traces remain consistent? A trace update on the Insight layer about customer behaviour may have implications for a Financial trace that was not touched in the same window. CrossValidator checks whether adjacent, untouched traces remain consistent with the committed updates.

Check/04 · Preservation What priors would now be violated? If committing the trace update would move a high-confidence trace past a configured ceiling tied to its source reliability, the orchestrator is notified and must either downgrade the prior (with reasoning logged) or reject the proposed update.

Check/05 · Probability conservation across counter-hypotheses (new in v5) For any trace whose `counter_hypotheses` distribution was updated by the current orchestration window, three sub-checks run:

1. Sum-to-one within tolerance. The posterior probabilities across the counter-hypothesis set must sum to 1 within a small numerical tolerance.
2. Evidence-bounded ceilings. No single counter-hypothesis can move past a configured ceiling tied to evidence reliability. A single tier-3 source cannot drive a counter-hypothesis from prior 0.10 to posterior 0.85.
3. Dominant-shift support. The dominant shift (which counter-hypothesis gained the most mass) must be supported by the diagnosticity recorded for the originating evidence (§15). A dominant shift toward a counter-hypothesis the recorded evidence is not diagnostic for is flagged.

Violations are logged as unresolved disagreements, the same treatment as the four existing checks. The check is mostly deterministic and runs as SQL or typed code, consistent with CrossValidator’s philosophy.

Check/06 · Canonical chapter consistency (new in v5.1) When a canonical chapter (§06a) is committed in re-author mode, Check/06 validates the structured outline against the `trace_snapshots` it cites. Five sub-checks run for each `outline_entry` whose `trace_snapshot_id` is non-null:

1. Snapshot recency. If the cited snapshot is not the most recent active snapshot on the cited trace, flag the entry as `stale_citation`.
2. Direction alignment. Compare `claim_direction` against the snapshot’s posterior trajectory (computed from the snapshot and its predecessor). Mismatch flags `direction_inconsistent`.
3. Magnitude band alignment. Compare `magnitude_band` against the snapshot’s magnitude field, mapped through the registry-defined band table. Mismatch flags `magnitude_mismatch`.
4. Confidence band alignment. Compare `confidence_band` against the snapshot’s `posterior_probability`, mapped through the standard band table. Mismatch flags `confidence_mismatch`.

5. Citation chain integrity. Verify that every `evidence_atom_id` cited is present in the snapshot’s `evidence_atom_ids` array, or in the array of any of the snapshot’s chronologically prior snapshots on the same trace. Citations that do not appear in the trace’s evidence history flag `unsupported_citation`.

Any flag is blocking. The chapter commit is rejected; the typed mismatch list is returned to the harness; the activity is retried with a regenerated prompt that includes the mismatches as binding context. After three failed retries, the re-author attempt is logged as failed and the prior chapter version remains active. In seed mode, Check/06 instead validates the proposed traces’ counter-hypothesis structure (rule 6 of §06) and materiality (rule 5 of §06); the direction/magnitude/confidence comparisons do not run because there is no prior snapshot to compare against. Full specification of the structured outline schema and Check/06 semantics is in §06d.

What CrossValidator writes Contradictions and unresolved disagreements only. No semantic state. It does not average. It does not vote. It does not propose traces. A disagreement surfaced by CrossValidator lives as an unresolved disagreement until either new evidence resolves it or the orchestrator makes an explicit, logged call to commit one side and mark the other as minority view.

Disagreement is a feature. It is information about the limit of current understanding, and it is lost forever the moment the system averages the two readings together.

CrossValidator corresponds roughly to v1's SA-12 Multi-Model Validation, promoted to its own layer because the base chapter agents have distinct operational character and it is no longer acceptable for their contradictions to surface only in the orchestrator's critique pass. v5 extension: Check/05 enforces the rule of §06 that every probability is computed against an explicit counter-hypothesis set, and the ceilings tied to source reliability prevent the substrate from accepting posteriors no source could legitimately produce. v5.1 extension: Check/06 binds canonical chapter prose to the trace graph by ID, so the audit trail extends through the prose; without it, an LLM-authored chapter could drift from the substrate while remaining stylistically plausible.

§12 Dot Connector / Insight Engine, cross-trace synthesis

The Dot Connector is the layer that connects the dots. It reads across every layer below it (Deep Insight, Financial Model, Synthetic Futures, CrossValidator, the full `atomic_observation` stream, the full `trace_snapshot` history, the active and dormant trace graph, the prediction ledger) and synthesises insights that no single layer could produce alone.

The name is literal. An individual `atomic_observation` is a point. An individual `trace_snapshot` is a point. A pattern across multiple snapshots on multiple traces (a moat trace weakening, a fragility trace strengthening, an opportunity-space trace contracting, all on the same entity within a 30-day window) is a shape. A pattern across the same `trace_type` across many peer entities (rising competitive-pressure traces across an entire sector within a month) is also a shape. The Dot Connector's job is to see the shape.

Cross-trace, not observation-by-observation An earlier draft of this architecture had a synthesiser firing every time an observation landed, converting each observation into a hypothesis. That is not what this layer is. An observation-by-observation hypothesis generator would be a transformation pipeline; the Dot Connector is a synthesiser. It runs on a schedule (nightly for active entities, weekly for quieter ones) and its inputs are the full substrate at the time of the sweep, not a single incoming event.

The practical consequence: if three `trace_snapshots` land on three different traces on the same day and each is individually unremarkable, the Dot Connector sees all three together on the next sweep and may recognise a pattern none of them implies alone. If they do not form a pattern, nothing is emitted, and that is also a valid outcome.

Trace patterns as first-class input The Dot Connector reads the `trace_snapshot` stream alongside the `atomic_observation` stream. A pattern across multiple `trace_snapshots` becomes available even when the underlying observations look diverse. Three examples:

1. Cross-trace coherence on one entity. A moat trace's posterior falls, an over-optimisation trace's posterior rises, a capital-efficiency trace's posterior falls. Across one entity, this is a coherent thesis-level shift. Across three peer entities in the same sector within a month, it is a sector pattern.
2. Counter-hypothesis regime shift. "Rising competitive pressure" gains posterior mass across many entities' moat traces in the same sector during the same window. A single update is noise; many updates is a regime signal.
3. Diagnosticity drift. A diagnosticity claim that was historically reliable starts proving over-claimed across many recent trace updates. The drift itself is an insight: the kinds of evidence the system relied on are losing their signal-to-noise.

These patterns are written into the substrate as `trace_relationships` rows (one of the twelve source-of-truth tables, §21) linking the contributing traces. When forecastable, the pattern spawns a prediction with falsifier and horizon, citing the `trace_relationship` row as its evidence chain.

Trace_relationships are the primary output The Dot Connector writes `trace_relationships`, not direct trace updates. A `trace_relationship` is a structured statement linking two or more traces with a relationship type (`coherent` , `contradictory` , `causal_candidate` , `co-occurring` , `dominated_by` , `dominates`) and an evidence chain. Some relationships are forecastable (a `dominated_by` link with a horizon implies the dominated trace’s posterior should fall within the horizon); others are not.

Most valuable patterns are not forecastable in a clean sense. “Capital allocation has quietly shifted toward defensive adjacencies over six quarters” is a relationship the Dot Connector might write between an entity’s capital-allocation trace and its strategic-positioning trace; it matters for a long-horizon view and may never produce a short-horizon scoreable claim. Forcing every relationship to become a prediction would either invent falsifiers that misrepresent the claim, or suppress the relationship entirely. Neither is acceptable.

What the Dot Connector produces Trace_relationship row. A typed link between two or more traces with a relationship_type, an evidence chain, and a confidence.

Layer evidence weights. Which layers contributed, and with what weight. A relationship that relies only on the Financial layer is less robust than one that converges across Financial, Deep Insight, and Synthetic Futures.

Evidence chain. The sequence of `trace_snapshots` and `atomic_observations` that, taken together, produce this relationship. Each link is a pointer, not a paraphrase, so the chain can be reconstructed from the ledger.

Forecastability flag. Whether the relationship can be converted into a prediction with a falsifier and a bounded horizon. If true, a prediction is emitted citing the relationship.

Relation to CrossValidator CrossValidator is a within-sweep gatekeeper: every trace update passes through it before committing, to catch contradictions. The Dot Connector is an across-sweep synthesiser: it reads the full substrate (including CrossValidator’s unresolved disagreements) and looks for patterns that those very disagreements might reveal. A persistent disagreement between Deep Insight and Financial trace updates over several weeks is itself a signal, and the Dot Connector can name it as a contradictory `trace_relationship`.

Relation to Threadweave Threadweave reads `trace_relationships`, not the raw observation stream. A relationship that names a durable pattern shapes Threadweave’s chapter projections; a relationship with a confirmed prediction feeds Threadweave’s reliability score for the relevant horizon tier. When Threadweave needs something the `trace_relationships` ledger does not cover, it falls back to the base models, but that fallback is the exception, not the rule.

Most systems that track changes stop at the change. StahlTrace connects the changes, on a schedule, and writes what it sees. v5 connects them across traces, across entities, and across time, on the same schedule, and stores those connections in their own ledger.

§13 Threadweave, projecting traces into chapters

Threadweave is the top layer. It projects the trace graph forward through opportunity space, pulling from any of the layers below as the question requires, and it assembles the customer-facing `chapter_projections` read

surface from the entity's promoted traces. Threadweave reads typed substrate — trace posteriors, magnitudes, persistences, and the `trace_relationships` graph — not prose; the canonical chapters (§06a) and Threadweave operate on the same trace graph from different angles. Threadweave is post-launch in the §24 phasing: it requires scored judgment across at least two horizon tiers to be meaningful, and that data does not exist at launch.

v6 makes no change to Threadweave. It is core end-state machinery, specified in v5.1 and carried forward unchanged. The v4.1 reference implementation is a single-shot trial, not a longitudinal projection layer, and it has nothing to teach Threadweave that Threadweave does not already specify. The one question the v4.1 analysis raised against this section — how a chapter-level score is derived from trace posteriors for a customer who asks "how does the system rate Company X's moat" — is a real open question, but it is a strategic one, and v6 flags it as open (D/08, §25) rather than answering it with v4.1-derived machinery. Whatever roll-up rule eventually answers it belongs to the end-state design, decided on its own merits, not imported from a trial.

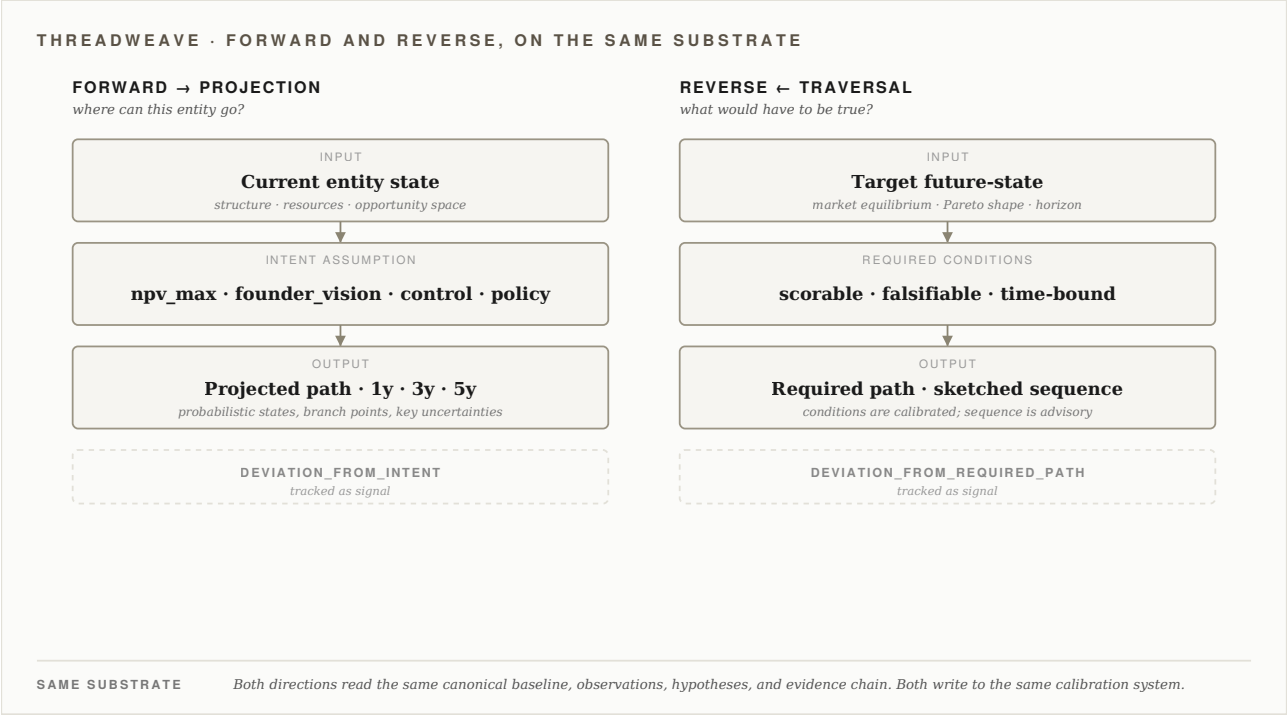


FIGURE 7 · THREADWEAVE BIDIRECTIONAL PROJECTION, FORWARD UNDER INTENT AND REVERSE FROM TARGET FUTURE-STATE.

§14 Support agents, and the per-chapter supervisor steer AMENDED

Four support agents work alongside the layer stack: the **Resolver** (identity resolution, attaching evidence to canonical entities), the **Orchestrator** (the 14-step sequencer), the **Forecaster** (prediction emission from material trace snapshots), and the **Evaluator** (outcome scoring and calibration). v5.1 described the Orchestrator as a dispatcher. v6 amends that description with a capability v4.1 demonstrated: the Orchestrator does not merely dispatch; it produces a **typed per-chapter supervisor steer**.

The supervisor steer

In v4.1, a dedicated top-level-node agent read the source bundle and wrote one to three sentences of emphasis for each of the fourteen chapters — a `for_chapter_agents` block with one entry per ST chapter, schema-required to be fully populated. The chapter specialist read its own key as a Phase-0 input before beginning its work. In the live trial these steers were entity-specific and substantive: for the fragility chapter, the steer named the dominant load-bearing risks (a preferred-stock overhang and active litigation) and instructed the specialist to assess each against the specific terms and accruals in the bundle. The steer is not a score and does not bind the specialist's judgment; it is a focusing instruction that ensures the most expensive cognitive work is spent on the chapter's actual load-bearing questions rather than rediscovered from scratch.

v6 makes this a typed Orchestrator output. During the cold-start phase (§17a) the Orchestrator emits a supervisor steer per chapter as a run-scoped top-level-node artifact (recorded in the `run_log` audit substrate, not a new source-of-truth table); each chapter specialist consumes its key at the start of its work. In steady state, the steer is refreshed when re-author is triggered, scoped to the chapters the accumulating evidence touches. The artifact shape mirrors v4.1's `for_chapter_agents`: a structured field per chapter, populated by the Orchestrator, read by the downstream specialist.

§15 Scoring methodology, what “scored by reality” actually means

Internal posterior probability	External confidence band
≥ 0.85	High
0.65 to 0.85	Medium-high
0.40 to 0.65	Medium
0.20 to 0.40	Medium-low
< 0.20	Low

	mechanism_confirmed	mechanism_unconfirmed	mechanism_contradicted
outcome_confirmed	Full credit. Reasoning matched reality.	Partial credit. Outcome confirmed, mechanism silent.	Penalised. Right answer, wrong reason.
outcome_falsified	Rare / noted. Falsifier triggered but mechanism operated as stated; a stronger force overrode it.	Penalised. Standard wrong-prediction case.	Penalised ×2. Wrong outcome and contradicted mechanism.

PREDICTION TYPES · HOW EACH IS SCORED

Prediction type	A typical claim	Scoring rule	Resolution
BINARY DIRECTION type 1 of 4 · confidence $\in [0,1]$	<i>"Next earnings will revise guidance downward."</i>	Brier score $(p - o)^2$ · lower is better	Outcome is true or false
DISCRETE PROBABILISTIC type 2 of 4 · vector summing to 1	<i>"Of {beats, meets, misses} the probability is {0.40, 0.45, 0.15}."</i>	Multi-class Brier + log-score · both per agent	Outcome is one of a finite set
CONTINUOUS type 3 of 4 · distribution, not point	<i>"5y operating cashflow CAGR, central 9.2%, 80% interval [6.5%, 12.1%]."</i>	CRPS · Continuous Ranked Probability Score	Outcome is a real number
STRUCTURED type 4 of 4 · three components, falsifier	<i>"Margin compression of ~150bps within 2 quarters, driven by supplier shock."</i>	Decomposed: direction · mechanism · horizon · magnitude	Resolves into 4 components, scored separately

FIGURE 8 · FOUR PREDICTION TYPES, EACH WITH ITS OWN SCORING RULE AND RESOLUTION CRITERION. THE STRUCTURED TYPE CARRIES A FALSIFIER AND RESOLVES INTO FOUR COMPONENTS SCORED SEPARATELY.

The Evaluator in §14 is described as the agent that matches outcomes to predictions and writes calibration. Both descriptions hide the actual mechanics. This section specifies what scoring rule applies to which prediction type, how partial resolution is handled, what happens to predictions whose outcomes never arrive, how mechanism attribution distinguishes right-answer-for-wrong-reason cases, and how trace-snapshot quality is scored on three tracks (posterior, magnitude, persistence) alongside prediction outcome.

	mechanism_confirmed	mechanism_unconfirmed	mechanism_contradicted
outcome_confirmed	Full credit. Reasoning matched reality.	Partial credit. Outcome confirmed, mechanism silent.	Penalised. Right answer, wrong reason.
outcome_falsified	Rare / noted. Falsifier triggered but mechanism operated as stated; a stronger force overrode it.	Penalised. Standard wrong-prediction case.	Penalised $\times 2$. Wrong outcome and contradicted mechanism.

This is the section that turns the architecture's central commercial claim, scored by reality, from rhetoric into engineering. A reader who finds the rest of the architecture rigorous but discovers "Brier scores" as the entire treatment of how reasoning is evaluated will discount everything else. The methodology has to be specified and defended.

Prediction types, a taxonomy Predictions in the system are not all the same shape. Four types, each scored differently.

Binary direction. "The next earnings release will revise guidance downward." Outcome is true or false. Confidence is a single probability in $[0, 1]$. Scoring rule: Brier score $(p - o)^2$, lower is better.

Discrete probabilistic. "Of {beats, meets, misses} consensus next quarter, the probability is {0.40, 0.45, 0.15}." Outcome is one of a finite set. Confidence is a probability vector summing to 1. Scoring rule: multi-class Brier (sum-of-squared-errors over the probability vector against the one-hot outcome) and log-score ($-\log p_{\text{realised}}$),

both reported. Log-score is more punishing for confident wrong answers; Brier is more familiar. Both per-agent so users with different loss preferences can read calibration their own way.

Continuous. “5y operating cashflow CAGR, central estimate 9.2% with 80% interval [6.5%, 12.1%].” Outcome is a real number; confidence is a distribution. Scoring rule: Continuous Ranked Probability Score (CRPS), the integral of the squared difference between the predicted CDF and the empirical CDF (a step function at the realised value). CRPS reduces to mean absolute error for point predictions and to Brier for binary outcomes; the right family for this taxonomy.

Structured (mechanism + magnitude + horizon). The form most Dot Connector hypotheses take: a directional claim, a mechanism, a falsifier, and a bounded horizon. Resolution is decomposed: did direction prove right, did mechanism prove right, did horizon prove right, was magnitude in range. Each component scores separately. Aggregate “structured score” is the geometric mean of the components, which weights all four equally and penalises a single failure proportionally.

Partial resolution, when an outcome triggers some but not all of a prediction Structured predictions resolve component-by-component. If an outcome confirms direction but contradicts mechanism, the prediction is recorded as `resolved_partial` with the specific decomposition: `direction=confirmed`, `mechanism=falsified`, `horizon=confirmed`, `magnitude=in_range`. Calibration aggregates each component separately; a partial resolution does not collapse to either “resolved” or “unresolved” but contributes to four distinct calibration tracks.

The architectural commitment: partial resolution is the typical case, not the exception. Predictions about complex systems rarely come true in exactly the form they were written. Forcing every outcome into a binary verdict throws away the information that distinguishes a system that is wrong about mechanism from one that is wrong about direction. The decomposition is the calibration signal.

Unresolvable predictions, when the question becomes moot Some predictions never resolve cleanly. The entity is acquired before the horizon expires. The market regime shifts so the original mechanism no longer applies. The prediction’s specific falsifier is rendered un-checkable by an exogenous event. Three handlings:

Marked unresolvable. Flagged with a recorded reason. Excluded from calibration aggregates so as not to penalise agents for outcomes that were never their question. Counted in a separate `resolution_rate` metric.

Resolution rate as a signal. An agent whose predictions are systematically unresolvable is producing predictions that are not actually predictions, they are observations dressed up as falsifiable claims. Resolution rate is a per-agent diagnostic that surfaces this. A low resolution rate triggers prompt review, not silent calibration drift.

The “non-decision” trap. An adversarial implementation could mark every wrong prediction as unresolvable to keep calibration looking clean. The architectural mitigation: the unresolvable flag can only be set by the Evaluator on a defined trigger (acquisition event, regime-shift event explicitly logged, falsifier becoming structurally un-checkable), never by the predicting agent itself. The unresolvable rate is a published per-agent metric subject to review.

A note on horizon vocabulary: the schema enum `horizon_tier` uses three values (short, mid, long) mapping to nominal horizons of approximately 1y, 3y, and 5y respectively. Threadweave projects at the literal year horizons; calibration aggregates at tier granularity. The mapping is approximate, not strict: a 14-month prediction is short-tier, a 4-year projection is long-tier.

Calibration aggregation Calibration is computed per (source, agent, horizon-tier) cell and per (source, agent, horizon-tier, `trace_type`). Published in three forms:

Time-weighted mean score. The headline number. Brier for binary, CRPS for continuous, geometric mean of components for structured. Time-weighted with exponential decay (recent predictions count more) so an agent's calibration tracks its current state, not its full history. Decay half-life is configured per agent and per horizon tier; conservative defaults at launch (6-month half-life for short-horizon, 18-month for mid, 36-month for long).

Reliability diagram. The canonical presentation. Predicted probabilities binned, realised frequencies plotted against bin midpoints. Perfect calibration is the diagonal. Deviations from the diagonal show where an agent is over- or under-confident at specific probability ranges.

Confidence intervals on the calibration estimate. A calibration score with no uncertainty band conflates a hot streak with genuine skill. Bootstrap confidence intervals are published on every calibration metric. An agent with 12 resolved predictions and a 0.18 mean Brier might have a 95% CI of [0.08, 0.31]; that is a different signal from an agent with 2,400 predictions and the same mean. Users see the CI alongside the point estimate.

Outcome-to-prediction matching An implementation note: the Evaluator's outcome verdict is implemented as deterministic Python. Falsifier matching, Brier/CRPS computation, decomposed component scoring, and verdict writing are pure code with no LLM call. The mechanism verdict (described next) retains the LLM ensemble. The split is architectural: outcome is operations on numbers; mechanism is judgment over evidence chains.

When an outcome arrives, the Evaluator runs a matching pass that resolves all open predictions whose falsifier the outcome triggers:

1. The outcome is recorded with its observed values (direction, magnitude, mechanism if attributable, timestamp).
2. The Evaluator queries open predictions for the entity within the outcome's relevant horizon window.
3. For each open prediction, the falsifier is evaluated against the outcome. The falsifier is a typed condition; matching is deterministic and re-runnable.
4. Predictions whose falsifier matches are scored according to type and recorded with verdict, components (for structured), and the outcome that resolved them.
5. Predictions whose falsifier does not match remain open. They are not penalised for non-resolution; they remain pending until their horizon expires or another outcome resolves them.
6. Predictions whose horizon expires without resolution are marked `resolved_horizon_expired` and scored conservatively (the falsifier was about something specific, the horizon passed, the absence of the falsifying event is itself a kind of resolution).

One outcome can resolve multiple predictions; one prediction stays open until its specific falsifier is triggered. The system makes specific claims with specific falsifiers, and resolution is per-falsifier, not per-outcome.

Mechanism attribution, right answer wrong reason

MECHANISM VERDICT →

Mechanism attribution scoring is independent of outcome scoring; both verdicts coexist on every prediction. The "right answer, wrong reason" cell is the architecture's primary defense against accidental calibration drift.

MECHANISM ATTRIBUTION MATRIX · OUTCOME VERDICT × MECHANISM VERDICT			
	Mechanism confirmed	Mechanism refuted	Mechanism unobserved
Outcome correct	Right for the right reason — full credit, source weight up	Right for the wrong reason — outcome credit, mechanism penalty	Right, mechanism untested — partial credit
Outcome wrong	Wrong despite mechanism — model error, recalibrate	Wrong, mechanism refuted — clean miss, source weight down	Wrong, mechanism untested — diagnosticity penalty

FIGURE 9 · MECHANISM ATTRIBUTION MATRIX, OUTCOME VERDICT × MECHANISM VERDICT, WITH CALIBRATION TREATMENT PER CELL. THE "RIGHT ANSWER, WRONG REASON" DIAGONAL IS WHAT MAKES MECHANISM ATTRIBUTION WORTH THE ENGINEERING COST.

Outcome scoring tells you whether the prediction matched reality. It does not tell you whether the prediction's reasoning matched reality. A prediction can be confirmed by an outcome that arrived for entirely different reasons than the prediction stated. A prediction can be falsified by an outcome that arrived despite the predicted mechanism actually operating, because some other factor dominated. Calibration that ignores this distinction trains on noise.

Every resolved prediction therefore receives a `mechanism_check` verdict in addition to its outcome verdict. Three states:

`mechanism_confirmed` . The outcome arrived and the stated mechanism is supported by the evidence chain between emission and resolution. The agent reasoned correctly. The case calibration should reward.

`mechanism_unconfirmed` . The outcome arrived but the evidence chain is silent on the stated mechanism. The mechanism may have operated, may not have, the available evidence does not say. Soft signal: not a failure, not a clean win. Calibration treats it as partial credit.

`mechanism_contradicted` . The outcome arrived but the evidence chain shows the stated mechanism did not operate, or operated in the opposite direction, or was overwhelmed by a different factor that drove the outcome. Right-answer-for-wrong-reason. The prediction is recorded as outcome-correct but mechanism-failed; calibration penalises it commensurately.

The Evaluator runs the mechanism check as a second pass after the outcome verdict is recorded.

The 2×3 verdict matrix

Mechanism confirmation rate For each (entity, agent, horizon-tier) cell, calibration aggregates publish a new metric alongside Brier/CRPS/ decomposed-score: `mechanism_confirmation_rate` , the fraction of outcome-confirmed predictions whose mechanism was also confirmed. An agent with high outcome calibration but low mechanism confirmation rate is silently overfitting to noise; the gap is itself a diagnostic. A persistent gap triggers prompt review, not silent calibration drift.

The reverse case (high mechanism confirmation but lower outcome calibration) indicates an agent that reasons well but emits predictions on questions where the mechanism is operating but is being overwhelmed by exogenous factors. That is a different kind of agent failure (selection of question, not quality of reasoning) and is treated separately.

Mechanism attribution in practice Mechanism attribution is a load-bearing architectural commitment, and the most likely place where the discipline could turn out aspirational rather than enforced. The honest concern: financial outcomes are over-determined. A stock moves; was it the predicted mechanism, an unrelated FX move, an inventory write-down, a peer-group rotation? The Evaluator has to make a call, and that call is genuinely hard.

Five architectural commitments make this operationally tractable rather than aspirational.

Mechanism specificity gates upstream. Predictions whose stated mechanism is too vague to verify are rejected before emission. A new validator (lightweight, prompt-based, runs as a Forecaster post-check) examines whether the mechanism names specific evidence patterns that would constitute confirmation or contradiction. Vague mechanisms like “demand softness” are rejected with a refinement request; specific mechanisms like “supplier X cost increase of Y% propagating to gross margin Z bps within W months, observable in cost-of-revenue line item” are accepted. Most of the downstream attribution problem dissolves when the mechanism is specified well; the upstream gate is the highest-leverage intervention.

Pre-stated falsification evidence requirements. Every prediction, on emission, names the specific evidence patterns in the resolution window that would confirm the mechanism, contradict it, or be insufficient to judge. Not abstract: which line items, which document types, which thresholds. The prediction record stores this as a typed field (`mechanism_evidence_requirements`). The Evaluator’s job becomes mechanical, check whether the named evidence appeared in the resolution window, rather than interpretive. The Evaluator can still report “evidence was insufficient to judge,” which is a legitimate verdict, but it cannot rationalise an answer the evidence does not support.

Independent Evaluator architecture. The Evaluator never reads the agent’s reasoning chain that produced the prediction. It reads only three inputs: the prediction text, the pre-stated `mechanism_evidence_requirements` , and the resolution-window evidence. The most important commitment in the section. Without independence, mechanism attribution collapses into self-confirmation: the same chain of reasoning that produced the prediction has motivated reasoning to confirm it. The Evaluator is a separate Temporal activity with no access to the originating agent’s run-log.

Ensemble verdicts with disagreement-as-signal. Mechanism verdicts run as ensembles: multiple Evaluator invocations with different prompt seeds, different evidence-window scopes (narrow vs wide), different model versions. The architecture treats Evaluator disagreement as informative rather than as something to average away. If three of five runs say confirmed and two say unconfirmed , the verdict is unconfirmed with a noted ambiguity flag, not confirmed with confidence 0.6. The architecture commits to noisy mechanisms being honestly noisy in the substrate.

Unconfirmed as the default expectation. The architecture commits to `mechanism_unconfirmed` being the most common verdict, not the rare exception. Most predictions resolve outcome-confirmed-mechanism-unconfirmed because evidence is genuinely silent on attribution: the outcome happened, multiple plausible mechanisms could have caused it, and the evidence chain in the resolution window does not discriminate. confirmed and contradicted are the discrimination signals; unconfirmed is the honest default. Calibration metrics report all three states with their relative frequencies, not just the binary outcome verdict.

A worked Evaluator example To make this concrete, walk through one prediction from emission to verdict.

Setup. Mid-cap European industrial supplier, `entity_id` EX-MID-IND-1. Three `trace_snapshots` commit on 2026-Q1: a Deep Insight snapshot updating T-2104 (supplier-exposure moat), a Financial Model snapshot

updating T-2110 (margin-compression capital_allocation), and a Synthetic Futures snapshot updating T-2117 (the entity's market cone narrowing). The Forecaster proposes a prediction.

Prediction emitted (T+1h).

Specificity gate (T+1h). The mechanism specifies a named supplier, a quantified pass-through ratio, observable line items, and a threshold for confirmation. The validator accepts.

Resolution window (T+90d). Q3 reports land. Reported gross margin compresses 142 bps versus Q2, inside the predicted band. Outcome verdict: confirmed .

Evaluator runs (T+90d + 4h). Five Evaluator invocations dispatch in parallel. Each receives only the prediction text, the pre-stated mechanism evidence requirements, and the Q3 resolution-window evidence (the 10-Q filing, earnings call transcript, and any news in the window). None receives the originating agent reasoning.

Evidence the Evaluators examine.

Cost of revenue / revenue ratio QoQ change. Pre-stated requirement: ≥ 150 bps rise. Observed: +163 bps. Met.

Management commentary on supplier-cost pressure. Pre-stated requirement: explicit reference as material driver in Q3 earnings call. Observed: CFO acknowledged "input cost pressures, particularly from our raw material supply base," without naming the specific supplier. Partially met, ambiguous.

Confounding factors. Three of the Evaluators flagged a Q3 inventory write-down of €18M, which would also push cost-of-revenue higher independently of supplier costs. The write-down is disclosed but not separately reconcilable from the supplier-cost effect.

Ensemble verdicts. Two Evaluators returned confirmed (line-item evidence is consistent with the predicted mechanism, management commentary directionally supports it). Three returned unconfirmed (the inventory write-down confound prevents clean attribution; the management commentary did not specifically name the supplier the prediction identified).

Final verdict. mechanism_unconfirmed with ambiguity flag. The outcome was correct; the mechanism is plausibly operative but evidence does not discriminate between the predicted mechanism and the inventory-write-down confound. The Forecaster's mechanism-confirmation rate at the 90-day horizon receives no credit for this prediction; the agent gets calibration credit for the outcome verdict only.

What the substrate records. The full Evaluator ensemble (five verdicts, the evidence each examined, the disagreement structure) is written as a single resolution record. Future prompt review of this Forecaster will see "right answer, ambiguous mechanism" as a recurring pattern, not a confident confirmation. Cross-entity mechanism patterns read this record alongside others to surface "agents tend to call this kind of mechanism right when an inventory write-down is also present," itself an insight the Dot Connector can act on.

The mechanism is not "confirmed" by majority vote. It is "unconfirmed" honestly because the evidence chain does not discriminate. The architecture's value is in being honest about ambiguity, not in producing confident verdicts.

Cross-entity mechanism patterns When mechanism_contradicted verdicts cluster across entities, agents, and time, the cluster itself is signal. The Dot Connector reads the mechanism-check ledger during its scheduled sweep alongside the canonical substrate. A pattern of "agents in this layer kept being right for the wrong reason about this kind of entity over this window" is exactly the cross-trace pattern the Dot Connector exists to surface. The

output is a `trace_relationship` row that names the recurring reasoning failure, not a causal claim about the world.

The factor co-occurrence ledger Beyond mechanism attribution at the per-prediction level, the substrate maintains a separate ledger of factors observed when reasoning about each entity. Each `trace_snapshot` or `trace_relationship` that explicitly invokes a factor (a regulatory regime, a competitor action, a macro variable, a structural condition) records it in the factor co-occurrence ledger, joined to the entity, the date, the originating trace, and (when resolved) the outcome it preceded.

The ledger is a frequency record, not a causal claim. It does not assert that factor F causes outcome O; it records that the system observed F preceding O at a given rate over a given history. The distinction matters: causal claims require assumptions the data cannot support, but frequency records support the kind of pattern detection that generates useful hypotheses without overcommitting to causal structure.

The Dot Connector reads the factor ledger during sweeps. When a factor's co-occurrence rate with a particular outcome class shifts meaningfully, either because a previously rare factor is becoming common, or because a previously reliable co-occurrence is breaking down, that shift is a candidate insight. The factor taxonomy itself grows over time as agents introduce new factors; the ledger does not require a closed taxonomy.

Why this is not a causal model A reader familiar with causal-inference literature might recognise the shape of what is being described and ask why the architecture stops short of explicit causal-graph modelling: directed acyclic graphs of factor influences with learned edge weights, do-calculus over interventions, structural causal models. The reasons are deliberate.

Causal claims about company dynamics in finance have a poor historical track record. Most published causal models do not replicate. The graveyard of failed strategies is full of beautiful causal graphs that turned out to be overfitting. Committing the architecture to causal claims the data cannot defend would be the first thing in this document a sophisticated reader could legitimately call hand-wavy.

Markov assumptions are wrong for the dynamics this system tracks. Path dependence is the violation of the Markov property, and path dependence is the architecture's central premise (§01, §03). A literal Markov model is the wrong mathematical object.

Mechanism attribution and factor co-occurrence are strictly weaker claims than causal modelling, and weaker claims are more defensible. Both produce useful diagnostic signals (right-for-wrong-reasons detection, co-occurrence drift detection) without committing to causal structure the data cannot yet support. They are also strictly compatible with adding causal modelling later: the substrate they require is a strict subset of what a future causal-graph layer would need.

The roadmap to causal modelling is captured in the open questions of §25; the present commitment is to the narrower, defensible primitives. This is the line the system holds at v4.

Threadweave reliability function For each entity and each horizon tier, the Threadweave reliability score is a function of three inputs:

1. The mean calibration score for the entity at that tier (lower is better).
2. The number of resolved predictions feeding that calibration (more is better, with diminishing returns above approximately 50).
3. The width of the calibration confidence interval (narrower is better, reflects data sufficiency).

The function combines these into a reliability score in [0, 1]. The exact functional form is configured per launch and recalibrated as data accrues; it does not depend on prior assumption, only on scored data. When data is

insufficient for a tier (fewer than approximately 10 resolved predictions), reliability falls back to “insufficient” rather than computing a fragile estimate. Threadweave consumers see “insufficient” explicitly, not a misleadingly high or low number.

Trace-snapshot quality scoring v5 makes the trace-snapshot the unit being scored. Three parallel scoring tracks run alongside Brier/CRPS/ decomposed-structured/mechanism-attribution.

Track 1 • Posterior calibration. For each trace, posterior probability at time *t* is later resolved against the eventual reality of the trace’s hypothesis, or against the future evidence the trace’s expected_evidence made claims about. Resolution uses Brier or CRPS depending on whether the trace’s hypothesis is binary or distributional. Traces accumulate calibration curves the same way sources and agents do. What this scores: was the system’s confidence in this trace well-calibrated against later reality?

Track 2 • Magnitude calibration. When a `trace_snapshot` records a magnitude (interpreted per `trace_type` : strength for moat traces, expected business impact for ad-hoc, see §40), that magnitude is later compared with realised magnitude in the `prediction_outcome` . An over-stated magnitude that comes in smaller than predicted, or an under-stated magnitude that comes in larger, is logged as magnitude error. What this scores: did the system correctly estimate how big the effect would be?

Track 3 • Persistence calibration. Each `trace_snapshot` records a persistence (duration for moat, expected relevance horizon for ad-hoc). Persistence is resolved by checking, at the named horizon, whether the trace’s hypothesis still holds and whether its materiality has decayed as predicted. Over-stated persistence (the effect faded faster than claimed) and under-stated persistence (the effect lasted longer) are tracked separately. What this scores: did the system correctly estimate how long the effect would last?

Source-reliability calibration In addition to the three trace-quality tracks, v5 makes source reliability a first-class calibration target. Every `evidence_event` references a source row (a Bloomberg headline, a SEC filing channel, an earnings transcript provider, a sell-side research desk, a satellite-data feed). The Evaluator updates per-source weights by horizon tier and by trace type as outcomes resolve: a source that has historically pushed traces in directions that `prediction_outcomes` later confirm earns higher weight; a source that has pushed traces toward outcomes that did not materialise earns lower weight. Source weights are bounded [0.25, 4.0] and are versioned (rule 3 of §04). The Trace Feed (§33) reflects this: a high-weight source publishes a Trace update with stronger surface treatment than a low-weight source, even when the underlying observation looks superficially similar.

Diagnostics calibration The diagnostics claim recorded in step 06 of §05 is a forward claim: this evidence should discriminate among the trace’s counter-hypotheses to this degree. After resolution, a diagnostics verdict is written: was the evidence as diagnostic as claimed, more so, or less? Persistent over-claiming of diagnostics is a per-agent and per-trace- type diagnostic, the same way mechanism over-confirmation is. What this scores: did the system correctly judge how much this evidence was going to teach it?

Aggregation Each track is published per (entity, agent, horizon-tier), per (`trace_type` , horizon-tier), and per (source, horizon- tier). Aggregates are versioned. The Evaluator writes trace-snapshot verdicts independently of outcome verdicts: a prediction can resolve `outcome_confirmed` / `mechanism_unconfirmed` while the underlying `trace_snapshots` resolve with high posterior calibration, accurate magnitude, and under-claimed diagnostics. The five scoring tracks (outcome, mechanism, posterior, magnitude, persistence, with diagnostics recorded alongside) can each say something different about the same closed prediction; that is the point.

The internal-vs-external precision boundary Internally, the system uses likelihood ratios and probabilities at full numeric precision. Externally, the customer-facing surface uses confidence bands unless calibration data supports decimals. The default external presentation:

Internal posterior probability	External confidence band
≥ 0.85	High
0.65 to 0.85	Medium-high
0.40 to 0.65	Medium
0.20 to 0.40	Medium-low
< 0.20	Low

The same convention applies to materiality (low / moderate / material / regime) and to activation (active / dormant / resolved / falsified). The architecture commits to no false Bayesian precision in the customer surface. The internal precision is what makes the back-end scoring honest; the external bands are what make the product credible. An agent or source whose calibration accumulates enough resolved outcomes to support narrower published bands earns them empirically; bands tighten with data, never with rhetoric.

What this section does not cover Selection bias on what gets predicted (does an agent emit predictions only on questions it expects to answer well?), the meta-question of “is calibration the right metric at all” for some downstream uses, and adversarial robustness of the scoring rule are deliberately out of scope. Each is a research question more than an architectural one. The architecture commits to making the data needed to study these questions available; it does not commit to specific answers.

A claim that cannot be scored is not a claim. A scoring rule that cannot be defended is not a moat. This section is what makes the central promise mean what it says.

§16 Identity resolution, anchoring claims to canonical entities

Every commitment in this document depends on one assumption: that a claim arriving today about “Apple” resolves to the same entity as a claim arriving in three years about “AAPL,” and that neither silently merges with a different company that happens to share the ticker after a reassignment. The Resolver is what makes that assumption load-bearing. This section specifies how.

Identity resolution is a problem with sharp edges. Tickers get reused. Names change. ISINs are per-security, not per-issuer. Companies merge, spin off, delist, and re-list. A naïve resolver (match on name, accept the answer) produces a system that quietly contaminates its own track record. A correct resolver is invisible when it works and audit-recoverable when it does not.

The identity stack Three levels of identifier, in decreasing order of authority:

LEI (canonical external). Legal Entity Identifier, ISO 17442. The 20-character GLEIF-issued code that uniquely identifies a legal entity globally. The single external identifier StahlTrace treats as authoritative. Every entity has exactly one current LEI; succession is tracked through GLEIF’s succession_event records, not by reassigning the LEI.

Aliases (everything else). Tickers, ISINs, CUSIPs, SEDOLs, common names, legal names, abbreviations. Stored in a separate table with an explicit validity window: the date range during which the alias was a correct way to refer to this entity. A claim referring to “Facebook, Inc.” in 2020 resolves to the same entity as a claim referring to “Meta Platforms, Inc.” in 2024 because both names are aliases for the same `entity_id` , with non-overlapping validity windows.

```
TABLE alias (  
  alias_id          text PK  
  entity_id         text FK-identity  
  alias_kind        enum    -- lei | isin | ticker | cusip | sedol | legal_name | common_name  
  alias_value       text    -- the literal string  
  exchange          text    -- for ticker; null for global aliases  
  jurisdiction       text    -- for legal_name disambiguation  
  valid_from        date  
  valid_to          date    -- null = currently valid  
  source            text    -- gleif | sec_filing | bloomberg | manual  
  superseded_by     text    -- alias_id of the alias that replaced this one  
  confidence         numeric -- for fuzzy-matched aliases  
)  
INDEX ix_alias_value ON (alias_kind, alias_value, valid_from, valid_to)  
INDEX ix_alias_entity ON (entity_id)
```

The structural commitment: tickers, ISINs, and names are never primary keys. They are search aids that resolve to LEI, which resolves to `entity_id` . A reader who understands only this paragraph understands the most important architectural decision in this section.

The resolver pipeline The Resolver is a service, not an LLM agent. The pipeline is deterministic logic over an authoritative reference store, with a queue for ambiguous cases. An incoming claim carries some subset of {LEI, ISIN, ticker+exchange, name, jurisdiction}. The Resolver attempts matches in this order, stopping at the first confident match:

1. LEI direct match. If the claim carries an LEI, look it up directly. If found and active, return. If found and superseded by a GLEIF succession event, follow the succession chain to the current entity. Confidence: 1.0.
2. ISIN → LEI lookup. Look up the security in the alias table to find the issuing LEI, then resolve. Confidence: 0.99.
3. Ticker + exchange + date. Look up the alias scoped to that exchange, with the claim’s `evidence_time` falling within the alias’s validity window. Tickers are never matched globally or without a date scope. Confidence: 0.95.
4. Name fuzzy match (with jurisdiction filter). If only a name is provided, fuzzy match against legal-name and common-name aliases, scoped by jurisdiction if known. Match score above 0.92 auto-resolves. Score in 0.75 to 0.92 is held in the bootstrap queue with the top 3 candidates surfaced. Below 0.75 is treated as a probable new entity.
5. Bootstrap queue. Anything below confidence threshold is held without committing. Resolution decisions are confirmed (initially manually, longer-term through ensemble cross-checks against multiple authoritative sources) before commit. Sustained backlog signals that the resolver’s confidence thresholds need recalibration.

Every successful resolution writes back to the substrate: the alias actually matched on, the alias’s validity window at time of match, the confidence score, the timestamp. A resolution decision made today can be recovered, audited, and replayed against an updated alias table without losing the original answer.

The alias table, provenance with validity windows

The validity window is the load-bearing field. When ticker AAPL on NASDAQ is reassigned in some hypothetical future, the original Apple alias gets `valid_to` = `reassignment_date` , and a new alias row is created with `valid_from` = `reassignment_date` . A 2026 claim about “AAPL” still resolves to Apple because the claim’s

`evidence_time` falls within the Apple alias's validity window. A 2031 claim resolves to whoever holds the ticker in 2031.

Corporate actions, six categories, six policies The Resolver runs a daily sync against GLEIF's published succession events plus exchange notices for ticker-level changes. Each category is handled differently:

Mergers, simple (one survivor, one absorbed). Auto-processed. The absorbed entity's `is_active` flag flips to false; its baseline becomes a sealed historical record; all aliases get `superseded_by` pointers to the surviving entity's aliases. New claims about the absorbed entity resolve to the surviving entity with an explicit note in the resolution metadata. The merger is recorded as a trace update of magnitude regime on the surviving entity, with the absorbed entity's prior baseline attached as evidence.

Mergers, complex (multi-party, with divestitures). Detected automatically from GLEIF, then held for additional resolution before the baseline transfer is committed. Multi-party mergers may have multiple "successors" in GLEIF, and only structured attribution can correctly assign which lines of business went where. The Resolver holds claims about the affected entities in a transitional state until disambiguation is complete.

Spin-offs and demergers. GLEIF records the new entity as a separate LEI but does not say which parts of the parent's trace graph (which Deep Insight traces, which Financial Model traces, which structural-future traces) should travel with the spin-off. The architecture commits to writing the attribution explicitly so that future reads can see what was inherited and what was retained. Initial attribution at launch is structured: the trace graph makes this clean because each trace carries a `chapter_anchor` and a scope that tag it as parent-relevant or subsidiary-relevant.

Delistings. Two sub-cases. Delisted but operating (taken private, listed elsewhere, restructured): the security alias gets a `valid_to` date but the entity continues. Wound up (liquidated, dissolved): the entity itself is archived. GLEIF's `RegistrationStatus` field distinguishes these. Auto-processed; both write a trace update of magnitude regime .

Name changes. Auto-processed. The old name alias gets a `valid_to` date; a new alias is created for the new name. The LEI persists. Recorded as a trace update of magnitude minor unless other factors elevate it.

Parallel quotations. Multiple ISINs and tickers per LEI is the steady state, not an event. The Resolver selects one listing as the primary for tracking purposes (price, volume, technicals), keeping all other listings as cross-reference aliases.

Primary listing, most liquid wins with stickiness For entities with multiple active listings, one is designated primary:

1. Compute median 90-day trading volume for each listing, denominated in USD at prevailing FX.
2. Select the highest. This is the primary.
3. Apply stickiness: the primary only changes if a different listing has been more liquid for at least 6 consecutive months. This prevents short-term liquidity spikes from flipping the primary.
4. Recompute monthly. Primary-listing changes are recorded as a trace update of magnitude minor .

The primary listing affects: which price series SF/04 Technicals uses to inform price-derived `atomic_observations` ; which volume profile is canonical; which exchange-specific corporate actions (splits, dividends) drive substrate events. It does not affect: identity (the entity is the same regardless of where it trades), financials (consolidated accounts apply globally), or evidence ingestion (filings from any jurisdiction are valid).

Customer override: the sealed client codification layer permits per-customer primary-listing overrides. A London fund tracking BHP via the LSE listing rather than the ASX listing can pin the primary to LSE in their own namespace, without affecting the canonical system.

Cold start, registering a new entity LEI in the claim. Best case. Look up the LEI in the GLEIF database (offline replica, refreshed daily), pull the entity's official record, create the Identity row. Aliases are seeded from GLEIF; additional aliases accrete as future claims arrive.

ISIN or ticker+exchange in the claim, no LEI. Use exchange or vendor data feeds (Bloomberg, Refinitiv, OpenFIGI) to look up the issuer's LEI. If found, proceed as above. If not, hold in the bootstrap queue.

Only a name in the claim. Search GLEIF by name + jurisdiction with a fuzzy-match score. High-confidence matches register automatically. Mid-confidence matches enter the bootstrap queue for ensemble disambiguation. Low-confidence matches are held without committing. The most common source of bootstrap-queue entries, and the most common source of subtle entity-conflation bugs if shortcut.

Identity schema, in brief The full DDL for the entities table lives in §21 alongside the other nine source-of-truth tables. The shape relevant here:

The Resolver writes here single-tenant: there is one canonical entities row per real-world entity, shared across customer tenants. Aliases (LEIs, ISINs, tickers, CUSIPs, legal names) live in a separate alias table joined by `entity_id`, so that ticker reuse, share-class fan-out, and parallel quotations do not require splitting the entity. Full schema in §21.

What this section does not cover Private companies without LEIs, entities outside finance (where LEI may not apply), and reference data licensing (Bloomberg, Refinitiv) are out of scope. The first is handled with internal-UUID-only entities flagged `no_lei` until a LEI is issued. The second is a v5 question. The third is a commercial concern, not an architectural one.

A claim resolves to an entity. An entity stays itself across mergers, listings, and renamings. Without that, every other commitment in this document is rhetorical.

§17 The runtime, the 14-step flow

The agent layer in §07 through §14 describes what each agent does. The trace journey in §05 describes what the architecture does as a whole. This section describes what runs them: the durable execution layer, the queues, the retry policies, the idempotency contracts, and the way the system stays correct when things fail. The 14-step flow is the runtime architecture — every step has an owner, an entry condition, a substrate write, and a recoverability guarantee.

Everything in the canonical core is append-only and timestamped. That commitment is meaningless if the runtime can lose work, double-write, or reorder events. The six harness contracts make the architecture's promises operational.

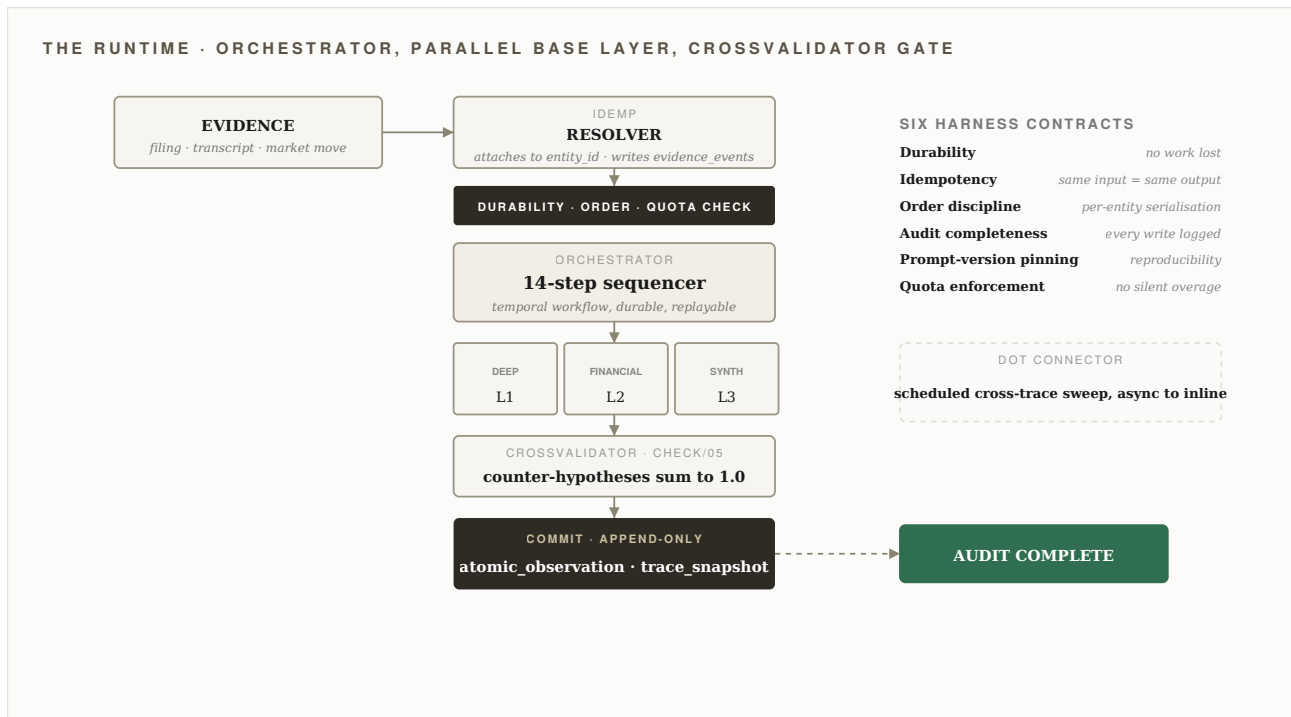


FIGURE 10 · RUNTIME FLOW FROM EVIDENCE TO COMMIT, WITH THE SIX HARNESS CONTRACTS ANNOTATED AS GATES. THE DOT CONNECTOR RUNS ON A SEPARATE SCHEDULED CADENCE, ASYNCHRONOUS TO THE INLINE LOOP. STEPS 1-8 OF THE 14-STEP FLOW LIVE INLINE; STEPS 9-12 ARE THE SCHEDULED LOOP; STEPS 13-14 ARE CONTINUOUS BACKGROUND.

Step ownership and runtime placement

#	Inline / Scheduled	Owner	Substrate write	Idempotency key
01 · Ingest	Inline	Resolver	evidence_events	hash(source_id, source_event_id)
02 · Deconstruct	Inline	Base layer agents	atomic_observations	hash(evidence_event_id , layer)
03 · Detect	Inline	Orchestrator	(in-memory only)	hash(atomic_observation_id)
04 · Match	Inline	Orchestrator + Dot Connector	(read-only)	hash(atomic_observation_id , sweep_kind)
05 · Create candidates	Inline	Layer agents	traces (activation ='candidate')	hash(atomic_observation_id , hypothesis_hash)
06 · Score	Inline	Layer agents + Evaluator	(in-memory; written with snapshot)	hash(atomic_observation_id , trace_id)
07a · Update children	Inline	Layer agents	(in-memory; written with snapshot)	hash(atomic_observation_id , trace_id, prompt_version)
07b · Update parents	Inline	Parent updater (Orchestrator-invoked)	(in-memory; written with snapshot)	hash(parent_trace_id, child_snapshot_ids[])
08 · Snapshot	Inline · CLOSSES INLINE	Orchestrator	trace_snapshots (with input_provenance for parent updates)	hash(evidence_event_id , trace_id, layer); hash(parent_trace_id, child_snapshot_ids[])

09 · Predict	Scheduled (within minutes)	Forecaster	<code>predictions</code>	<code>hash(trace_snapshot_id)</code>
10 · Outcome	Scheduled (when reality lands)	Evaluator	<code>prediction_outcomes</code>	<code>hash(prediction_id, resolution_evidence_id)</code>
11 · Calibrate	Scheduled (after each outcome)	Evaluator	<code>calibration</code>	<code>hash(prediction_outcome_id)</code>
12 · Promote	Scheduled (nightly)	Threadweave	<code>chapter_projections</code>	<code>hash(entity_id , chapter_anchor , sweep_date)</code>
13 · Trend	Continuous (query-time)	Threadweave + Dot Connector	(no write; query against <code>trace_snapshots</code>)	n/a
14 · Sleep	Continuous (<code>activation</code> review)	Orchestrator	<code>traces.activation</code>	<code>hash(trace_id, activation_review_date)</code>
15 · Author chapter	Triggered, not sequential	Canonical chapter author specialist (per §06c)	<code>canonical_chapters</code> (one row per version); on seed mode also <code>traces</code> and <code>trace_snapshots</code>	<code>hash(entity_id , chapter_type, prompt_version, trigger_id)</code>

Steps 1 to 8 are inline (typically under 14 seconds for a single observation against a known entity). Steps 9 to 12 are scheduled (sweep cadence varies by entity activity tier). Steps 13 and 14 run continuously in the background, with step 13 being a pure query (rule 1 of §06: no manual trend field) and step 14 being a periodic activation review. Step 15 is triggered, not sequential: it does not run after step 14 in flow order; it runs on its own cadence on the four trigger conditions specified in §06b (material accumulation, scheduled refresh, user request, falsifying outcome). The summary: steps 1–8 inline, 9–12 scheduled, 13–14 continuous, 15 triggered.

The six harness contracts The harness is bound by six constraints the architecture imposes on it. These follow from the epistemic rules in §04 and the access discipline in §22.

Durability. No agent invocation, `trace_snapshot` commit, prediction emission, or scored outcome can be lost in transit, even if the LLM provider, the database, or the orchestrator fails mid-step. Every state transition is recoverable from a persistent record.

Idempotency. Every retry of every agent step must produce the same downstream state as the first attempt. An `evidence_event` that has already been ingested cannot be ingested again. A `trace_snapshot` written for a given (`evidence_event`, trace, layer) cannot be duplicated by retry. A prediction emitted for a given `trace_snapshot` cannot be re-emitted. A nightly chapter-projection sweep that ran successfully cannot be re-run as if it hadn't.

Order discipline. Two `evidence_events` about the same entity arriving within the same second must be serialised through the Orchestrator for that entity, not run in parallel. Across entities, parallelism is fine. The harness enforces per-entity serialisation without enforcing global serialisation.

Audit completeness. Every agent invocation writes to the run-log before any side effect is committed downstream. If the side effect succeeds, the log records success. If it fails, the log records the failure. There is no path through the system that produces a substrate write without a corresponding log entry.

Prompt-version pinning. Every agent invocation records the exact prompt hash and model version that produced the output. A change to a subagent's prompt is a new version; old outputs are never silently re-attributed to the new version. Trace authoring (step 05) is pinned the same way; a trace authored under version X is tagged with X, and rejected candidates are also pinned to their version.

Quota enforcement. Every dispatch that will call an LLM consults the quota service before the activity begins. Tenant-scoped activities check tenant token and entity-count budgets; canonical activities check the canonical capacity envelope. Blocked dispatches surface a structured error rather than silently failing. Usage is metered at activity granularity so retries do not double-charge.

The stack Orchestration: Temporal. Every agent dispatch is a Temporal workflow. Every subagent invocation is a Temporal activity. Workflows are deterministic and replayable; activities are at-least-once with idempotency keys. Per-entity serialisation is enforced via Temporal task queues keyed by `entity_id`. The Orchestrator is the workflow that owns the dispatch sequence; subagents are activities it calls.

State: Postgres + pgvector. All canonical state (the twelve source-of-truth tables of §21 plus calibration and `run_log`) lives in Postgres. Schema migrations are append-only where possible. Foreign keys are enforced. Every write is wrapped in a transaction; transactions either commit fully or not at all. The `run_log` also lives in Postgres, with an OpenSearch projection for full-text search.

Coordination: Redis. Short-lived coordination only. Distributed locks for the rare cases where Temporal's task-queue serialisation is not sufficient (e.g. cross-entity invariants during the nightly Dot Connector sweep), and a small cache of frequently-read calibration snapshots. Redis is not a source of truth and never holds state that is not reproducible from Postgres.

Blobs and search: S3 + OpenSearch. Raw evidence payloads (filings, transcripts, large jsonb documents) live in S3, addressed by content hash. OpenSearch indexes evidence text and agent learning notes for retrieval. Both are projections of state that ultimately resolves to Postgres pointers; neither is independently authoritative.

Time series: TimescaleDB. `prediction_outcomes` and time-series source feeds (price, volume, peer movement) live in TimescaleDB hypertables, on the same Postgres engine, for efficient time-windowed queries.

The idempotency contract Every agent invocation carries an `idempotency_key` derived deterministically from its inputs (the table above lists each step's key formula). The key is checked before the work is done. If a record exists for that key, the result is returned without re-invoking the agent. The system is safe to retry indefinitely without producing duplicate `trace_snapshots`, duplicate predictions, or duplicate chapter projections. Rule 3 of §06 (no double counting) is enforced by step 08's idempotency key: a given `evidence_event` can produce at most one `trace_snapshot` per (trace, layer).

Retry policy Resolver, Orchestrator. Exponential backoff, 5 retries over ~2 minutes, then dead-letter. Failure here blocks the inline loop; alerting is immediate.

Base-layer subagents (Deep Insight, Financial, Synthetic Futures). Exponential backoff, 3 retries over ~90 seconds, then mark the proposed trace updates as `subagent_failed` and continue with whatever subagents did return. CrossValidator runs against the partial set. The Orchestrator records the gap.

CrossValidator. Exponential backoff, 3 retries. If CrossValidator fails after retries, no `trace_snapshot` commits, the inline loop fails closed. Failure here is louder than failure in any individual subagent.

Trace authoring (step 05). Exponential backoff, 3 retries. If the layer agent fails to converge on a candidate after retries, the originating observation is held; the gap is logged in the `run_log`. The observation is reclassified as non-material if no existing trace absorbs it on the next dispatch attempt.

Dot Connector (sweep). Long-running activity with checkpointing. If interrupted, resumes from last checkpoint. If a single entity in the sweep fails, the sweep continues for other entities and the failed entity is requeued for the next sweep cycle.

Forecaster, Evaluator, Threadweave promotion. Exponential backoff, 5 retries over ~5 minutes. All write through idempotency keys, so retries are safe even if the activity actually succeeded but the response was lost in transit.

The nightly sweep, concrete mechanics The Dot Connector's scheduled cross-trace synthesis is implemented as a Temporal cron workflow. Runs at 02:00 UTC each day for tier-1 (active) entities, weekly on Sundays for tier-2 (quiet) entities, monthly for tier-3 (dormant) entities. The tier of each entity is recomputed weekly from observation volume, trace activity, and user attention signals.

For each entity in the sweep, the workflow:

1. Acquires the per-entity Temporal task-queue lock.
2. Reads the substrate snapshot for the entity at `sweep_start_time`, including the trace graph (active and dormant), recent `atomic_observations`, recent `trace_snapshots`, and CrossValidator's unresolved disagreements.
3. Invokes the Dot Connector activity with idempotency key `hash(entity_id, sweep_date)`.
4. If `trace_relationships` are emitted, writes them. For relationships with `forecastable=true`, downstream Forecaster activities are triggered to write predictions citing the relationship.
5. Threadweave then runs the chapter-projection step (step 12), promoting traces that clear the materiality threshold into `chapter_projections`.
6. Activation review (step 14) walks dormant traces with recent matching observations to check whether the reactivation gate is met (rule 4 of §06: no keyword-only reactivation).
7. Releases the lock and writes a sweep audit record.

Failure modes LLM provider fails or rate-limits. Activity-level retry with backoff. Provider failover wraps a primary and secondary provider, fails over after N failed attempts on the primary. If both providers fail, the activity dead-letters and the workflow surfaces a visible error.

Postgres unreachable. All workflows pause. No partial writes are committed because every multi-step state transition is wrapped in a single transaction. When Postgres returns, workflows resume from where they left off.

Temporal itself fails. Worker processes restart from durable state. In-flight activities resume; in-flight workflows replay deterministically from event history. No state is lost. No `trace_snapshots` are double-committed.

A subagent returns malformed output. Schema validation rejects the output before any commit. The activity is retried with a regenerated prompt. If retries exhaust, the subagent's contribution is dropped from the inline loop and the gap is logged.

Two `evidence_events` about the same entity arrive simultaneously. Both are accepted at the Resolver. Both enter the Orchestrator's per-entity task queue. Temporal serialises them by arrival order. The second one runs against the trace state that resulted from the first, not against a stale snapshot. There is no race.

What this section does not cover Deployment topology, infrastructure-as-code, secrets management, network architecture, and observability dashboards are out of scope. Operational concerns that will evolve with scale.

The harness is not the moat. But a wrong harness invalidates the moat. The 14-step flow above, plus the six harness contracts, plus the nine enforcement rules of §06, are what make every other commitment in this document mean what it says.

§17a The cold-start input phase NEW

The 14-step flow (§17) describes how a single piece of evidence enters and leaves the system. It is, implicitly, a continuing-state loop: it assumes a trace graph already exists for the entity, so that an incoming observation has traces to update. **That assumption does not hold for a new entity.** When an entity first enters the universe, there is no trace graph for evidence to move. The cold-start input phase is the bootstrap that *creates* the initial trace graph, before the longitudinal calibration loop has anything to chew on.

v6 makes the cold-start phase first-class because v4.1 proved it is a distinct mechanism with its own failure modes, and because the §24 launch path runs a cold-start on every one of the launch entities. The phase is the v4.1 eleven-stage orchestration, exercised end-to-end on a live ticker, lifted onto the front of the continuing flow:

#	COLD-START STAGE	WHAT IT DOES	SUBSTRATE OUTPUT
1	INTAKE	Register the entity, resolve identity, allocate the run	<code>entities</code> , <code>run_log</code>
2	INIT_REGISTRY	Seed fund-scope methodological conditions	(registry snapshot)
3	FACTUAL_REFRESH	Web-search fund-scope facts (10Y yield, SOFR, index level)	<code>atomic_observations</code>
4	SOURCE_EXTRACTION	Pull filings (10-K, latest 10-Q, 8-Ks, proxy, transcript) into a section-granular typed bundle	<code>evidence_events</code>
5	TOP_LEVEL_NODE	Build the stats block, entity identity/origins, and the per-chapter supervisor steers	<code>atomic_observations</code> , <code>run_log</code>
6	DEEP_DIVE_PARALLEL	Dispatch the ten company-facing chapter deep-dives in parallel	(deep-dive briefs — working artifacts)
7	SEED_AUTHOR	Materialise the deep-dive briefs as seed-mode canonical chapters (§06b)	<code>canonical_chapters</code> , candidate <code>traces</code>
8	SCORE_AND_SNAPSHOT	Candidate traces walk steps 05–08; counter-hypothesis gate, scoring, first snapshot, CrossValidator commit	<code>traces</code> , <code>trace_snapshots</code>
9	CONVERGE	Cross-chapter convergence sweep; budgeted re-spawn (3 iterations) on substantive contradictions	<code>trace_relationships</code>
10	META_CHAPTERS	Score the meta-chapters (thinking-quality, endgame distribution) over the converged graph	<code>trace_snapshots</code> (ST-13, ST-14)
11	DELIVERABLE	Render the PM-facing PDF (§06e) and the run summary	(deliverable + <code>run_log</code>)

The cold-start outputs become the entity's **initial** `trace_snapshots`, `atomic_observations`, and canonical-chapter seeds. Once the bootstrap completes, the entity transitions into the continuing 14-step flow: subsequent evidence updates the traces the cold-start created, the Forecaster begins emitting predictions, and the longitudinal loop begins. **The cold-start is the bootstrap; the loop is the system.**

Two design properties carry over from v4.1 intact and are load-bearing. First, **source determinism**: the source-extraction stage writes a single section-granular bundle once, and every downstream chapter agent cites into it by `(source_id, section_id)` rather than re-fetching. A re-spawn during convergence reads the same source content as the original spawn, so re-scoring is not confounded by source drift. Second, **no partial ship**: if any chapter fails to return a valid result after one retry, the phase emits an explicit non-convergence report and does not produce a deliverable that silently covers thirteen of fourteen chapters. Fake coherence is worse than explicit non-convergence — the same discipline the framework has enforced since v3.

§17b Degrade-safe input handling NEW

v5.1 assumed the agent set is online and producing. It documented no behaviour for the case where a single agent times out, returns malformed output, or finds its substrate dependency unavailable. At one entity this is a tolerable silence. At the hundred-entity launch universe it is a liability: without an explicit degrade-safe contract, the whole loop is one transient market-data or filings-feed outage away from halting.

v4.1 shipped the discipline and v6 adopts it as an architectural contract. **Every agent's input contract classifies each input as required or optional.** A *required* input absent halts that agent's run and emits a typed failure — the system does not guess. An *optional* input absent puts the agent into a degraded mode, in which it runs on the inputs it has and records the absence explicitly in its output's `provenance.degraded_inputs[]` field. The Orchestrator never halts the whole run on a single optional-input failure.

The pattern is concrete. If the stats block fails to build (market-data feed unavailable), the FM agents see no stats block and fall back to a less-informed mode rather than fabricating numbers — and the `data_provenance.missing_fields[]` on the (partial or absent) stats block records exactly what was missing. If one chapter's deep-dive brief fails, that chapter's scoring runs in its un-briefed mode for that chapter only, while the other thirteen proceed. The degradation is always visible downstream: a customer reading the PM deliverable sees, in provenance, which inputs were degraded for which chapters, so a thinly-sourced judgment is never silently presented as a well-sourced one.

This composes with the honest-non-scoring convention of §40a. Degrade-safe handling governs *agent inputs* (an upstream artifact failed to arrive); the **INSUFFICIENT** convention governs *deep-dive findings* (the evidence that did arrive does not support an answer). Both are forms of honest non-reporting, and both are recorded rather than papered over. The architecture's commitment is not that nothing ever fails; it is that every failure is visible, typed, and isolated to the smallest scope that contains it. Note that §17b is an operational discipline on agent input contracts, not a change to the core 14-step flow or the Orchestrator's sequencing — the runtime of §17 is unchanged.

§18 Security model, authentication authorization encryption response

The architecture so far treats access discipline as a data-level concern (the access matrix in §22) and tenant isolation as a runtime concern (§17 + Part V). Neither addresses how a human or service gets into the system, or what happens when their access should not be there anymore. This section specifies the security primitives the rest of the architecture depends on.

Security in this design is not a feature; it is a precondition. The seal in Part V, the audit trail in §17, the per-tenant isolation in §27, and the metering in §31 are all only as strong as the authentication and authorization that gate them.

Authentication Self-service authentication. OIDC-based, via a managed identity provider (Auth0, Clerk, Stytn, or equivalent; choice deferred). Email + password with mandatory email verification. MFA optional for builders and viewers, mandatory for admins. Session duration: 24 hours active, with rolling refresh.

Enterprise SSO. SAML 2.0 and OIDC both supported for Enterprise-tier customers. Customer's identity provider is the authentication source; StahlTrace consumes assertions and maps to internal user records. JIT user provisioning supported; SCIM for managed user lifecycle deferred to v4.1.

Service-to-service authentication. mTLS between internal services with short-lived certificates (24-hour rotation, automated). API tokens for customer-side integration; the only long-lived credentials in the system, blast radius bounded by per-tenant scoping.

Authorization at the human level Three user roles, parallel to the agent-level access matrix:

Admin. Full tenant control: create/delete agents, manage users, change subscription tier, view billing, view audit log, configure webhooks, generate API tokens. The tenant's billing-responsible role; one mandatory per customer.

Builder. Author and run agents in the tenant, configure validation rules, override Threadweave parameters, author tenant-private traces, subscribe and unsubscribe entities (within tier allowance), read tenant-scoped usage. Cannot manage users, cannot change billing, cannot generate API tokens.

Viewer. Read-only access to tenant outputs (agent results, predictions, insights, `trace_snapshots`, the customer's view of canonical entity traces). Cannot author agents, cannot subscribe entities, cannot read raw usage logs.

Role assignment is per-user-per-tenant. A user cannot belong to two tenants under the same identity; switching tenants requires separate user records. A deliberate constraint that simplifies the authorization model at the cost of some convenience for consultants who serve multiple customers.

Encryption In transit. TLS 1.3 for all external traffic. Internal mTLS for service-to-service. No plaintext over any network segment.

At rest. AES-256 for database storage (Postgres, TimescaleDB, S3, OpenSearch). Per-tenant encryption keys for tenant-scoped data, with envelope encryption: a tenant master key encrypts per-table data keys, the master key is held in a managed KMS (AWS KMS or equivalent). Canonical-side data uses a separate canonical master key.

Key management. All encryption keys in a managed KMS; no plaintext keys in application memory beyond the lifetime of a single request. Key rotation is on a 90-day schedule for canonical keys, on customer-driven schedule for tenant keys (default 90 days, configurable to 30 days for compliance-sensitive customers).

Audit of access Every read of tenant data by canonical operators is logged with a tamper-evident chain: the operator's identity, the elevation justification, the timestamp, the rows read, the duration of the elevation. The audit log is itself stored with append-only semantics; modifications to the audit log require a separate elevation that is itself audited. Customer admins can view all canonical reads of their tenant data through the audit dashboard.

Customer-initiated reads. Standard tenant operations. No elevation needed; standard authentication suffices.

Internal operator reads. StahlTrace internal personnel reading tenant data (for support, debugging, or incident response) requires explicit elevation: a documented reason, a time-bounded grant (default 4 hours, hard maximum 24 hours), and the customer is notified before the elevation begins (synchronous notification for non-emergency, asynchronous for emergency with post-hoc audit). No standing access; every read is logged.

Canonical-side reads. Operators reading canonical-only data (no tenant scope) follow standard authentication. Audit captures the read but no elevation is required.

Token revocation and session management Tenant-key revocation. A customer admin can revoke a tenant's encryption keys, which renders all tenant data unrecoverable. This is a destructive operation gated by multiple confirmations and a 24-hour cooling-off period for non-emergency revocations. Used in compromise scenarios where the customer needs to be sure no past data can be read.

API token revocation. Immediate. Tokens are checked against a revocation list on every request. Revoked tokens fail closed within seconds of revocation propagation.

Session revocation. Admin-initiated session termination forces re-authentication. Used when offboarding a user or when a session is suspected of compromise.

Compromise response LLM provider API keys. Treated as compromised by default if any anomaly suggests it. Rotated automatically on a schedule and on demand. Activity audit logs allow post-hoc reconstruction of which prompts were sent under which key.

Database credentials. Held in KMS, never in code. Rotated on a 30-day schedule; emergency rotation on 1-hour timeline. Application reconnection is automatic.

Tenant compromise. If a tenant suspects their credentials are compromised, the response is: revoke all API tokens for the tenant, force re-authentication for all human users, audit log review for the suspected exposure window, optional tenant-key rotation if the customer wants to ensure encrypted data at rest is no longer readable with prior keys.

Canonical compromise. A canonical compromise is treated as an existential incident. The runbook (out of scope for this document) involves: isolating affected systems, preserving forensic state, customer notification (with regulatory disclosure where required), recovery from append-only logs, and post-incident review. Every commitment in this architecture depends on the canonical layer being uncompromised; the discipline of treating canonical compromise as existential is what keeps the discipline real.

What this section does not cover Specific compliance certifications (SOC 2, ISO 27001, regional equivalents) are operational, not architectural. The architecture is designed to make those certifications achievable; pursuing them is a v4.1 commercial decision.

Security is the floor. Every commitment above this floor only holds if the floor holds.

Part III · The data substrate Where the moat lives on disk. Twelve source-of-truth tables, implemented on a small physical stack. The trace primary at the centre; `evidence_events` and `atomic_observations` on the inbound side; `trace_snapshots`, `trace_relationships`, `predictions`, `prediction_outcomes`, and `chapter_projections` on the outbound side; `canonical_chapters` and `chapter_type_registry` as the prose authoring surface (new in v5.1). Plus auxiliary calibration, `trace_hierarchy`, and `run_log` substrates. An access matrix that enforces the orchestrator pattern in code. Agents never touch SQL.

The data substrate

Where the moat lives on disk. Twelve source-of-truth tables, implemented on a small physical stack, the trace primary at the centre. v6 makes no structural change to the substrate; it clarifies how the v4.1 trial's working artifacts flow into the existing tables (§20).

§19 Data acquisition, what feeds the substrate

The architecture's epistemic discipline depends on properties of the data the system cannot fabricate: reliable timestamping, deterministic entity resolution, durable provenance, and source-tier metadata. These are not operational concerns layered on top of the substrate, they are architectural commitments the substrate is built to enforce. Without them, append-only timestamping is meaningless, mechanism attribution is unverifiable, and calibration discipline collapses into theatre.

The four input streams Launch coverage relies on four input streams. Each carries a different latency profile, a different provenance posture, and a different role in the substrate.

Annual filings (10-Ks and equivalents). Authoritative annual disclosure. SEC EDGAR for US-listed entities. Equivalent registers per jurisdiction: ESMA's European financial-disclosure database, Companies House (UK), AFM (Netherlands), BaFin (Germany), AMF (France). EDGAR is the canonical reference standard; equivalent registers are tier-equivalent for non-US entities. Latency: inline; webhook or polling on a 15-minute cadence; substrate-ready within an hour of public availability.

Quarterly disclosures (10-Qs and earnings calls). Quarterly structured filings via the same registers. Earnings call transcripts via S&P Capital IQ Transcripts (enterprise tier) or AlphaSense / Sentieo (lower price points). Latency: filings inline as for 10-Ks; transcripts inline-once-transcribed, typically 4 to 24 hours after the call. Provenance: filing URL plus accepted timestamp for filings; transcript provider plus call timestamp plus speaker attribution for calls.

Ten-year financial history. Normalised balance sheet, income statement, and cashflow statement covering at minimum a ten-year window per entity. S&P Capital IQ as primary working assumption; FactSet Fundamentals as alternative; LSEG Workspace as third option. Each provider normalises line items across companies and reporting standards, the work the architecture does not do internally and explicitly does not try to. Latency: scheduled

rebuilds, never inline; nightly full pulls for active entities, weekly for tier-2 entities, monthly for the long tail. Raw vendor responses retained as evidence blobs in S3 so `trace_snapshots` can be reconstructed from the original signals if vendor data is later revised.

News flow. Real-time news with source-tier metadata. Dow Jones DNA as primary working assumption; Bloomberg News API as alternative; LSEG/Reuters News as third option. Free-tier development uses NewsAPI or equivalent with explicit understanding of degraded source-tier reliability. Latency: inline with severity-based

prioritisation. Breaking news routes through the inline loop within seconds; routine news goes to the nightly Dot Connector sweep. Provenance: article URL, publication timestamp, source tier (regulatory wire, primary newswire, financial press, secondary press, blog/social), resolved entity references.

Reference data, the join key layer Identity resolution (§16) requires authoritative reference data that maps entity-identifying signals to the system's canonical `entity_id`. Two free authoritative sources cover the launch universe:

GLEIF. Global Legal Entity Identifier Foundation. Authoritative LEI registry. Free, daily concatenated file download. Refreshed daily as a scheduled job. The source of truth for LEI-to-entity mapping.

OpenFIGI. Bloomberg's free financial-instrument identifier service. Maps tickers, ISINs, CUSIPs, and SEDOLs to FIGI codes which can then be cross-walked to LEIs. Used by the Resolver as the primary path from market signals to canonical entities.

Reference data is acquired by scheduled jobs, not inline. The Resolver caches recent lookups; the cache is refreshed nightly to absorb new listings, mergers, and ticker reuses.

Price data feed The price data feed is a first-class evidence stream. End-of-day pricing flows through the same `atomic_observation` pipeline as filings and news; price-derived observations attach to the same trace graph and are subject to the same nine enforcement rules as any other evidence, with one additional discipline: predictions emitted from price-relevant traces carry an `expected_price_reaction` band so a later price move that lands inside the band does not double-count as fresh evidence (rule 3 of §06).

Price acquisition. End-of-day OHLCV per primary listing per entity, plus benchmark indices (regional broad-market index, sector index, factor portfolios) and FX rates. Vendor: same data providers as ten-year financial history (S&P Capital IQ, FactSet, LSEG). Latency: nightly batch for end-of-day data; intraday is deferred.

Decomposition. Raw return is computed for each entity-window pair. Market-adjusted, sector-adjusted, and factor-adjusted returns are computed by subtracting the benchmark or factor-portfolio return from the raw return. The residual is what remains after these adjustments, plus volume z-score, plus persistence (does the move hold at +5d, +20d), plus event-specificity (did peers move similarly), plus reflexivity class (does this entity's stock price feed back into its operating reality). All computed deterministically in Python, no LLM call; the decomposition is carried as fields on the resulting `atomic_observation`, which then routes through the standard 14-step flow to update affected traces.

The Resolver as quality gate Every artifact arriving at the substrate passes through the Resolver, whose responsibilities extend beyond entity resolution to architectural quality gates. Six gates, each enforced before any artifact reaches the canonical store:

Schema validation. The artifact matches the expected structure for its source (XBRL for filings, structured JSON for vendor financials, conforming article schema for news). Malformed artifacts route to a reject queue with explicit failure reason.

Timestamp sanity. The artifact's claimed publication timestamp must be in the past, must be after the entity's earliest known activity, and must be consistent with any other timestamp evidence. Future timestamps and impossible timestamps are blocking errors.

Entity-resolution confidence. The Resolver computes a resolution confidence score per artifact. Scores below threshold (default 0.95 for unambiguous LEI matches; 0.85 for ticker-based matches with corroborating context)

route to the bootstrap queue rather than auto-attaching. Auto-attached artifacts carry the resolution confidence as metadata.

Deduplication. The Resolver hashes the artifact content and checks against existing substrate entries before committing. Duplicate artifacts (the same press release wired through three news services) are recorded once with multi-source provenance, not three times.

Language detection. Non-English artifacts are tagged with detected language and routed through a translation pipeline before atomic-observation extraction. Translation quality is itself a metadata field.

Artifacts that fail any gate route to an explicit reject queue. Failures are not silent. Recurring failure patterns surface as architectural improvements rather than silent data loss.

Why acquisition is verifiable artifacts, not LLM retrieval A sophisticated reader will ask: given that LLMs can already summarise public companies, retrieve recent news, and approximate financial figures via training data and on-demand search, why does StahlTrace operate a separate acquisition layer at all?

LLMs deliver real value on three categories of work, and the agent layer uses them extensively for exactly these: reading and structuring source documents the system has acquired (turning a 10-K into typed `atomic_observations`, identifying factor mentions, proposing candidate traces); resolving entity references in unstructured text; synthesising across many sources.

What LLMs do not deliver, and what the architecture's epistemic discipline categorically requires:

Verifiable artifact provenance. An LLM can summarise "what's in Novo Nordisk's most recent 10-K." It cannot produce the 10-K itself as a verifiable artifact with an EDGAR URL, an SEC-stamped accepted timestamp, and an unaltered source document the system can replay later for audit. Append-only depends on the artifact existing as a fact in the world before the system observed it. LLM summaries are not facts in the world; they are generated approximations.

Reliable specific numbers. LLMs hallucinate specific figures with calibrated-sounding confidence. For finance customers, confidently-wrong specific numbers are the failure mode that destroys trust. Vendor-normalised financial history is acquired precisely because the architecture cannot afford LLM-generated numbers entering the substrate as if they were measurements.

Coverage you can audit. An acquisition layer guarantees coverage by virtue of having actually pulled the artifacts; an LLM-only system has coverage by hope.

Predictions before outcomes. The architecture's calibration discipline depends on predictions being timestamped before the outcomes that resolve them. An LLM with web access at prediction time may inadvertently retrieve information from after the prediction window, contaminating the calibration. The acquisition layer's strict timestamping isolates predictions from outcomes by construction.

The agents read. The acquisition layer records. LLMs are excellent agents and inadequate recorders. The architecture separates the two by design.

Customer-supplied feeds, forward compatibility Some customers will want to feed proprietary research, internal estimates, expert-network notes, or alternative data into the same pipeline as canonical sources. The customer-supplied feed channel inherits the same Resolver gates (schema validation, timestamping, entity resolution, deduplication, source-tier annotation) but carries a distinct source tier (`customer_supplied`) and is sealed to the customer's tenant per §29.

Customer-supplied feeds never contaminate canonical calibration. They feed customer-tenant agents and customer-tenant Threadweave projections, but their predictions and outcomes resolve against customer-tenant calibration only.

What is deliberately deferred Foreign-language disclosure beyond translation. Non-English filings are accepted via equivalent registers and routed through translation, but jurisdictions where filing structures differ materially from XBRL-style standards (Japan EDINET, China CSMAR with full coverage, smaller emerging markets) are deferred.

Intraday market data. End-of-day pricing, fundamental ratios, and event-driven price moves are in scope. Tick-level intraday is deferred to v5+.

Alternative data. Satellite imagery, credit-card aggregates, web-traffic analytics, app-store data. Commercially valuable but fundamentally different in shape from the four launch streams. They require their own acquisition and quality discipline. Deferred to operations as customers ask for it.

Social media as primary source. Monitored only when it surfaces in regulatory filings, primary-newswire coverage, or executive statements meeting the §38 Public Actor Trace guardrail. Not a primary source; the noise-to-signal ratio at scale defeats calibration discipline.

§20 Data substrate, twelve source-of-truth tables AMENDED

v6 makes no structural change to the substrate. The twelve source-of-truth tables of v5.1 are the end-state design and they stand: `entities` and `sources` (the join-key and calibration layer); `evidence_events` and `atomic_observations` (the inbound chain, raw fact → typed claim); `traces` and `trace_snapshots` (the persistent reasoning layer); `trace_relationships` (cross-trace synthesis); `predictions` and `prediction_outcomes` (the forward-test layer); `chapter_projections` (the machine-assembled customer read surface); and `canonical_chapters` with `chapter_type_registry` (the prose authoring surface). The auxiliary substrates (`calibration`, `trace_hierarchy`, `run_log`) are likewise unchanged. The trace remains the primary object; nothing competes with `trace_snapshots` for authority over trace state.

The only amendment in this section is a clarification, not a new table. The v4 / v4.1 reference implementation produced several typed working artifacts during its cold-start trial — a *stats block* (the pre-computed numerical anchor the FM agents read), a *top-level node* (entity identity, origins, and the per-chapter supervisor steers), a set of *deep-dive briefs* (the per-chapter specialist findings), and a *reverse-DCF surface* (the numerical PIE grid). These were filesystem artifacts in a single-shot, ephemeral run-tree. **v6 does not promote them to source-of-truth tables.** They are *deep-dive-layer working artifacts* — inputs to canonical-chapter authoring and to the FM agents — and in the end-state they live as content within the existing tables, not as new ledgers:

- The **deep-dive briefs** are the seed-mode input to canonical-chapter authoring (§06b, §06d). In the end-state, a brief is the working draft that materialises into a `canonical_chapters` row and its candidate traces; it does not need its own source-of-truth table, because the canonical chapter *is* its persistent form.
- The **top-level node** and its **supervisor steers** (§14) are run-scoped orchestration artifacts. They belong in `run_log` (the audit substrate) or as a derived per-run view, not as a new source-of-truth ledger.
- The **stats block** and the **reverse-DCF surface** are numerical context the FM agents read (§09). They are `atomic_observations` of the entity's market state and a derived numerical artifact respectively —

observations and derived context, not new primary objects. The two-surface discipline (structural score on the trace, numerical grid as context) is enforced by keeping the grid out of the scoring path, not by giving it a privileged table.

This is the v4.1-vs-end-state line drawn cleanly. v4.1 is a trail — a working trial that proved the cold-start mechanics and the deep-dive agentic layer under real dispatch. Its filesystem artifacts are how a single-shot trial stored its intermediate work. The architecture's end-state substrate is the twelve-table trace model, and the cold-start's outputs flow into that model (as canonical chapters, atomic observations, traces, and audit-log entries) rather than expanding it. The substrate did not grow; the cold-start layer learned how to write into the substrate that already exists.

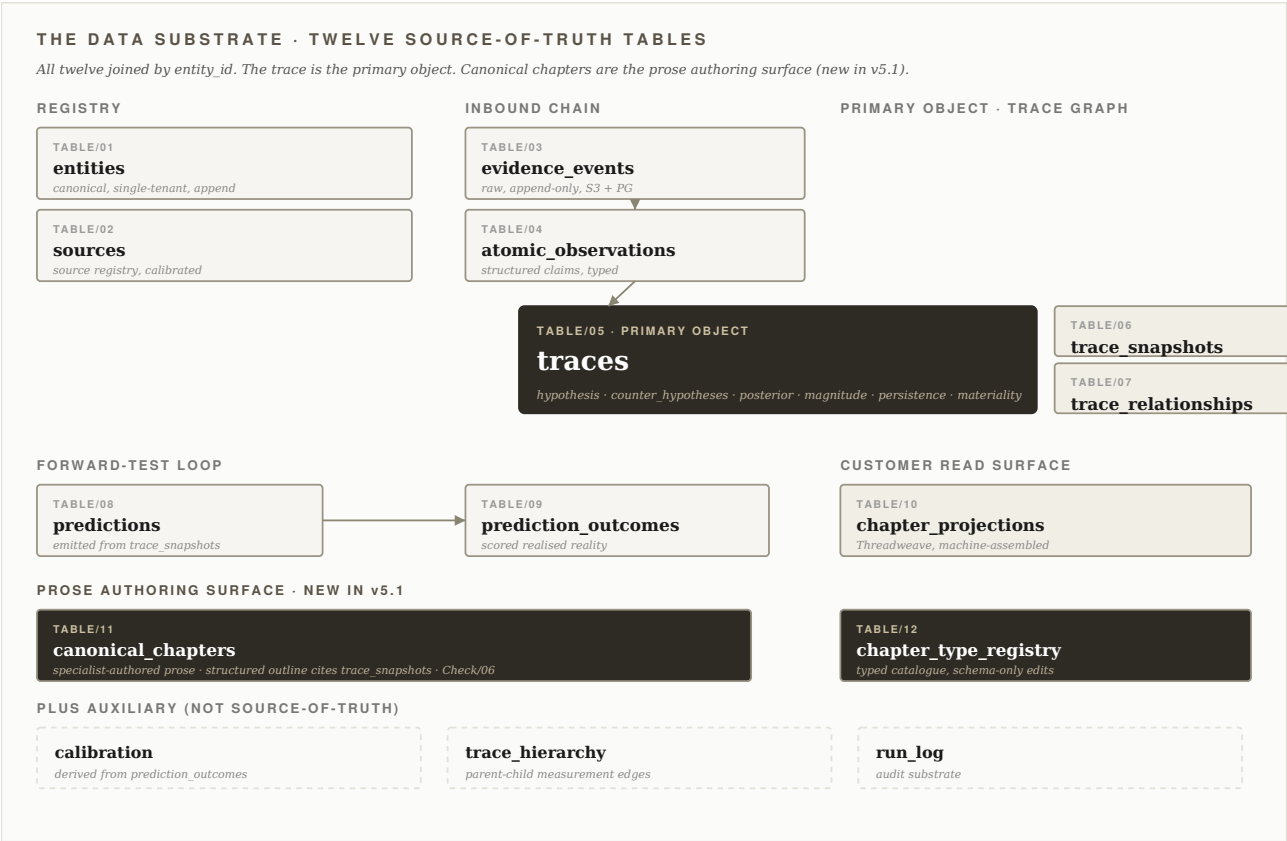


FIGURE 3 · THE DATA SUBSTRATE: TWELVE SOURCE-OF-TRUTH TABLES ANCHORED ON THE TRACE PRIMARY.

§21 Schemas, the tables that carry the moat

Ten load-bearing tables in v5 (down from v4's twelve, after the delta+belief_update collapse). Traces are the primary object. Trace_snapshots are the history. Trace_relationships are the cross-trace synthesis. Predictions and prediction_outcomes close the forward-test loop. Chapter_projections are the customer-facing read surface. Entities, sources, evidence_events, and atomic_observations are the inbound chain.

entities • the entity registry

```
TABLE entities (
  entity_id      text PK          -- internal UUID, never changes
  lei            text UNIQUE      -- canonical external identifier
  legal_name     text            -- current legal name
  common_name    text            -- display name
  jurisdiction    text            -- ISO 3166-2 country code
  primary_listing text NULL       -- ticker plus exchange, where applicable
  registration_status enum        -- active | inactive | merged | dissolved
  parent_entity_id text NULL FK--entities -- for subsidiaries with their own LEI
  created_at     timestampz
  superseded_at  timestampz NULL  -- non-null if entity was merged into another
  superseded_by  text NULL FK--entities
  gleif_last_synced timestampz
  semantic_embedding vector(1536) NULL -- entity description embedding for clustering
)
INDEX ix_entities_lei            ON (lei)
INDEX ix_entities_jurisdiction  ON (jurisdiction, registration_status)
INDEX ix_entities_active        ON (entity_id) WHERE registration_status = 'active'
INDEX ix_entities_embedding     USING ivfflat (semantic_embedding vector_cosine_ops)
```

The Resolver is the single writer. Aliases (historical LEIs, ISINs, tickers, CUSIPs, legal names, ticker reuses) live in a separate aliases table joined by `entity_id` so that share-class fan-out, parallel quotations, and ticker reassignment do not require splitting an entity. The aliases schema is omitted here for brevity — it carries forward unchanged from v4 with `entity_id` , `alias_kind` , `alias_value` , `valid_from` , `valid_to` , `source` , `superseded_by` , `confidence` .

sources • the source registry

```
TABLE sources (
  source_id      text PK
  source_name     text
  source_tier     enum
  source_kind     enum
  jurisdiction    text NULL
  tenancy_scope   enum
  tenant_id       text NULL FK
  base_reliability numeric
  current_reliability numeric
  reliability_horizon enum NULL
  ingested_first_at timestampz
  last_active_at  timestampz
  active          bool
)
INDEX ix_sources_active        ON (source_id) WHERE active = true
INDEX ix_sources_tier          ON (source_tier, source_kind)
INDEX ix_sources_tenant        ON (tenant_id) WHERE tenancy_scope = 'tenant_private'
```

Sources are the architecture's `calibration` target for source reliability (§15). A new source enters with `base_reliability` derived from its tier; `current_reliability` is updated by the Evaluator after each `prediction_outcome` that traced back through this source. A source whose evidence has historically pushed `traces` in directions that `prediction_outcomes` confirmed earns higher `current_reliability`; a source

whose evidence pushed `traces` toward outcomes that did not materialise earns lower. Bounded [0.25, 4.0] to prevent any single source from being silenced or made dominant.

`evidence_events` · *the immutable inbound record*

```
TABLE evidence_events (
  evidence_event_id      text PK
  entity_id              text FK-entities
  tenant_id              text
  source_id              text FK-sources
  source_native_id       text
  source_url             text NULL
  ingested_at            timestampz
  evidence_time           timestampz
  jurisdiction           text NULL
  payload_kind           enum
  payload_ref            text
  payload_hash           text
  payload_bytes          bigint
  language               text NULL
  redaction_class        enum
  -- Lineage (rule 3 of §06: no double counting)
  original_source_id     text NULL FK-sources
  evidence_lineage_id    text
  independence_score     numeric
  superseded_by          text NULL FK-evidence_events
  superseded_reason      text NULL
)
INDEX ix_ev_entity_time ON (entity_id, evidence_time DESC)
INDEX ix_ev_ingest      ON (ingested_at DESC)
INDEX ix_ev_source_native_id ON (source_id, source_native_id)
INDEX ix_ev_active      ON (entity_id, evidence_time DESC)
INDEX ix_ev_lineage     ON (evidence_lineage_id, ingested_at)
```

Append-only. Corrections do not overwrite; they append a new row that supersedes the prior. `evidence_time` and `ingested_at` answer different questions (what did the world do; when did we hear about it) and diverge constantly — the divergence is itself information (rule on backdating).

The lineage fields enforce rule 3 of §06 (no double counting). Every `evidence_event` carries an `evidence_lineage_id` that groups it with all derivative events from the same upstream signal: an EDGAR 10-K filing, a wire-service rewrite of that filing, a sell-side note quoting the filing, and a Twitter post linking to the wire all share one `lineage_id`. The `independence_score` is computed at ingest by comparing the new event's payload to prior events sharing the `lineage_id`; a low independence score down-weights the event's contribution to trace updates. The Resolver computes lineage at the point of ingest, not retroactively.

atomic_observations · the structured claim

```
TABLE atomic_observations (
  atomic_observation_id text PK
  evidence_event_id      text FK-evidence_events
  entity_id              text FK-entities
  tenant_id              text
  produced_by_layer      enum
                        -- deep_insight | financial_model |
                        -- synthetic_futures | crossvalidator
                        -- specific subagent identifier
  produced_by_agent      text
  produced_at            timestampz
                        -- wall-clock; rule 1 of §04
  observation_type        enum
                        -- earnings_print | guidance_revision |
                        -- capital_allocation | regulatory_finding |
                        -- competitor_move | management_change |
                        -- product_launch | macro_signal |
                        -- price_signal | structural_shift |
                        -- ...curated, extensible
  business_dimensions    enum_array
                        -- moat | demand | supply | capital |
                        -- regulation | fragility | growth |
                        -- technology | governance |
                        -- structural_future
  claim                  text
                        -- the structured claim text
  claim_payload           jsonb
                        -- typed structured form per observation_type
                        -- (e.g. for earnings_print: {beat_pct,
                        -- surprise_z, segment_breakdown, ...})
  source_reliability_at_emit numeric
                        -- snapshot of source's current_reliability
  novelty                 numeric
                        -- 1 - similarity to recent observations
  independence            numeric
                        -- 1 - correlation with prior evidence
                        -- on candidate-matching traces
  diagnosticity_claim     numeric
                        -- forward claim, scored at outcome
  estimated_materiality   enum
                        -- noise | low | moderate | material | regime
)
INDEX ix_ao_entity_time      ON (entity_id, produced_at DESC)
INDEX ix_ao_evidence_event   ON (evidence_event_id)
INDEX ix_ao_dimension        ON USING gin (business_dimensions)
INDEX ix_ao_material         ON (produced_at DESC)
                           WHERE estimated_materiality IN ('material','regime')
```

The base layer agents are the writers. One `evidence_event` typically yields multiple `atomic_observations` (an earnings call transcript may produce a `guidance_revision` observation, a `capital_allocation` observation, and a `competitor_move` observation). The `business_dimensions` array narrows the candidate trace search in step 04 of §05.

traces · the primary object

The most important table. Every trace is a living hypothesis.

```

TABLE traces (
  trace_id          text PK
  entity_id         text FK-entities
  tenant_id         text
  scope             enum
                    -- canonical or customer tenant
                    -- entity_specific | sector_wide |
                    -- peer_group | structural_market
  trace_type        enum
                    -- moat | growth_quality | capital_allocation |
                    -- management_integrity | demand_durability |
                    -- supply_resilience | regulatory_exposure |
                    -- technology_position | structural_future |
                    -- ad_hoc | required_condition |
                    -- ...open-ended, curated
  chapter_anchor    enum
                    -- thesis | deep_insight | financial |
                    -- synthetic_futures | threadweave_path |
                    -- required_path | open_hypotheses |
                    -- evidence_appendix
  hypothesis         text
  counter_hypotheses jsonb
                    -- the named claim, single sentence
                    -- [{label, prior_p, posterior_p,
                    --   evidence_basis, forecastable,
                    --   falsifier_if_forecastable}]
                    -- sums to 1 within tolerance

  -- Lifecycle (eight states, see §41)
  activation         enum
                    -- candidate | active | promoted | canonical |
                    -- dormant | reactivated | resolved | falsified
                    -- single field carrying full lifecycle position

  -- Current values, updated on every trace_snapshot commit
  posterior_probability numeric NULL
                    -- 0..1, most recent posterior
                    -- NULL only when activation='candidate' before
                    -- first scored snapshot
  magnitude          numeric NULL
                    -- interpretation depends on trace_type:
                    -- moat traces → strength
                    -- ad_hoc traces → expected business impact
                    -- structural_future → weight on this future
  persistence        numeric NULL
                    -- interpretation depends on trace_type:
                    -- moat traces → duration (years)
                    -- ad_hoc traces → relevance horizon (days)
                    -- structural_future → time-to-realisation
  materiality        enum
                    -- noise | low | moderate | material | regime
                    -- gates promotion (rule 5 of §06)
  confidence          numeric
                    -- system's confidence in the posterior itself
                    -- (a meta-measure: how much evidence backs
                    -- this posterior, distinct from the
                    -- posterior value)
  monitoring_priority numeric
                    -- 0..1, attention level set by parent updaters
                    -- (rule 9 of §06: top-down attention without
                    -- belief contamination). High priority lowers
                    -- the materiality threshold for triggering
                    -- snapshots and raises sweep position.

  -- Provenance and links
  created_at         timestamptz
  last_evidence_at   timestamptz NULL
                    -- timestamp of most recent contributing
                    -- evidence_event
  evidence_ids       text[] FK-evidence_events
                    -- contributing evidence
  prediction_ids     text[] FK-predictions
                    -- emitted from this trace
  related_trace_ids  text[] FK-traces
                    -- linked via trace_relationships
                    -- (cross-graph; not the parent-child
                    -- measurement edges, which live in
                    -- trace_hierarchy)

  -- Lifecycle metadata

  authored_by_agent  text
                    -- which layer agent created this trace
  retired_at         timestamptz NULL
  retired_reason     text NULL
                    -- stale_no_updates | falsifier_triggered |
                    -- replaced_by_better | category_consolidation
  superseded_by      text NULL FK-traces
                    -- when retired_reason='replaced_by_better'
)
CHECK (posterior_probability IS NULL OR jsonb_array_length(counter_hypotheses) > 0)

```

```
-- rule 6 of §06: no probability without -- counter-hypotheses INDEX ix_traces_entity_active ON ( entity_id )  
WHERE activation IN ('active', 'promoted', 'canonical', 'react INDEX ix_traces_entity_chapter ON ( entity_id ,  
chapter_anchor) WHERE activation IN ('promoted', 'canonical') INDEX ix_traces_dormant ON ( entity_id ,  
last_evidence_at) WHERE activation = 'dormant' INDEX ix_traces_type ON (trace_type, entity_id ) INDEX  
ix_traces_high_materiality ON ( entity_id , materiality) WHERE activation = 'active' AND materiality IN  
('material','regime') INDEX ix_traces_recent_evidence ON (last_evidence_at DESC) WHERE activation IN  
('active','dormant')
```

The CHECK constraint enforces rule 6 of §06 at the substrate level: a trace cannot carry a posterior probability without an explicit counter-hypothesis distribution.

The magnitude and persistence columns are interpreted by trace_type . For moat `traces` , magnitude is the strength of the moat (a strong moat returns a high magnitude) and persistence is its duration in years. For ad_hoc `traces` , magnitude is the expected business impact and persistence is the expected relevance horizon in days. The interpretation is documented in §40.

trace_snapshots · the history

```
TABLE trace_snapshots (  
  trace_snapshot_id      text PK  
  trace_id               text FK→traces  
  evidence_event_id      text FK→evidence_events  
  atomic_observation_id text FK→atomic_observations  
  entity_id              text FK→entities  
  tenant_id              text  
  produced_at            timestampz      -- wall-clock; rule 1 of §04  
  evidence_time           timestampz      -- when in world  
  produced_by_layer       enum  
  produced_by_agent       text  
  prompt_version          text  
  -- Prior values (immutable after commit)  
  prior_posterior         numeric NULL  
  prior_magnitude         numeric NULL  
  prior_persistence       numeric NULL  
  prior_counter_hyp       jsonb          -- snapshot of counter_hypotheses pre-update  
  prior_materiality       enum  
  prior_activation        enum  
  -- Posterior values (immutable after commit)  
  posterior_probability    numeric NULL  
  magnitude               numeric NULL  
  persistence             numeric NULL  
  posterior_counter_hyp   jsonb          -- normalised, sums to 1 within tolerance  
  materiality             enum  
  activation              enum  
  -- Bayesian arithmetic preserved for replay (rule 2 of §06: no hand-scored journey)  
  likelihood_ratios       jsonb          -- LR per counter-hypothesis (child updates)  
                                     -- NULL on parent-update snapshots  
  source_reliability_used numeric          -- source weight at the moment of update  
                                     -- NULL on parent-update snapshots  
  diagnosticity_used      numeric          -- diagnosticity claim used in arithmetic  
                                     -- NULL on parent-update snapshots  
  -- Parent-update provenance (NULL on child / evidence-driven snapshots)  
  update_kind             enum            -- evidence_driven | parent_update  
                                     -- evidence_driven: triggered by an  
                                     --   atomic_observation matching this trace  
                                     -- parent_update: triggered by a child  
                                     --   trace_snapshot moving, computed by  
                                     --   the parent updater  
  input_provenance        jsonb NULL      -- on parent_update snapshots, captures the  
                                     -- child snapshots consumed:  
                                     --   [{child_trace_id, child_snapshot_id,  
                                     --     child_posterior, child_magnitude,  
                                     --     child_persistence, edge_weight,  
                                     --     edge_semantic, updater_function_v}]  
                                     -- makes the parent update fully replayable  
                                     -- from the recorded inputs (rule 2 of §06)  
  -- Evidence packet  
  evidence_packet_ref     text            -- S3 pointer to full packet  
                                     -- NULL on parent-update snapshots  
  -- Evaluator verdicts (write-once on resolution)  
  retrospective_score     numeric NULL     -- Track 1: posterior calibration  
  magnitude_verdict       enum NULL       -- Track 2: under | accurate | over  
  persistence_verdict     enum NULL       -- Track 3: under | accurate | over  
  diagnosticity_verdict   enum NULL       -- as_claimed | over_claimed |  
                                     --   under_claimed | unverifiable  
)  
CHECK (posterior_probability IS NULL OR jsonb_array_length(posterior_counter_hyp) > 0)
```

```
INDEX ix_ts_trace_time ON (trace_id, produced_at DESC) INDEX ix_ts_entity_time ON (entity_id,  
produced_at DESC) INDEX ix_ts_evidence_event ON (evidence_event_id, trace_id) -- rule 3: idempotency  
INDEX ix_ts_unscored ON (produced_at) WHERE retrospective_score IS NULL INDEX ix_ts_layer ON  
(produced_by_layer, produced_at)
```

UNIQUE (evidence_event_id , trace_id, produced_by_layer) -- rule 3 of §06: -- no double counting

The UNIQUE constraint on (evidence_event_id , trace_id, produced_by_layer) enforces rule 3 of §06 at the substrate level: a given evidence_event can update a given trace at most once per layer. A wire-service rewrite of the same SEC filing is matched to the original evidence_event and discarded.

trace_relationships · *typed cross-graph relationships*

Trace_relationships are the typed graph edges that are not parent-child measurement edges (those live in trace_hierarchy). They are the Dot Connector's primary output: cross-trace patterns, contradictions, evidentiary support, causal candidates. The brief specifies the vocabulary; CrossValidator's structural check enforces it at write time.

```
TABLE trace_relationships (
  relationship_id      text PK
  entity_id           text NULL FK-entities -- null when relationship is cross-entity
  tenant_id           text
  source_trace_id      text FK-traces       -- the trace this relationship originates from
  target_trace_id      text FK-traces       -- the trace it points to
  trace_ids            text[] FK-traces     -- when the relationship is n-ary,
                                           -- includes both source and target plus others
  relationship_type    enum                 -- supports | attacks | causes | explains |
                                           -- contradicts | correlates_with |
                                           -- same_source_as | evidence_for |
                                           -- component_of (hierarchy mirror, see below)

  emitted_at          timestamptz
  sweep_id            text                  -- which Dot Connector sweep
  layer_evidence       jsonb                -- {layer: weight} contributions
  evidence_chain       text[]               -- contributing trace_snapshot_ids
                                           -- and atomic_observation_ids in order

  confidence           numeric
  forecastable         bool                 -- if true, a prediction is emitted
                                           -- citing this relationship
  retrospective_score  numeric NULL        -- Evaluator, write-once
)
INDEX ix_tr_entity_time      ON (entity_id, emitted_at DESC)
INDEX ix_tr_source_target    ON (source_trace_id, target_trace_id, relationship_type)
INDEX ix_tr_traces           ON USING gin (trace_ids)
INDEX ix_tr_sweep            ON (sweep_id)
INDEX ix_tr_forecastable     ON (entity_id) WHERE forecastable = true
```

The nine relationship types are typed to capture distinct graph semantics:

- supports. Source trace's posterior moving in one direction makes target trace's posterior more likely in the corresponding direction. Soft, not strict.
- attacks. Source trace's posterior moving up suggests target trace's posterior should move down. Used when an emerging mechanism undermines a canonical trace (e.g. AI migration agents attack data migration friction).
- causes. Asserts a causal candidate from source to target. Always weaker than calibrated causal claims; the architecture treats this as a hypothesis worth tracking, not a confirmed causal model.
- explains. Source trace, if true, accounts for some of the evidence currently supporting target trace. Used to prune redundant traces.
- contradicts. Source and target's posteriors cannot both be high; they make incompatible claims about the world.
- correlates_with. Posteriors move together historically without asserted direction or causation. Useful for the Dot Connector's pattern detection.
- same_source_as. Two traces drawing from the same evidence cluster; surfaces double-counting risk when they appear independent.
- evidence_for. Source trace's posterior is itself evidence on the target trace's hypothesis. Subtler than supports : the source's existence as a confirmed mechanism counts as evidence for the target's broader claim.
- component_of. Soft mirror of the structural trace_hierarchy parent-

child edge for use in cross-graph queries. The authoritative parent-child relationship lives in `trace_hierarchy` ; this enum value lets the Dot Connector traverse the same edge from the relationships graph when synthesising patterns.

`trace_hierarchy` · the parent-child measurement model

The substrate's representation of how parent `traces` compose from child `traces` . Each row is a directed edge from a parent trace to a child trace, carrying the measurement-model metadata the parent updater consumes. The hierarchy is a DAG, not a tree: a child can have multiple parents (data migration friction is a component_of switching-cost moat AND evidence_for an ad-hoc trace about implementation revenue).

```
TABLE trace_hierarchy (
  edge_id          text PK
  parent_trace_id  text FK→traces
  child_trace_id   text FK→traces
  entity_id        text FK→entities      -- both parent and child must agree
  tenant_id        text
  -- Measurement model
  weight           numeric               -- 0..1, child's contribution to parent
                                           -- measurement; weights across a parent's
                                           -- children should sum to 1 within tolerance
                                           -- (the parent updater renormalises if not)
  role             text                 -- human-readable role label, e.g.
                                           -- 'technical switching barrier',
                                           -- 'process switching barrier'
  semantic         enum                 -- substitute | complement | bottleneck
                                           -- determines how persistence aggregates
                                           -- (see §40 worked example):
                                           -- substitute: parent persistence = max child
                                           -- complement: parent persistence ≈ min child
                                           -- bottleneck: parent persistence dominated
                                           -- by this child specifically
  updater_function  text                 -- reference to the parent updater function
                                           -- in the parent updater registry; pinned
                                           -- per parent (one function per parent,
                                           -- shared across all its children)
  authored_at      timestampz
  authored_by_agent text
  superseded_at    timestampz NULL      -- edges are versioned; revising the
                                           -- measurement model appends a new edge
                                           -- and supersedes the prior
  superseded_by    text NULL FK→trace_hierarchy
)
INDEX ix_th_parent      ON (parent_trace_id) WHERE superseded_at IS NULL
INDEX ix_th_child       ON (child_trace_id) WHERE superseded_at IS NULL
INDEX ix_th_entity      ON (entity_id, parent_trace_id) WHERE superseded_at IS NULL
UNIQUE (parent_trace_id, child_trace_id) WHERE superseded_at IS NULL
```

-- no duplicate edges between the same -- parent-child pair

The Orchestrator is the sole writer. Edges are append-only with explicit supersession when measurement models are revised. The `updater_function` field references a registry of parent-updater implementations; the registry itself is versioned and pinned per edge so that historical parent snapshots replay correctly even after the updater function is later improved.

When an edge is created, CrossValidator runs a structural check: parent and child must share `entity_id` , the relationship cannot create a cycle in the existing hierarchy graph, and the weights across the parent's existing children must remain coherent (the parent updater renormalises if not, but a sustained pattern of unbalanced weight assignments is a code smell flagged in operator telemetry).

predictions · *the immutable forecast*

```
TABLE predictions (  
  prediction_id      text PK  
  entity_id         text FK-entities  
  trace_id          text FK-traces      -- emitting trace  
  trace_snapshot_id text FK-trace_snapshots -- emitting snapshot  
  trace_relationship_id text NULL FK-trace_relationships  
                                     -- non-null when prediction came from  
                                     --   a Dot Connector cross-trace pattern  
  
  tenant_id         text  
  emitted_at        timestamptz  
  emitted_by        text              -- Forecaster agent id  
  horizon_tier       enum              -- short | mid | long  
  horizon_days       int  
  resolve_by        timestamptz  
  prediction_type    enum              -- binary | discrete | continuous |  
                                     --   structured_decomposed  
  direction          enum NULL        -- up | down | flat | n/a  
  probability         numeric          -- immutable  
  baseline_probability numeric        -- counterfactual without the trace move  
  predicted_magnitude numeric NULL    -- when applicable  
  predicted_persistence numeric NULL  -- when applicable  
  mechanism          text  
  falsifier          text  
  mechanism_evidence_requirements jsonb -- typed pre-stated patterns  
  expected_price_reaction jsonb NULL  -- band for double-counting prevention  
                                     --   (rule 3 of §06)  
  verdict            enum NULL        -- written once on resolution  
  mechanism_check     enum NULL        -- written once on resolution  
  resolved_by_outcome text NULL FK-prediction_outcomes  
)  
INDEX ix_pred_resolve_by      ON (resolve_by) WHERE verdict IS NULL  
INDEX ix_pred_entity_emitted ON (entity_id, emitted_at DESC)  
INDEX ix_pred_trace          ON (trace_id, emitted_at DESC)  
INDEX ix_pred_horizon        ON (horizon_tier, resolve_by)
```

prediction_outcomes · *the resolution*

```
TABLE prediction_outcomes (  
  prediction_outcome_id text PK  
  prediction_id         text FK-predictions  
  entity_id             text FK-entities  
  tenant_id             text  
  observed_at           timestampz      -- when in world the resolution happened  
  ingested_at           timestampz      -- when the system observed it  
  resolution_evidence_id text NULL FK-evidence_events  
                                -- if a single evidence row resolves  
  resolution_source     enum            -- market_price | guidance_revision |  
                                -- regulatory_decision | filing_disclosure |  
                                -- transaction | merger_close |  
                                -- horizon_passed_no_evidence | other  
  
  realised_direction    enum  
  realised_magnitude    numeric NULL  
  realised_persistence  numeric NULL  
  observed_components   jsonb            -- structured: each component the prediction  
                                -- named, resolved separately ($15)  
  mechanism_observed    enum            -- confirmed | unconfirmed | contradicted  
  mechanism_evidence    jsonb            -- the evidence basis for the verdict  
  outcome_verdict       enum            -- correct | incorrect | partial | inconclusive  
  resolves_trace_snapshots text[] FK-trace_snapshots  
                                -- snapshots whose Track 1/2/3 verdicts  
                                -- this outcome will close  
  
  notes                 text NULL  
)  
INDEX ix_po_entity_observed    ON (entity_id, observed_at DESC)  
INDEX ix_po_prediction        ON (prediction_id)  
INDEX ix_po_unscored_calibration ON (ingested_at) WHERE NOT EXISTS (  
                                SELECT 1 FROM calibration c
```

WHERE c.derived_from_outcome = prediction_outcome_id)

Append-only. One outcome may resolve a single prediction directly and several open `trace_snapshots` simultaneously (an earnings beat resolves the earnings prediction and contributes to the `calibration` of every `trace_snapshot` whose posterior moved on the prior earnings expectation).

```

TABLE chapter_projections (
  projection_id      text PK
  entity_id         text FK-entities
  tenant_id         text
  produced_at       timestampz
  produced_by_run    text
  intent_assumption enum
  chapters          jsonb
  promoted_trace_ids text[] FK-traces
  reliability_per_horizon jsonb
  mechanism_confirm_rate jsonb
  forward_or_reverse enum
  reverse_target_state_id text NULL
  prev_projection_id text NULL FK-chapter_projections
  is_current        bool
)
INDEX ix_cp_entity_current      ON (entity_id, intent_assumption)
                                WHERE is_current = true
INDEX ix_cp_entity_version     ON (entity_id, produced_at DESC)
INDEX ix_cp_low_reliability    ON (entity_id) WHERE
                                (reliability_per_horizon->>'3y')::numeric < 0.50

```

Threadweave is the single writer. Promotion of a trace into promoted_trace_ids is gated by materiality (rule 5 of §06). The chapter document is the customer's primary read; it is what shows up in the Trace Feed (§33) when the user requests deep dive.

canonical_chapters · the prose authoring surface (new in v5.1)

```
TABLE canonical_chapters (
  chapter_id          text PK
  entity_id           text FK->entities
  chapter_type        text FK->chapter_type_registry
  version             int          -- monotonic per (entity_id, chapter_type)
  supersedes_version  int NULL     -- v(n-1)
  status              enum          -- draft | active | superseded | withdrawn
  authored_at         timestampz
  author_specialist_id text        -- DI/04, FM/01, ...
  prompt_version      text        -- pinned per §17
  model               text        -- model identifier
  content_md          text        -- prose body, markdown
  structured_outline  jsonb        -- per §06d schema; load-bearing for Check/06
  built_from          jsonb        -- {trace_snapshot_ids[],
                                   --   atomic_observation_ids[],
                                   --   prior_chapter_id}
  reasoning_notes     text NULL    -- diff narrative vs prior version
  authoring_mode      enum          -- seed | re_author
  trigger             enum          -- material_accumulation | scheduled |
                                   --   user_request | falsifying_outcome |
                                   --   bootstrap
  tenant_id           text        -- canonical or customer
)
INDEX ix_cc_active      ON (entity_id, chapter_type) WHERE status = 'active'
INDEX ix_cc_history     ON (entity_id, chapter_type, version DESC)
INDEX ix_cc_specialist  ON (author_specialist_id, authored_at DESC)
UNIQUE (entity_id, chapter_type, version)
```

Append-only. Status transitions: draft → active , active → superseded , active → withdrawn . No row is ever deleted; superseded rows remain queryable as the historical record of the system's narrative understanding.

The structured_outline jsonb is validated at the application layer (Pydantic / JSON schema) rather than as a CHECK constraint, because the outline references trace_snapshot_ids whose existence and consistency are verified by Check/06 (§06d) at commit time. CrossValidator's Check/06 is what binds prose to substrate.

chapter_type_registry · the typed catalogue (new in v5.1)

```
TABLE chapter_type_registry (
  chapter_type          text PK
  author_specialist_id  text      -- one specialist per chapter_type
  trace_types_in_scope  text[]    -- which trace_types feed this chapter
  material_accumulation_threshold jsonb -- {min_new_snapshots,
                                   --   min_posterior_shift,
                                   --   min_moderate_count}
  scheduled_refresh_interval_months int -- 6 default for slow chapter_types,
                                   --   3 for faster, 1 for regulatory_alert
  tier_assignment        enum      -- sonnet | opus
  peer_comparison_required bool    -- moat, endgame: true; leadership: false
  notes                 text
)
```

Edited only by schema migration. Specialists and the harness have read-only access. New chapter_types are proposed in writing, reviewed, and added by migration. Launch coverage: moat , capital , fragility , demand_durability , leadership , anomalies , sentiment , endgame , regulatory_exposure , structural_future . Open-ended; deferred discussion in §A5.

```
TABLE calibration (
  calibration_id      text PK
  entity_id          text NULL FK-entities -- null for system-wide rows
  source_id          text NULL FK-sources -- when row is per-source
  agent_id           text NULL           -- when row is per-agent
  trace_type         enum NULL          -- when row is per-trace_type
  horizon_tier       enum               -- short | mid | long
  track              enum               -- outcome | posterior | magnitude |
                                         -- persistence | mechanism | diagnosticity
  scope              enum               -- per_source | per_agent |
                                         -- per_entity_agent | system

  -- Aggregate scoring
  brier_mean         numeric NULL       -- for binary/discrete
  crps_mean          numeric NULL       -- for continuous
  posterior_calibration numeric NULL     -- Track 1
  magnitude_error_mean numeric NULL     -- Track 2: |predicted - realised|
  persistence_error_mean numeric NULL   -- Track 3
  diagnosticity_error numeric NULL
  mechanism_confirm_rate numeric NULL
  n_resolutions      int
  weight_adjustment  numeric            -- multiplier; bounded [0.25, 4.0]
  window_start       timestamptz
  window_end         timestamptz
  updated_at         timestamptz
  derived_from_outcome text NULL FK-prediction_outcomes
                                         -- which outcome triggered this row
  prev_calibration_id text NULL FK-calibration
)
INDEX ix_calib_source_track ON (source_id, track, horizon_tier, updated_at DESC)
INDEX ix_calib_agent_track ON (agent_id, track, horizon_tier, updated_at DESC)
INDEX ix_calib_entity_track ON (entity_id, track) WHERE entity_id IS NOT NULL
INDEX ix_calib_trace_type ON (trace_type, track) WHERE trace_type IS NOT NULL
```

The Evaluator is the single writer. Calibration rows are append-only. The chain `prev_calibration_id` → `calibration_id` is what makes weight history auditable per rule 3 of §04.

```

TABLE run_log (
  log_id                text PK
  workflow_id           text
  activity_id           text -- Temporal workflow run id
  parent_run_id         text -- Temporal activity id
  tenant_id             text NULL FK→run_log
  agent_kind            enum -- canonical | customer
  agent_id              text
  entity_id             text NULL
  emitted_at            timestampz
  prompt_version        text -- pinned prompt hash
  model                 text -- claude-opus-4 | claude-sonnet-4 | ...
  input_ref             text -- S3 pointer; large inputs not inline
  input_hash            text -- sha256 for replay determinism check
  output_ref            text -- S3 pointer
  output_hash           text
  tokens_in             integer
  tokens_out            integer
  cost_usd_micros       bigint
  latency_ms            integer
  outcome_kind          enum -- success | tool_error | timeout |
                             -- refusal | schema_violation |
                             -- quota_exceeded
  emitted_rows          jsonb -- {table: [row_ids...], ...}
                             -- the writes this invocation produced
)
INDEX ix_runlog_workflow ON (workflow_id, emitted_at)
INDEX ix_runlog_agent_time ON (agent_id, emitted_at DESC)
INDEX ix_runlog_entity_time ON (entity_id, emitted_at DESC) WHERE entity_id IS NOT NULL
INDEX ix_runlog_prompt_ver ON (prompt_version, agent_id, emitted_at DESC)
INDEX ix_runlog_failed ON (emitted_at) WHERE outcome_kind <> 'success'

```

Storage is split: row metadata in Postgres for the indexes, large input/output blobs in S3 addressed by content hash, full-text searchable copy in OpenSearch for incident investigation. The same content hash on input means the same prompt version on the same model produced the same output (or the same error), which is what makes the harness contract on idempotency (§17) auditable.

Views, the agent read surface

No agent touches SQL. Every agent reads the substrate through a typed view, a thin read layer that assembles exactly the context needed, nothing more. This is where token budgets are enforced.

The v5 views:

- `entity_context_view(entity_id)` : current trace graph (active and dormant), recent `atomic_observations`, recent `trace_snapshots`, recent `trace_relationships`, latest `chapter_projection`.
- `trace_context_view(entity_id, trace_id)` : a single trace's full state and recent snapshot history, used by layer agents in step 07 of §05.
- `trace_match_view(entity_id, observation_id, dimensions)` : the candidate trace search for step 04 of §05; returns active, dormant, and canonical `traces` matching the dimensions of the observation.
- `prediction_resolution_view(prediction_id)` : read by the Evaluator at outcome time; contains prediction text, mechanism_evidence_requirements, and resolution-window evidence — but not the originating reasoning chain (Evaluator independence, §15).
- `chapter_projection_view(entity_id)` : the customer-facing read used by the Trace Feed (§33).
- `dormant_reactivation_view(entity_id)` : dormant `traces` with recently-matching

observations, used by the Orchestrator’s step-14 activation review (rule 4 of §06: gated against keyword-only reactivation).

§22 Access matrix, who reads and writes what

The substrate’s discipline is only as strong as the access pattern that enforces it. Every table has a single semantic writer (a “pen-holder”) and a defined set of readers. CrossValidator surfaces contradictions but does not write semantic state. The Orchestrator alone commits `trace_snapshots`, `trace_relationships`, and `canonical_chapters` to the canonical core. The Evaluator alone writes outcome verdicts and calibration. The `chapter_type_registry` is read-only at runtime; only schema migration writes it.

Read/write matrix R = read, W = write (semantic), W* = write via typed proposal that the Orchestrator commits, W = write-once retrospective field only, W** = write-once verdict and calibration fields only, em-dash = no access. Column headers below abbreviate the twelve source-of-truth tables, `trace_hierarchy`, and calibration: ent. = entities, src. = sources, ev. = `evidence_events`, ao. = `atomic_observations`, tr. = traces, ts. = `trace_snapshots`, trel. = `trace_relationships`, th. = `trace_hierarchy`, pred. = predictions, po. = `prediction_outcomes`, cp. = `chapter_projections`, cc. = `canonical_chapters`, ctr. = `chapter_type_registry`, cal. = calibration.

Agent	ent.	src.	ev.	ao.	tr.	ts.	trel.	th.	pred.	po.	cp.	cc.	ctr.	cal.
Resolver	W	R	W	—	—	—	—	—	—	—	—	—	—	R
Orchestrator	R	R	R	R	W	W	W	W	—	—	—	W	R	R
Deep Insight (6)	R	R	R	W	W*	W*	—	W*	—	—	—	W*	R	R
Financial Model (2)	R	R	R	W	W*	W*	—	W*	—	—	—	W*	R	R
Synthetic Futures (4)	R	R	R	W	W*	W*	—	W*	—	—	—	W*	R	R
CrossValidator	R	R	R	R	R	W* (rej)	—	R	—	—	—	R	R	R
Dot Connector	R	R	R	R	R	R	W	R	—	—	—	R	R	R
Threadweave	R	R	R	R	R	R	R	R	R	R	W	R	R	R
Forecaster	R	R	R	R	R	R	R	R	W	—	—	R	R	R
Evaluator	R	R	R	R	W**	W***	W***	R	W***	W	—	R	R	W
Outcome ingest	R	R	R	—	—	—	—	—	—	W	—	—	—	—

R read · W semantic write · W* write via typed proposal the Orchestrator commits · W** write-once retrospective field · W*** write-once verdict/calibration fields · — no access. Columns: ent.=entities, src.=sources, ev.=evidence_events, ao.=atomic_observations, tr.=traces, ts.=trace_snapshots, trel.=trace_relationships, th.=trace_hierarchy, pred.=predictions, po.=prediction_outcomes, cp.=chapter_projections, cc.=canonical_chapters, ctr.=chapter_type_registry, cal.=calibration.

Write-discipline notes • Base-layer specialists propose trace creates and `trace_snapshot` commits via typed tool call (W*). The Orchestrator commits with weight applied. CrossValidator writes to `trace_snapshot` only when it records an unresolved disagreement, never a semantic update; it never writes a semantic posterior. • The Resolver is the only writer of entities and `evidence_events`. No other agent touches identity or raw evidence. • Source-reliability updates flow through calibration: the Evaluator’s writes to calibration after each `prediction_outcome` cause the source’s `current_reliability` field to be recomputed by the calibration projection (in practice a derived view; the underlying source row is updated atomically with the calibration row so the `prev_calibration_id` chain is always consistent). • Evaluator’s write to traces (W**) is restricted to

retrospective fields that close out resolved or falsified traces; it does not touch posterior, magnitude, persistence, or `counter_hypotheses` (those are immutable on the `trace_snapshot` once committed). • Evaluator's write to `trace_snapshots` (W***) is restricted to `retrospective_score`, `magnitude_verdict`, `persistence_verdict`, and `diagnosticity_verdict`. The prior, posterior, evidence packet, and counter- hypothesis distribution at emission are immutable. • Replay engine has read-write on traces (the only writer besides Orchestrator) because trace state is reconstructed from the `trace_snapshot` history. The replay path is the verification mechanism for trace state; nothing outside the replay engine can claim authority over current trace values. • Threadweave is the sole writer of `chapter_projections`. Promotion (step 12 of §05) into `promoted_trace_ids` is gated by materiality (rule 5 of §06) and trace strength (rule 8 of §06).

- Layer agents propose `trace_hierarchy` edges (W*) when they author parent or child traces with composition relationships; the Orchestrator commits the edge after CrossValidator's structural check (no cycles, weight coherence). The parent updater function reference on each edge is pinned at commit; revisions are append-only with explicit supersession. Rule 9 of §06 (no parent overwriting child beliefs) is enforced at this layer: the parent updater reads child snapshots only and never writes to children.
- Layer specialists propose canonical chapter writes (W*) — the prose body, the structured outline, and (in seed mode) the proposed traces and `trace_snapshots` that materialise from the outline. The Orchestrator commits the chapter row after CrossValidator's Check/06 (§06d) passes. In re-author mode, the prior active row is moved to superseded in the same transaction. The `chapter_type_registry` is read-only at runtime; only schema migration writes it, ensuring the registry cannot be mutated by an agent during operation.

What this enforces The matrix enforces, in code, that:

1. The originating agent of a prediction never reads its own outcome (Evaluator independence, §15).
2. No agent except the Orchestrator can commit semantic state to the trace graph; layer agents can only propose.
3. The replay path is the verification mechanism for trace state; nothing outside the replay engine can claim authority over current values.
4. Traces and `trace_snapshots` have exactly one semantic writer (the Orchestrator). One ledger per material event, one history per trace, one substrate of truth.
5. Source-reliability calibration cannot be silently overridden: every weight change is a new calibration row with `prev_calibration_id`, fully auditable.
6. Chapter promotion is gated: a trace cannot enter the customer-visible chapter document without clearing materiality and strength thresholds (rules 5 and 8 of §06).

Part IV · The operation How it runs, what to ship, what is still open. A worked example traces one signal from arrival to scored outcome through the 14-step flow. The launch path is preserved from earlier versions with v5-specific phasing for the trace substrate. Open questions surface things not yet decided. A small list of operational concerns deliberately out of scope.

PART IV

The operation

How it runs, what to ship, what is still open. A worked example traces one signal from arrival to scored outcome. The launch path is preserved with the cold-start named as the canonical-chapter seeding bootstrap (§24); the open questions (§25) carry the strategic decisions v6 deliberately leaves open.

§23 The loop, a worked example end to end

Trace ID	Hypothesis (compressed)	trace_type	posterior_probability	magnitude	persistence	materiality
T-481	Long-term supplier partnership is a durable competitive advantage	moat	0.78	0.74 (strength)	8 yrs	material
T-512	Capital allocation is disciplined and value-accretive	capital	0.71	0.66 (strength)	5 yrs	material
T-617	Smooth reported results conceal supplier-concentration fragility	fragility	0.42	0.55 (severity)	3 yrs	material
T-9101	Regulatory enforcement risk in the supply chain (dormant)	regulatory	0.30	0.40 (severity)	2 yrs	watch

TIMELINE → ELAPSED FROM SIGNAL ARRIVAL · LOG SCALE

T-512 capital 0.71 → 0.58



FIGURE 11 · WORKED-EXAMPLE TIMELINE. ENTITY ENT-48221, SIGNAL AT T+0, INLINE LOOP CLOSING IN 13 SECONDS, SCHEDULED SWEEP AT +6 HOURS, OUTCOME RESOLVES AT DAY 71. THE FULL ARCHITECTURAL CYCLE IN ONE TRACE.

Mid-cap European industrial, ENT-48221. Already bootstrapped, with an existing trace graph. A regulatory disclosure lands: the company is switching its primary supplier due to an enforcement action on the prior one. What happens next, from evidence ingest to outcome resolution seventy-one days later, routed through the 14-step flow.

Pre-existing trace graph at T-0 ENT-48221 carries 14 active traces at the time the disclosure arrives. The four traces that will move in this example:

Steps 1 to 8, the inline loop, T+0 to T+13s

T Step Owner Action

T Step Owner Action

Steps 9 to 12, the scheduled loop, T+6h to T+24h At +6h+12s the Dot Connector sweep begins for ENT-48221. It reads the entity's full trace graph (now with five fresh `trace_snapshots`), recent `atomic_observations`, recent CrossValidator disagreements, and crucially the `trace_snapshots` for three peer entities that received related supplier-disruption evidence over the past 30 days.

The cross-trace synthesis recognises a pattern: the supplier disruption at ENT-48221 overlaps with sector-wide regulatory tightening visible in the peer entities' `trace_snapshots`. At +6h+18s, `trace_relationship` TR-7a1 is written:

relationship_type: co_occurring (subtype: enforcement_driven_capex_redirection) trace_ids: [T-481, T-9101, peer.T-481, peer2.T-481, peer.T-9101] evidence_chain: [`trace_snapshots` from ENT-48221 plus three peers, 30-day window] forecastable: true confidence: 0.74

At +6h+22s, step 09 fires: T-481's snapshot crosses the prediction threshold (materiality=material, posterior change >0.2). Forecaster emits prediction P-331a citing both T-481's snapshot and the `trace_relationship` TR-7a1: direction=down, probability=0.34, horizon=90 days, mechanism="enforcement-driven supplier change forces capex redirection and margin compression on next earnings cycle", falsifier="reaffirmed guidance at next

earnings call without margin commentary on supplier costs”, expected_price_reaction band [-3%, -1%] (rule 3 of §06: pre-stated band prevents double-counting later price moves as fresh evidence).

At +24h, step 12 fires. Threadweave’s nightly chapter projection runs. The five updated traces plus TR-7a1 all clear materiality (rule 5 of §06); they promote into the entity’s `chapter_projections` row. The 3y path weight shifts 4% toward “bounded growth”; the chapter document published to the customer’s Trace Feed (§33) shows the promoted traces in the moat, capital_allocation, and structural_future chapters, with the cross-entity sector pattern surfaced as a top-line `trace_relationship`.

Step 10, outcome and scoring, T+51d to T+71d Day 51 (T+51d). A peer entity flags supply-chain disruption in its quarterly call. ENT-48221 stock declines 4.2% on the day; market-adjusted return -3.1%, sector-adjusted -2.4%, residual -1.8%. An `atomic_observation` is produced from the price-derived `evidence_event` with the decomposition fields attached. The unexpected_residual against P-331a’s pre-stated band (-2.0% midpoint) is +0.2%, well inside the band: the move is approximately in

line with what the system already expected if T-481’s posterior is correct. No new `trace_snapshot` is generated for T-481 from this price move (rule 7 of §06: no update from absence; rule 3 of §06: no double counting — the price move is already inside the pre-stated band). Other traces’ snapshots reflect the realised market move only where the move’s residual is meaningful evidence for those traces.

Day 63 (T+63d). Guidance revised down by 6%. Evidence_event written, `atomic_observations` produced, `trace_snapshots` committed for the affected traces (the prediction P-331a does not commit a fresh `trace_snapshot` until step 11; outcome ingestion is a separate path).

Day 71 (T+71d). Outcome ingestion writes `prediction_outcome` PO-331a. Evaluator runs (step 11).

Outcome verdict on P-331a: correct . Brier 0.088. Direction matched, magnitude was within the predicted range.

Mechanism verdict ensemble: five independent Evaluator invocations dispatch, each receiving only the prediction text, the pre-stated mechanism_evidence_requirements, and the resolution-window evidence (the 10-Q, earnings call transcript, sector news in the window). None receives the originating reasoning chain. Three return `mechanism_confirmed` ; two return `mechanism_unconfirmed` . Final verdict: `mechanism_confirmed` with ensemble agreement 3/5, ambiguity flag.

Trace_snapshot verdicts on the snapshots committed at T+13s:

Step 11, calibration updates DI-04’s source-and- `trace_type` -weighted reliability tightens on Track 1 (the moat update was well-calibrated). DI-05’s weight loosens slightly on Track 2 (the over-optimisation magnitude was overstated). The Dot Connector’s enforcement_driven_capex_redirection cross-trace pattern gains calibration weight for sector- correlation forecasts. The source edgar_xbrl retains its current_reliability of 0.93. The Forecaster’s expected_price_reaction band is unchanged (the realised move was inside the band).

Steps 13 and 14, continuous Step 13 (trend) runs continuously as queries against the `trace_snapshot` history. Customers querying T-481’s trajectory see the posterior path 0.78 → 0.46 → resolved confirmed. No agent ever wrote a “trend” value (rule 1 of §06).

Step 14 (sleep). Two weeks after resolution, the Orchestrator’s activation review marks T-481 as activation='resolved' (the prediction confirmed and no new contradicting evidence has arrived), but the trace and its full snapshot history remain in the substrate. T-9101 stays active because its hypothesis (supplier- concentration risk) remains relevant to ongoing observations. T-617 stays active with reduced materiality.

Inline loop, evidence arrival to committed snapshots: under 14 seconds. Sweep loop, snapshots to chapter promotion: typically under 24 hours. Learning loop, prediction to scored outcome with per-track verdicts: 71 days in this example. Each timescale matters for a different part of the product; the last is what makes the product compound.

§24 Launch scope, what to build, with the cold-start budget AMENDED

The architecture is designed to scale to the full six-layer stack across hundreds of entities. It does not obligate shipping with all of it. The critical path is the shortest route to a closed loop on a narrow universe; everything after is widening.

Launch universe. Approximately 100 listed EU industrial mid-caps. Enough to see patterns; small enough to hand-review; market data is clean and cheap; outcome resolution is fast. **Launch agent set.** Nine agents, not seventeen: Resolver, Orchestrator, Deep Insight (DI/01 Supply only at launch), Financial Model (FM/01 Capital + FM/02 Forecasting), Synthetic Futures (SF/01 Demand only), CrossValidator (with Check/05 and Check/06 active from day one), Dot Connector (stripped — nightly sweep, clearest forecastable cases only), Forecaster, and Evaluator. Threadweave is post-launch.

The 90-day path is unchanged in its phase structure (weeks 1–2 identity and substrate; weeks 3–4 bootstrap canonical chapters and seed corpus; weeks 5–6 live loop with snapshots in a drawer; weeks 7–8 the loop closes as outcomes begin resolving; weeks 9–12 the trace substrate hardens to canonical). The week-1 substrate stand-up is unchanged from v5.1: the same twelve source-of-truth tables plus the auxiliary substrates, with `tenant_id` everywhere. v6 adds no tables. What v6 changes in the launch window lives in weeks 3–4, where the cold-start phase (§17a) is now the named bootstrap that seeds the canonical chapters and their candidate traces.

The cold-start budget (new in v6)

v5.1's launch path described the continuing loop but did not surface the cost of the bootstrap. v6 makes it an explicit line item, because every one of the 100 launch entities requires a cold-start (§17a) before it has a trace graph for the loop to update. The v4.1 reference implementation gives concrete figures: a full per-ticker cold-start ran in roughly four-to-six hours of LLM-dispatch wall-clock and cost on the order of \$20–40 per ticker on the API with prompt caching; the ten deep-dive briefs alone took about ten minutes of parallel Sonnet dispatch at roughly \$10–15.

At an indicative \$30 per cold-start, the 100-entity universe is roughly **\$3,000 in one-time cold-start cost**, plus the continuing calibration cost over weeks 1–7 before outcomes start resolving. This is a small line item against the launch budget, but it belongs in §24 as a sub-section rather than buried in coverage economics, because it is front-loaded: the cold-start spend lands in weeks 3–4 (the bootstrap), not spread across the quarter. The weeks-3–4 gate is unchanged in substance — ship when the trace schema stops changing for seven consecutive days, the rejection-and-reroute rate is under 25% of proposals, and layer-agent ensemble agreement on candidate trace authoring stabilises above 0.65 — but the bootstrap that drives those metrics is now named as the cold-start phase and budgeted accordingly.

The MVP boundary, restated. v6 inherits v5.1's binding commitment: there is no launch path that defers predictions, outcomes, and calibration to a "phase 2." The longitudinal loop is the moat; deferring it produces a current-state trace graph contaminated by hindsight, which any competent reviewer can build in a weekend with a

different stack. The cold-start phase v6 adds is explicitly the *bootstrap* for the loop, not a substitute for it. A v6 that shipped the cold-start and stopped would be the exact failure mode the reference documents warn against. The cold-start gives the customer something to read on day one; the loop closing in weeks 7–8 is what the architecture is for.

§25 Open questions, what still needs decisions AMENDED

The v5.1 open questions (materiality thresholds, horizon tiers, bootstrap chronology, calibration-aggregation cells, selection bias) carry forward unchanged. The v4.1 analysis surfaced several more. v6 takes the position that the strategic ones — the ones that touch core machinery — should be **decided on the architecture's own terms, not settled by importing a trial's choices**. One agent-layer question is resolved in v6; the rest remain open and are flagged here rather than answered in prose.

Resolved in v6

D/07 · Deep Insight decomposition. RESOLVED — committed. The six DI specialists are realised as chapter-keyed agents over the fourteen-chapter dragnet. The base layer is the ten company-facing chapter agents (the deep-dive authors); the four meta-chapters score the run rather than the company and sit in the priors/validation tier — ST-1 Population Priors and ST-2 Knowability feed in as priors, ST-11 System Coherence is the CrossValidator, ST-13 Thinking Quality is the Evaluator (§07, §08, §06c). The agents author; the shared scoring, validation, and calibration layer is chapter-agnostic and unchanged. What remains open is finer-grained only — sub-chapter granularity within an ST chapter (§A4) and where DI/06 Sentiment sits (D/09 below).

Strategic questions surfaced by v4.1, left open

D/06 · Customer-facing chapter count. The canonical-chapter authoring layer uses the fourteen dragnet chapters (§06c). Whether the customer-facing `chapter_projections` surface stays at its v5.1 form, expands to fourteen, or collapses many-to-one is a Threadweave/read-surface decision (core), left open. v6 declines to import a collapse rule from the v4.1 trial.

D/08 · Chapter-level scoring. How a chapter-level score is derived from trace posteriors — for the customer who asks "how does the system rate Company X's moat" — is a real gap, but it belongs to the core Threadweave design (§13). v4.1's damped chapter-rollup is a candidate method, noted but not adopted.

D/11 · Honest non-scoring in core trace state. The `INSUFFICIENT` convention is committed at the deep-dive-brief layer (§40a). Whether it should propagate into a `scoring_status` field on `trace_snapshots` — letting a trace itself hold an explicit insufficient-evidence state distinct from a wide confidence band — is a change to core scoring, left open.

New canonical-chapter-layer questions

D/09 · Sentiment's home in the dragnet. DI/06 Sentiment has no v4.1 ST-chapter ancestor. v6 sites it provisionally in ST-1 Population Priors, but Sentiment is conceptually distinct and may warrant its own `chapter_type`. The decision waits on whether sentiment traces calibrate well enough to justify it.

D/10 · Structured-outline patterns for the four meta-chapters. Ten of the fourteen chapters are company-facing and inherit the P16 Ch.11 checklist as their seed template. The four meta-chapters (ST-1, ST-2, ST-11,

ST-13) score properties of the run rather than the company, and v4.1 provided no structured-outline pattern for them. These must be authored before the meta-chapters can be seed-authored rather than hand-built.

§26 Operational concerns deliberately out of scope

This document is an architecture document, not an operations playbook. Several large topics are absent because they belong elsewhere; naming them here is the architecture's commitment to not having silently overlooked them.

Deployment topology. Cloud provider selection, region distribution, multi-region failover, edge presence. The architecture is cloud-agnostic in its commitments and provider-specific in its operations. Decisions are deferred to operations.

Infrastructure-as-code. Terraform, Pulumi, Crossplane, in-house tooling. The contracts in §17 must hold across whatever infrastructure expresses them; the expression itself is operational.

Observability and alerting. Dashboards, SLO definitions, on-call rotations, paging policy. The architecture commits to making the substrate observable (every commit logged, every agent invocation pinned to prompt version) but does not specify how operators consume that observability.

SOC 2 / ISO 27001 / regional compliance. The security primitives in §18 are designed to make these certifications achievable. Pursuing them is a commercial decision tied to enterprise sales motion.

Penetration testing, bug bounty, red-team programs. Operational. The security model in §18 is the architectural commitment; the testing of that model against adversarial actors is operational.

Customer onboarding and support tooling. How customers actually start, what kind of help-desk interface their analysts have, how billing disputes are handled. All commercial-operational, not architectural.

Vendor selection above the architecture layer. S&P Capital IQ vs FactSet, Auth0 vs Clerk, AWS KMS vs GCP KMS. The architecture commits to capabilities (normalised financial history, OIDC, managed key store) and is deliberately silent on vendor identity.

Cost optimisation playbook. Token budgets per agent class, infrastructure right-sizing schedules, spot vs reserved capacity allocation. The metering discipline of §31 makes optimisation possible; specifying optimisation is operational.

Localisation and i18n beyond data acquisition. The data acquisition layer handles non-English filings via translation (§19). UI localisation, region-specific compliance text, local payment methods are commercial-operational.

The architecture earns its commitments by being silent on the things it does not commit to. Naming them as out-of-scope is the discipline that makes the in-scope commitments mean what they say.

Part V · The sealed client layer The customer-facing extension surface. Reads the canonical trace graph freely. Writes nothing back. The seal is a runtime contract enforced by tenant isolation, audited continuously, and named honestly. v5 makes the seal a contract over trace operations: customers can author tenant-private traces, propose tenant-private `trace_snapshots`, and configure tenant-specific chapter projections, all without any path back to the canonical core.

The sealed client layer

The customer-facing extension surface. Reads the canonical trace graph freely, writes nothing back. The seal is a runtime contract enforced by tenant isolation, audited continuously, and named honestly. Unchanged in v6.

§27 The sealed client workbench, additive tenant-isolated

Some customers want to extend the canonical system with their own logic: house theses, proprietary scoring rubrics, internal signals, risk overlays, proprietary trace types. They do not want to share, cannot afford to leak, and would not codify inside a vendor's namespace. The client workbench exists for those customers. It is self-service, tenant-isolated, and additive: customers build their own agents alongside the canonical engine without ever modifying it, without exposing their logic, and without any path back to the canonical core.

The workbench is built on top of the canonical six-layer engine and the trace graph it produces. It inherits the canonical view of each entity (the active trace graph, the `trace_snapshot` history, the `trace_relationships`, the predictions, the `prediction_outcomes`, the `chapter_projections`) in read-only form. From there, the customer creates additional agents, authors private chapter formats, authors tenant-private traces, produces tenant-private `trace_snapshots`, configures Threadweave parameters within their namespace, and maintains a private learning loop scored against customer-defined outcomes. Everything they build is theirs. Nothing they build crosses back.

The customer's logic does not alter the canonical system. Customer agents run alongside canonical agents, not in place of them. The canonical Threadweave continues to run for every entity, unchanged; what the customer can do is configure their own parameter overrides (horizon weights, competitor cohorts, sector tilts, trace-type weight overrides) that produce a customer-specific `chapter_projection` derived from the same canonical engine. Canonical calibration learns from canonical predictions and canonical `trace_snapshots` only. The canonical system is unaware of the customer layer's existence by design.

Sealing is enforced as a tenant-isolation contract in the runtime, not as an architectural impossibility. Customer agents are real code running in customer namespaces, behind isolation boundaries that the rest of the system does not have a path through. There is no API, no internal queue, and no shared write surface that lets a customer agent reach into canonical state. The seal is in the access matrix, the schema partitioning, the queue keys, and the deployment topology. It is not unbreakable in principle; it is unreachable in practice. We name this honestly because architectural honesty is the precondition for the seal mattering at all.

§28 The workbench, what customers can build

The workbench surfaces six classes of capability. Each is sealed in the runtime; together they make the canonical system genuinely extensible without compromising the moat.

Custom agent definition. Customers author agent system prompts, tool configurations, validation rules, and parameter overrides within their tenant namespace. The agent runs against the canonical trace graph (read-only) and writes to tenant-scoped tables. Canonical agents do not see these.

Custom trace authoring. Customers can author tenant-private traces that supplement the canonical trace graph for any entity. A custom trace carries the same schema as a canonical trace but is scoped to the tenant. Custom- trace updates run through the same 14-step flow (priors, counter-hypotheses, posterior, magnitude, persistence, falsifier) but the snapshots are tenant-private. Canonical agents do not see custom traces. Customers can read the canonical trace graph for any entity they have access to, supplement it with their own traces, and run their own analysis on the union.

Custom `atomic_observation` production. Customer-defined agents can decompose tenant-private evidence streams into tenant-scoped `atomic_observations`. The canonical Orchestrator does not dispatch on these; the canonical Dot Connector does not include them in its sweep. Customer agents can produce their own `trace_relationships` from these observations.

Custom prediction. Customer-authored predictions emitted from tenant-private traces live in tenant-scoped tables. Canonical Threadweave does not consume them. Canonical Evaluator does not score them. Canonical calibration does not learn from them.

Custom outcome and learning. Scored against customer-defined outcome streams. Customer Evaluator runs in the tenant namespace. Customer calibration is computed and stored per tenant. A customer agent's calibration trajectory has no influence on the weight canonical applies to its own equivalent agent.

Custom Threadweave parameter overrides. Customer-specific horizon weights, competitor cohorts, sector tilts, reliability shapes, and trace-type weight overrides are tenant-scoped configuration. The canonical Threadweave engine reads these only when running for that tenant; canonical Threadweave's default behaviour is unchanged for every other tenant and for the canonical-only view.

A worked customer example A long-only equity fund subscribes to the EU industrial mid-cap universe. Their analyst extends the canonical layer with three additions:

1. A custom trace on each tracked entity for "supply-chain political-risk exposure," with `trace_type` = 'ad_hoc', a tenant-defined dimension (`political_risk`), and an expected-evidence packet they have specified (sanctions exposure, geographic concentration of suppliers, lobbying expenditure on relevant trade policy).
2. A custom Deep Insight specialist XX/01 ESG-screen that runs after every canonical inline loop and writes tenant-private `atomic_observations` and `trace_snapshots` about ESG-screen criteria the canonical layer does not track.
3. A custom Threadweave override that downweights the long-horizon reachable states of any entity scoring below the fund's ESG threshold.

All three are tenant-private. The canonical trace graph for these entities continues to update for every other customer using the canonical engine; this customer's view of the same entities adds their custom traces and their custom chapter projections on top.

§29 The seal, what never crosses the boundary

Client workbench · additive logic, sealed outputs Inherits the canonical entity trace read-only. Customer-built agents run alongside canonical, never replace it.

T H E S E A L · B L O C K E D No traces, no `trace_snapshots`, no predictions, no calibration crosses from the client back to canonical. All client-side outputs stay tenant-private; canonical learning is unaffected by anything inside the seal.

TIER 1 · CANONICAL CORE · SHARED · THE MOAT

StahlTrace canonical · the six-layer learning engine entities · sources · `evidence_events` · `atomic_observations` · traces · `trace_snapshots` · `trace_relationships` · predictions · `prediction_outcomes` · chapter_proje

The client layer reads the canonical trace graph freely. Nothing written inside the seal feeds canonical calibration, predictions, or chapter projections. Canonical and client learning loops remain independent.

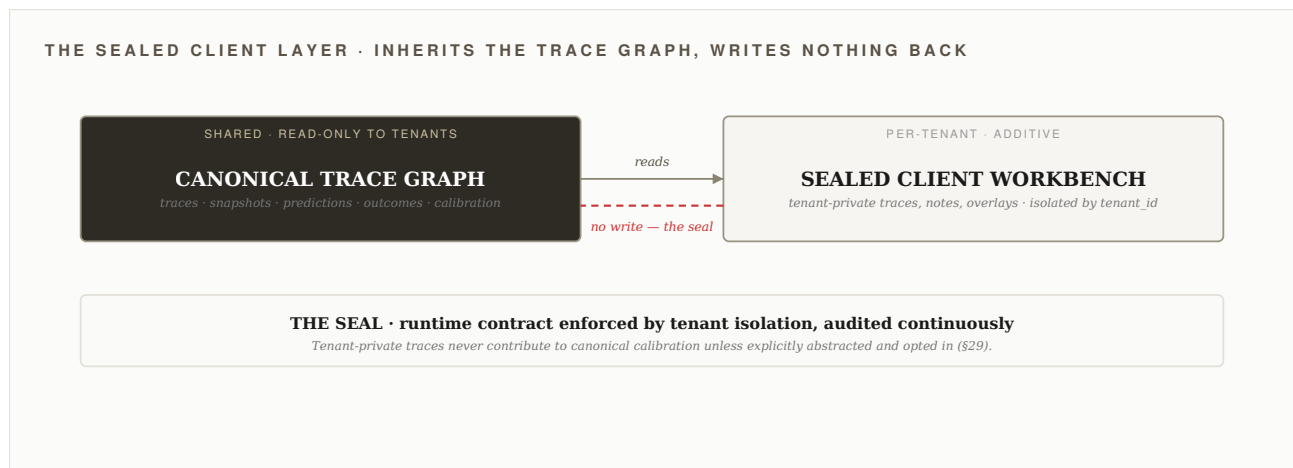


FIGURE 4 · THE SEALED CLIENT LAYER, INHERITING THE CANONICAL TRACE GRAPH WITHOUT FEEDING BACK INTO IT. THE FOUR CLIENT-SIDE SURFACES (CHAPTERS, TRACES, PREDICTIONS, LEARNING LOOP) ARE TENANT-PRIVATE; THE SEAL BLOCKS EVERY BACK-FLOW.

The seal is a runtime contract enforced by tenant isolation. The following things never cross from customer namespaces back into canonical, not because the architecture makes it impossible in principle, but because no path, no API, no shared write surface exists in the running system that would let it happen. The seal is unreachable in practice, audited continuously, and named honestly.

What is sealed Customer `atomic_observations`. Tenant-private evidence streams produce tenant-scoped `atomic_observations`. Canonical agents have no read path to tenant-scoped rows. The canonical Orchestrator never dispatches on them; the canonical Dot Connector never includes them in its sweep.

Customer traces and `trace_snapshots`. Tenant-private traces live in trace rows scoped by `tenant_id`. The `trace_snapshots` that move them live in `trace_snapshots` rows scoped by `tenant_id`. The `trace_relationships` associated with them live in tenant-scoped rows. Canonical agents do not read tenant-scoped trace rows. Canonical Dot Connector does not synthesise across them.

Customer predictions and `prediction_outcomes`. Customer-authored predictions emitted from tenant-private traces, and the `prediction_outcomes` that resolve them, live in tenant-scoped tables. Canonical Threadweave

does not consume them. Canonical Evaluator does not score them. Canonical calibration does not learn from them.

Customer `chapter_projections`. Tenant-specific `chapter_projections` derived from tenant-private traces are tenant-scoped. The customer's analyst sees their own chapter document; canonical operators do not.

Customer agent prompts and configurations. Agent system prompts, tool configurations, parameter overrides, validation rules, and customer-proprietary reasoning patterns live in a tenant-private prompt and config store. Canonical operators do not have read access; tenant administrators can read and write within their tenant.

Customer Threadweave parameter overrides. Customer-specific horizon weights, competitor cohorts, sector tilts, reliability shapes, and trace-type weight overrides are tenant-scoped configuration.

Customer-derived structured outputs. Reports, exports, dashboards, and downstream feeds derived from customer logic stay in the tenant namespace. Canonical never aggregates across tenants for any output that includes customer-derived signal.

Canonical schema, views, and engines. Customers consume canonical typed views as the inheritance surface. The Threadweave engine itself runs canonically; customers configure parameter overrides that the engine consumes when running for their tenant. A change to canonical schema, views, or engine logic propagates to every tenant; tenant-specific extensions stay local. Same applies to the canonical layer agents and the counter-hypothesis generators: their canonical operation is shared; tenant-specific extensions are tenant-private.

The opt-in abstracted contribution path Some customers will, over time, accumulate tenant-private `trace_snapshots` that represent valuable training signal: real reasoning under uncertainty paired with later outcomes, structurally novel because the customer's domain expertise produced them. The architecture commits to the seal holding by default and offering an opt-in path for those customers who choose to contribute abstracted versions to the canonical corpus.

The path is structural:

1. Customer-side authoring. The customer's tenant administrator initiates an abstracted contribution from a tenant-private `trace_snapshot` trajectory. The original snapshots stay tenant-private.
2. Abstraction. The contributed version is a redacted derivative: the entity is anonymised (replaced with a structural descriptor like "mid-cap European industrial supplier with concentrated supplier base"), the evidence packet is abstracted to its structural pattern (specific dates, magnitudes, and identifiable details replaced with categorical descriptions), the customer's agent identity is removed, and the customer's proprietary reasoning is paraphrased to remove distinguishing prompts or rubric language.
3. Customer review. The customer reviews and explicitly approves the abstracted version before it is committed.
4. Canonical commit. The abstracted version commits to the canonical corpus with `privacy_class = tenant_opt_in_abstracted`. It carries no reference back to the originating tenant. The customer's quota does not bill for canonical use of the contributed case.
5. Canonical use. The abstracted `trace_snapshot` trajectory is eligible for inclusion in the canonical corpus per §35. It is treated as a canonical row and contributes to the corpus the same way other canonical rows do.

The opt-in is per-trajectory, not per-tenant. A customer who opts in once does not opt in for everything. The customer can revoke a contributed abstracted trajectory; revocation removes it from future canonical reads but

does not retroactively undo any model-level use that has already happened (since canonical agents do not directly train on the corpus, this is a non-issue at v5; if it changes in v6+, revocation semantics are revisited).

The architecture does not require this path. Customers who never opt in pay no penalty. The canonical corpus grows without it; the path exists as a way for customers who want to participate in the corpus to do so without breaking the seal. See §35 for the corpus implications and §42 phase 4 for the rollout sequence.

Usage metering as a deliberate seal exception, narrowly scoped Token counts, entity registrations, trace counts, `trace_snapshot` volumes, and aggregate usage roll up to canonical- side billing. What flows is metadata about consumption, never the agent prompts that produced the consumption, never the `atomic_observations` or `trace_snapshots` that resulted, never the customer's reasoning. The seal in spirit holds; the metering surface is documented in §31.

The seal is one-way and irreversible by default. Canonical feeds client. Client never feeds canonical, except through the explicit, per-case, abstracted, customer-approved opt-in path of §29. If a feature of the canonical system would require client data to function, that feature is out of scope. Not a reason to break the seal.

Decision 11 in §25 flagged the privacy gradient on tenant-private trace links to canonical traces. The recommendation is the opt-in abstracted contribution path described here, ship in phase 4 of §42.

§30 Delivery and commercial shape

The architecture supports three commercial postures. Each maps to a different customer profile, a different latency expectation, and a different commercial structure.

Subscription tier (self-service workbench). Customer signs up, picks a tier, gets API access and the workbench. Three tiers (Starter, Professional, Enterprise) with progressive capabilities: entity coverage, trace-vector storage, custom-agent slots, Threadweave parameter overrides, integration features. Pricing per-seat-per-month plus per-entity-per-month plus token-based overage. The volume tier; most customers land here.

Enterprise SSO and managed deployment. Enterprise customers get SAML/OIDC SSO, dedicated tenant capacity, custom integration support. Same architectural surface; enhanced operational support. Pricing is committed-use with floor and ceiling.

Co-development engagement (rare). A small number of customers pay for direct collaboration on customer-specific extensions: bespoke validators, vertical adaptations, deep workbench customisation. Time-and-materials plus subscription floor. Reserved for strategic accounts; not the default motion.

The architectural surface is the same across all three. Sealing is the same. The runtime contract is the same. What differs is the operational support around the architecture, not the architecture itself.

Customer model Every customer is a tenant. Every tenant has a unique `tenant_id` that scopes all tenant-private rows. Every tenant has a billing-responsible admin (one mandatory), zero or more builders, zero or more viewers. The customer model in §18 specifies what each role can do.

Entity subscription Customers subscribe to entities from the canonical universe. A subscription gives the customer read access to that entity's canonical trace and write access to tenant-scoped extensions. Subscriptions count against tier allowance. Unsubscribing releases the read entitlement but preserves any tenant-scoped state the

customer wrote (so a customer who unsubscribes and later re-subscribes can recover their custom traces, custom deltas, and custom learning trajectory).

§31 Metering, billing, and customer accounts

Metering exists because StahlTrace cannot charge for what it cannot see. The metering surface is the smallest possible window into customer consumption: enough to bill correctly, not enough to leak the seal.

What is metered LLM token consumption. Per-tenant, per-agent-class, per-day. Summed for billing. The agent's prompt and output content is not part of the metering record; only token counts are.

Entity registrations. Per-tenant, per-day. The number of entities a tenant has subscriptions on. Billed against the tier's entity allowance with overage on a per-entity-per-month basis.

Trace storage. Per-tenant, the count of tenant-private traces across all subscribed entities. Tier allowances apply (Starter: 0 custom traces per entity, Professional: up to 10 per entity, Enterprise: unlimited within reason). Overage charged on a per-trace-per-month basis.

Trace_snapshot volume. Per-tenant, the count of tenant-private `trace_snapshots` per month. The throughput of the tenant's trace journey. Tier allowances apply with overage charged on a per-snapshot basis.

Trace_relationship storage. Per-tenant, the count of tenant-private `trace_relationships` retained. Tier allowances apply with overage charged on a per-relationship-per-month basis.

API request volume. Per-tenant, per-endpoint, per-day. Rate-limited per tier with overage allowed and billed.

Storage. Per-tenant total bytes (`atomic_observations`, `trace_snapshots`, `predictions`, `prediction_outcomes`, custom configurations). Charged on a per-GB-per-month basis, with the first N GB included in tier.

Active session count. For seat-based tier components.

How metering enforces quotas Every dispatch that will call an LLM consults the quota service before the activity begins. Tenant-scoped activities check tenant token and entity-count budgets; canonical activities check the canonical capacity envelope. Blocked dispatches surface a structured error to the customer rather than silently failing. Usage is metered at activity granularity so retries do not double-charge.

Billing transparency Tenant admins see real-time usage dashboards showing consumption against tier allowance and projected overage for the current billing period. Per-entity, per-agent, and per-trace breakdowns are available. The dashboards do not reveal canonical-side operational metrics (canonical capacity utilisation, peer-tenant comparison, internal cost structure).

Billing reconciliation Monthly. The metering store rolls up daily usage into monthly invoices. Disputes are handled out-of-band; the metering store is append-only so the underlying record is recoverable.

Customer accounts and lifecycle Onboarding. Customer admin signs up, configures tenant, invites builders and viewers, subscribes entities. The metering counter starts at activation.

Quota changes. Tier upgrades take effect immediately. Tier downgrades take effect at the next billing period to avoid mid-period quota cliffs.

Suspension. Non-payment or terms-of-service violation triggers suspension. Suspended tenants retain read access to their existing tenant-private state but cannot write or run agents. The 30-day grace period before any deletion is hard-coded.

Cancellation. Customer-initiated cancellation triggers a 30-day data retention window during which the customer can export tenant-private state. After the window, tenant-private state is deleted. Aggregated metering history retains for the regulatory minimum.

§32 API and access surface

The architecture's external surface is an API. Customers integrate by reading canonical state, writing tenant-private state, and consuming streamed events. The API is what makes the architecture usable rather than merely admirable.

API shape REST + JSON for synchronous operations. Server-sent events (SSE) for streaming subscriptions to real-time deltas, `trace_snapshots`, insights, hypotheses, and predictions. WebSocket for bidirectional flows where required (rare; reserved for high-throughput use cases).

Authentication API tokens for customer-side integration; OIDC for human users in the dashboard. Tokens are tenant-scoped; a token authenticates the calling identity and resolves the tenant context for every request. Per-token rate limits and per-token revocation, both immediate.

Endpoints, by category Identity and entity registration. Look up an entity by ticker, ISIN, name, LEI; subscribe and unsubscribe; list subscribed entities. Read-mostly.

Canonical trace graph. Read entities, sources, `evidence_events`, `atomic_observations`, traces, `trace_snapshots`, `trace_relationships`, predictions, `prediction_outcomes`, calibration, and `chapter_projections`. Filtered by entity, by date range, by layer, by `trace_type`, by activation state. The canonical read surface for tenants.

Custom agent management. Author, update, version, retire customer-defined agents. List tenant-scoped agents with their current calibration trajectories.

Custom trace management. Author, update, retire tenant-private traces. Read tenant-private trace graphs. Read merged (canonical + tenant) trace graphs for entities the tenant subscribes to.

Custom `atomic_observation`, `trace_snapshot`, and prediction submission. Write tenant-scoped `atomic_observations`, `trace_snapshots`, `trace_relationships`, and predictions on entities the tenant subscribes to. Server-side validation enforces typed schemas including the rule-6 counter-hypothesis sum-to-one CHECK constraint.

Outcome ingestion. Customers can stream their own outcomes (proprietary data feeds, internal benchmarks) for tenant-scoped scoring.

Threadweave parameter overrides. Configure horizon weights, competitor cohorts, sector tilts, trace-type weight overrides for the customer-specific `chapter_projections`.

Subscriptions and streams. Real-time SSE streams for deltas, `trace_snapshots`, insights, hypotheses, predictions on subscribed entities. The Trace Feed (§33) is the v3.1 streaming surface, extended in v4 to surface `trace_snapshots` as first-class events alongside deltas.

Audit and dashboards. Read tenant-scoped audit logs, usage metrics, calibration dashboards, billing.

Error model Structured errors with stable error codes. HTTP status code matches the semantic class (400 for client error, 401 for auth, 402 for quota exhausted, 403 for tenant-isolation violation attempts, 409 for conflict, 422 for typed schema validation failure, 500 for server error). Error responses include a stable code field for programmatic handling and a human-readable message.

SDKs Python and TypeScript SDKs at launch; others (Java, Go, Rust) deferred to demand. The SDKs wrap the REST/ SSE surface with type generation from the canonical schema, retry policies, and idiomatic streaming primitives.

Webhook integration Tenants can configure webhooks that fire on tenant-scoped events: a new prediction, a resolved outcome, a new high-priority insight, a trace moving past a falsifier-distance threshold. Webhooks are at-least-once with idempotency tokens; tenants are responsible for de-duplicating on their side.

What the API does not expose Canonical-side internal metrics (capacity, peer-tenant comparison, internal cost structure). Direct database access. Raw evidence payloads beyond what the entity trace surfaces (evidence is read through the typed views). Cross-tenant aggregations of any kind.

§33 The Trace Feed

THE DELTA FEED SURFACE · EIGHT RESONANCE PRINCIPLES

SINCE YOU LAST LOOKED · 18H AGO · 14 UPDATES ACROSS YOUR WATCH-LIST

The week's through-line on your watch-list is capital discipline under pressure. ASML reaffirmed its mid-cycle guidance against analyst skepticism.

↳ CONTRADICTS PRIOR THESIS

Eli Lilly secures EU approval for once-monthly orforglipron formulation, materially compressing Novo's structural lead in obesity therapeutics.

AI · CLAUDE

You've been reading the Novo thesis for 28 days. This update contradicts the supply-defended-moat assumption

RESONANCE PRINCIPLES VISIBLE IN THIS VIEW

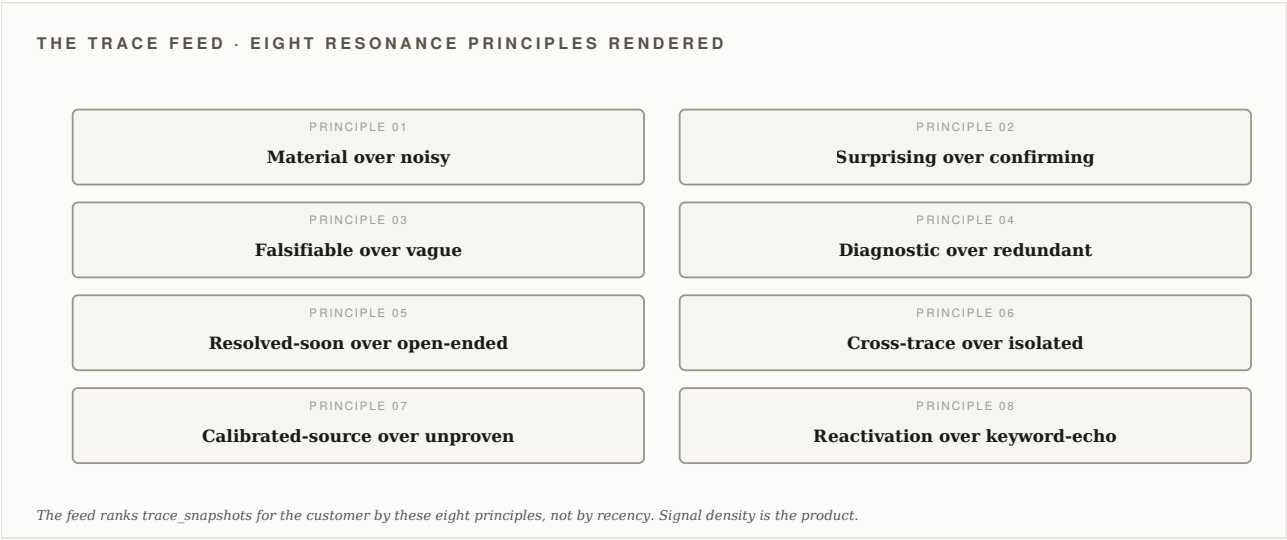


FIGURE 5 · THE TRACE FEED SURFACE, EIGHT RESONANCE PRINCIPLES RENDERED. THE FEED SHOWS ARCHITECTURAL ELEMENTS AS THEY APPEAR TO THE OPERATOR: THE THROUGH-LINE SYNTHESIS, THE PER-UPDATE CONTRADICTION BADGES, AI AS INLINE COLLABORATOR, AND THE QUIET-AFFORDANCE RESPECTING MUTES.

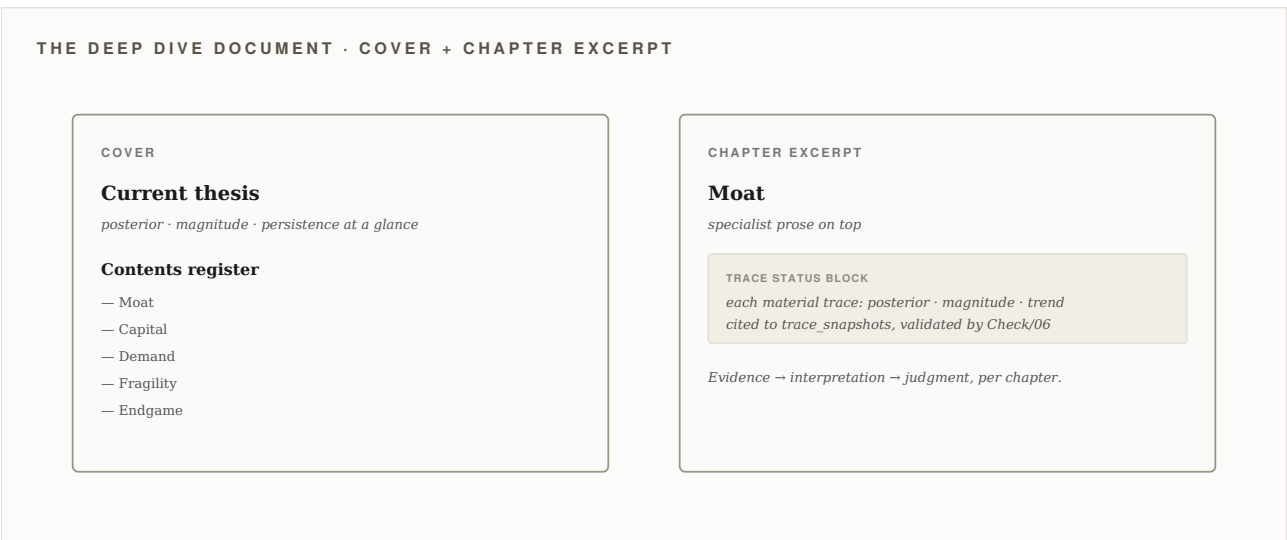


FIGURE 6 · THE DEEP DIVE DOCUMENT LAYOUT. COVER PAGE (LEFT) SHOWS CURRENT THESIS, THREADWEAVE RELIABILITY ACROSS HORIZONS, MECHANISM CONFIRMATION HISTORY, AND A CONTENTS REGISTER. THE CHAPTER EXCERPT (RIGHT) SHOWS HOW EACH CHAPTER FOREGROUNDS RECENT DELTAS AND CITATIONS ALONGSIDE NARRATIVE.

The Trace Feed is the streaming surface where customers consume the system’s reactions in real time. v3.1 introduced it as the streaming counterpart to the entity-trace API: a per-tenant SSE stream of deltas, insights, hypotheses, and predictions filtered by entity subscription. v4 extends the Feed to surface trace_snapshots as first- class events alongside deltas, with backwards-compatible event types so v3.1 consumers continue to function unchanged.

Event types on the Feed evidence_event .ingested . An evidence event has been attached to a subscribed entity. Carries evidence_event_id , entity_id , source_id, ingested_at, evidence_time , and a payload reference. Most customers do not subscribe to this stream directly; it is the rawest layer.

`atomic_observation`.committed . One or more atomic observations have been written from an evidence event. Carries `observation_id`, `evidence_event_id`, `observation_kind`, and the entities and dimensions detected.

`trace_snapshot`.committed . A `trace_snapshot` has been committed for a subscribed entity. Carries `snapshot_id`, `trace_id`, `entity_id`, prior posterior, `posterior_probability`, prior magnitude and persistence, posterior magnitude and persistence, the counter-hypothesis distribution, the dominant rival shift, the source reliability and diagnosticity used, and references to the contributing `evidence_event` and `atomic_observation`. Customer-facing presentation uses confidence-band language per §15, with the underlying numerics available for customers who explicitly request raw form.

`trace.activation_changed` . A trace has changed activation state (active → dormant, dormant → active via reactivation, active → resolved, active → falsified). Carries `trace_id`, prior activation, posterior activation, reason.

`trace_relationship`.published . A Dot Connector `trace_relationship` has been published linking traces across or within entities. Carries `relationship_id`, `relationship_type`, the `trace_ids` linked, evidence chain, forecastable flag, and a pointer to any child prediction.

`prediction_outcome`.resolved . An outcome has resolved one or more open predictions for a subscribed entity. Carries `prediction_outcome_id`, `entity_id`, the predictions resolved, the verdicts (outcome, mechanism), and the three per-track trace-snapshot verdicts (posterior calibration, magnitude, persistence) for the `trace_snapshot` the prediction was emitted from.

`chapter_projection`.published . A new `chapter_projection` has been published for a subscribed entity. Carries `projection_id`, `entity_id`, the version, the `trace_ids` that were promoted, and the per-horizon reliability scores.

Filter language Customers can filter the Feed by entity, by event type, by layer, by `trace_type`, by materiality, by falsifier-distance threshold, by horizon tier. The filter language is typed and validated server-side; invalid filters return 422 errors rather than silently delivering everything.

Delivery semantics At-least-once delivery with monotonic event sequence numbers per tenant. Customers acknowledge events explicitly; un-acknowledged events redeliver after a configurable window. The Feed is durable: a customer can resume from any sequence number within the retention window (default 30 days).

Backwards compatibility The v5 Feed introduces trace-centric event types as the canonical surface. Customers migrating from earlier integrations consume the new events through a Feed-version header; the architecture commits to maintaining version-pinned event delivery for at least 24 months from v5 launch so consumers can migrate at their own pace.

What the Feed does not do It does not stream raw evidence payloads (the evidence is referenced by ID; customers fetch the payload through the entity-trace API if needed). It does not stream cross-tenant patterns. It does not stream canonical-side operational events. It does not stream tenant-private `trace_snapshots` (the Feed is canonical to tenant; tenant- private state stays in the tenant namespace).

The Feed is the customer's window into the architecture's ongoing work. v5 reshapes the window around traces: `trace_snapshots`, falsifier movements, activation changes, `trace_relationships`, predictions, `prediction_outcomes`, `chapter_projections`. The nine enforcement rules of §06 hold for everything that flows through the Feed.

Part VI · R&D roadmap What becomes possible after the architecture runs. The canonical architecture (Parts I to V) commits to what gets shipped. This part describes directions the architecture has been designed to enable later,

but does not commit to building. Reading the canonical architecture as if it included these commitments would mislead. Reading this part as a roadmap of what is already being built would also mislead. The separation is deliberate, and the chapter is short by design.

R&D roadmap

What becomes possible after the architecture runs. The canonical architecture (Parts I to V) commits to what gets shipped; this part describes directions designed-for but not committed. Unchanged in v6.

§34 Framing, what this part is and is not

The architecture in Parts I to V is a working system. Operate it for long enough, against enough entities, with enough discipline, and several things emerge as natural consequences of the operation rather than as separate engineering ambitions. In v4, with the trace journey running, the corpus that emerges is materially richer than the v3.1 version: not just resolved predictions with provenance, but full trace-snapshot trajectories paired with reality. Six directions described here. None is a v4 launch commitment.

What is being committed in this chapter is something narrower: the canonical architecture is designed in a way that makes these directions possible later, and the substrate captures the data those directions would require. Whether and when to pursue them is a decision that depends on operating data the architecture does not yet have.

The chapter is conservative in its claims. The substrate that emerges from operating the architecture is genuinely valuable; whether it leads to a categorically different kind of forecasting AI, a transformative model of business reasoning, or simply better prompts and broader coverage, is a question for empirical testing on real accumulated data. The honest version of this chapter does not predict the answer. It describes the substrate, the directions it enables, and the conditions under which each direction would become tractable.

Three primary directions, one underlying argument The corpus is what gets produced. A multi-year structured record of business reasoning paired with reality. In v4, the unit of corpus is the closed `trace_snapshots`: prior state, evidence packet, agent action, system review, accepted update, later outcome, per-track calibration result. After 24 months of operation across the launch universe, the corpus is a research asset that does not exist anywhere else, because nobody else runs the architecture that produces it.

Prompt evolution is what gets done with it. The first concrete capability the corpus enables. Once the system can see, across a real cohort, which reasoning patterns produce mechanism-confirmed predictions and well-calibrated `trace_snapshots`, and which produce mechanism-contradicted ones and poorly-calibrated `trace_snapshots`, prompts can be refined systematically rather than by intuition.

Coverage economics is what makes the corpus large enough to matter. The corpus is only as valuable as the diversity of entities and regimes it covers. The architecture has been designed so that the marginal cost per tracked entity is dominated by automation rather than by analyst time. That cost structure opens the door to coverage of the uncovered universe.

Read together: the architecture produces a corpus, the corpus enables prompt evolution, economics enables enough coverage to make the corpus large enough to mean anything. None of these is research speculation. All three follow from operating the system as designed for long enough.

§35 The corpus, trace lifecycles paired with reality

The richest dataset that emerges from running this architecture is not the predictions, the calibrations, or the chapter projections. It is the paired record of reasoning and outcome: every trace the system ever held, every `trace_snapshot` that moved it, the counter-hypotheses considered at each update, the diagnosticity claimed for the evidence, the activation transitions over the trace's lifecycle, and the eventual reality that confirmed, contradicted, or refined the hypothesis. The corpus is the `trace_snapshot` history joined to the `prediction_outcomes` that resolved each trace.

This kind of data does not exist in public corpora. Equity research reports are written for persuasion, not falsifiability; they tend to be retrospectively rationalised rather than scored against original commitments. Academic finance datasets contain prices, fundamentals, and event timestamps, but rarely the structured reasoning that preceded the events. Internal hedge fund archives are private and even when accessed are rarely structured for falsifiability. Most importantly, no existing corpus carries the counter-hypothesis distribution and diagnosticity-claim structure that the trace graph produces, which means no existing corpus supports the empirical study of what good Bayesian reasoning under uncertainty actually looks like at scale.

What is in the corpus Resolved trace lifecycles with full provenance. For each trace that has reached `activation='resolved'` or `activation='falsified'`: the full snapshot history (every prior, every posterior, every counter-hypothesis distribution, every diagnosticity claim, every source reliability used), the activation transitions, the predictions emitted along the way, the `prediction_outcomes` that resolved them, the per-track calibration verdicts (Track 1: posterior calibration, Track 2: magnitude, Track 3: persistence), and the `chapter_projections` the trace contributed to. The unit of corpus is the trace lifecycle, not a single snapshot.

Resolved predictions with full provenance. For each closed prediction: the originating `trace_snapshot` and trace, the agent that proposed it, the prompt version used, the stated mechanism, the falsifier, the horizon, the timestamp, the `prediction_outcome` that resolved it, the verdict (confirmed, falsified, partial, unresolvable), and the mechanism verdict (confirmed, unconfirmed, contradicted).

Trace_relationship histories. The Dot Connector's cross-trace syntheses, with the `trace_snapshots` they drew from, the predictions they spawned, the `prediction_outcomes` those predictions eventually resolved to. A `trace_relationship` that proved load-bearing across multiple downstream predictions is structurally distinguishable in the corpus from one that resolved nothing.

Factor co-occurrence histories. The full factor co-occurrence ledger across years and entities, joined to `prediction_outcomes` and to `trace_snapshots` so that factor co-occurrence with trace moves becomes a separately- studyable phenomenon from factor co-occurrence with predictions.

Intent profiles and deviations. Per-entity intent assumptions, the evidence supporting each, the deviations recorded when companies acted unexpectedly, the eventual resolution of those deviations. A record of when stated intent and revealed intent diverged.

Future-state validations. Structural future-world claims (`trace_type='structural_future'`) authored at one time, with their evidence, their probabilities, and the eventual reality at the target horizon. Each structural-future trace carries its own lifecycle in the corpus.

Counter-hypothesis trajectories. For each trace, the historical sequence of counter-hypothesis distributions considered at update time, the dominant shifts that were claimed, the diagnosticity verdicts that came back at resolution. The corpus enables empirical study of which counter-hypotheses turn out to dominate at which kinds of evidence patterns, across which entity types, across which market regimes.

Reactivation histories. Traces that went dormant and were later reactivated, the diagnosticity threshold that the reactivating observation cleared, the eventual resolution. Empirical study of when sleep-then-wake patterns are well-calibrated and when they are noise.

What the corpus enables, conservatively framed The corpus is good for research that is currently impossible because the data does not exist. Several research questions become tractable that are not tractable today:

Reasoning patterns that produce well-calibrated `trace_snapshots` . Across entity types and market regimes, which reasoning structures (counterfactual checks, base-rate anchoring, explicit falsifiability, mechanism decomposition, counter-hypothesis enumeration) correlate with high posterior calibration on Track 1, accurate magnitude calibration on Track 2, and accurate persistence calibration on Track 3. Becomes an empirical question rather than a methodological preference.

Drift in what is actually predictive of outcomes over multi-year horizons. Signals that worked in one regime and stopped working in the next are observable in the corpus rather than only in retrospect. The shape of signal decay (how fast, how systematically, with what early indicators) becomes researchable.

Calibration on long-horizon structural forecasts. The kind of predictions §13 describes as “structural-future” claims becomes empirically grounded rather than speculative. A few years of corpus data is enough to begin scoring agents on whether their structural predictions about markets at 3 to 5 year horizons resolve accurately.

The empirical structure of counter-hypothesis distributions. What rival hypotheses dominate at what kinds of evidence patterns, in what entity types, in what regimes. Whether the system’s counter-hypothesis sets are too narrow (missing dominant explanations that emerge), too broad (carrying counter-hypotheses that never gain mass), or correctly scoped. Currently unstudied at scale because no data of this shape exists.

Diagnosticity drift detection. When a class of evidence (a particular filing pattern, a particular news source, a particular price residual shape) starts proving over-claimed or under-claimed in diagnosticity at scale, the drift itself is signal. Currently unstudied because no system tracks claimed diagnosticity at this granularity over time.

Privacy and contribution Canonical trace lifecycles. Generated by the canonical architecture against the canonical entity universe. Eligible for canonical research without further opt-in.

Tenant-private trace lifecycles. Generated inside customer tenants. Not eligible for canonical research; stay tenant-private.

Tenant-opt-in abstracted trace lifecycles. The contribution path of §29. Tenant-private trajectories that the customer has explicitly approved for canonical inclusion in abstracted form. Eligible for canonical research; abstracted to remove identifying detail.

The architecture commits to being able to grow the canonical corpus without depending on tenant-private contributions. The opt-in path exists for customers who want to participate, not as a structural prerequisite for the corpus to be useful.

Time horizon For the corpus to support the research questions above with statistical power, several thousand resolved trace lifecycles per `trace_type` are needed. At launch (100 entities, 8 to 25 active traces per entity, several material updates per trace per year), the canonical universe is on track to accumulate this volume in 2 to 3 years of operation. The R&D directions in §36 onward become tractable on roughly that timeline.

The corpus is not a research speculation. It is what the system produces by operating. The question is what to do with it once it exists.

§36 Prompt evolution, calibration applied to the prompts not just the outputs

The first concrete capability the corpus enables is systematic prompt evolution. Once the system can see, across a real cohort, which reasoning patterns produce mechanism-confirmed predictions and well-calibrated `trace_snapshots`, and which produce mechanism-contradicted ones and poorly-calibrated `trace_snapshots`, prompts can be refined empirically rather than by intuition.

What “prompt evolution” means here Today, agent system prompts are authored by engineers based on a combination of design intent, intuition, and small-scale experimentation. They evolve through manual revision: someone notices a pattern, hypothesises a fix, edits the prompt, observes whether things improve. This works at small scale but does not compound.

At corpus scale, prompt evolution becomes evidence-driven. The Evaluator has, for each prompt version of each agent, a set of resolved predictions with mechanism verdicts and resolved `trace_snapshots` with three-track calibration. A prompt revision that systematically improves mechanism-confirmation rates, improves Track 1 posterior calibration, improves Track 2 magnitude calibration, or reduces Track 3 persistence error is empirically better. A prompt revision that does not is empirically worse, regardless of how plausible it sounded.

What the architecture enables, what it does not Enables. Tracking calibration trajectories per (prompt version, agent class, `trace_type`, horizon tier, market regime). Surfacing patterns of failure (this prompt over-claims diagnosticity on regulatory-exposure traces). Comparing prompt variants empirically when scored data accumulates.

Does not enable, by design. Automatic prompt rewriting at scale based on calibration. The architecture deliberately stops short of meta-prompting (an agent that rewrites other agents’ prompts based on observed performance) for two reasons: (1) the meta-agent’s own calibration discipline would have to be established first, and the substrate to do that does not exist at v5; (2) automatic prompt rewriting introduces a feedback loop between prompts and the corpus that contaminates prompts as a calibration variable. Manual revision based on empirical signals from the corpus is the v5 commitment.

Trace-specific prompt classes The trace graph introduces three evaluation surfaces (Track 1 posterior, Track 2 magnitude, Track 3 persistence) which means three dimensions on which prompts can be evaluated. The counter-hypothesis-generation prompts and the diagnosticity-claim prompts inside layer agents are the prompt classes most directly affected: their performance is measured empirically the same way the rest of the system is.

§37 Coverage economics

The corpus is only as valuable as the diversity of entities and regimes it covers. Three things determine the marginal cost per tracked entity at scale:

Acquisition cost. Filings, news, transcripts, vendor data. Roughly fixed per entity per month at launch (vendor licences) and falls per-entity as universe grows (licences are typically not per-entity above some baseline).

Inline-loop compute. Per material observation, the inline loop runs the layer agents + CrossValidator + the per-trace Bayesian update arithmetic of step 07. Token cost scales with material-observation volume, not with entity count directly.

Sweep compute. The Dot Connector's nightly sweep runs per active entity. Token cost scales with entity count and with active-trace count per entity. Active universe at launch is small enough that this is bounded.

The total marginal cost per tracked entity at v5 launch is estimated at \$7 to \$9 per month at 100 entities. The cost falls roughly hyperbolically as universe grows because vendor licences amortise.

The economic argument: at \$7 to \$9 per entity per month, the architecture can defensibly track the uncovered universe (publicly listed companies that current research economics ignore because per-entity analyst cost is too high). At a few thousand entities, the corpus reaches volumes that make the §35 research questions tractable. The economics enable the corpus; the corpus enables the research; the research compounds the system's value.

§38 Public Actor Trace, the Worldview & Game Layer

The architecture's primary commitment is finance. It is also domain-general. Public Actor Trace is the deferred extension to other domains where persistent agents make decisions under uncertainty, and where the same lived-record, trace-snapshot, mechanism-attribution discipline produces a comparably valuable corpus.

What it is A separate vertical (or set of verticals) using the same architecture: same six-layer stack, same trace journey, same substrate, same scoring discipline, applied to entities that are not companies. Examples include: government policy actors (decisions under uncertainty, accountable to electorates and constituents, with measurable outcomes), regulatory bodies (decisions with deferred consequences and observable enforcement patterns), large institutional investors (positions with disclosed timestamps and resolved performance), foundations and large- grant programmes (multi-year commitments with measurable outcomes).

Why it is deferred Three reasons. The architecture's discipline has not been validated against any vertical yet; pursuing two simultaneously dilutes the discipline. The data acquisition layer is finance-specific at launch (EDGAR, GLEIF, S&P Capital IQ); other verticals require parallel acquisition pipelines. The customer motion is finance-specific at launch; pursuing other verticals requires a parallel sales motion the company is not yet equipped for.

What the architecture commits to The substrate is domain-general; entities are typed, but the typing is not finance-specific. The trace schema is category-driven, and categories are open-ended; new domains add new categories. The agent set is structurally composable; new domains author new specialists alongside the finance-launch set. The Threadweave intent framework explicitly accommodates non-NPV objectives (political_or_regulatory_objective , other).

The architecture's domain-generalty is the option value. v4 commits to finance; the substrate is what makes Public Actor Trace possible later, not what closes it off.

Guardrails on public-actor coverage When Public Actor Trace ships, three guardrails are committed: (1) coverage is restricted to actors with sufficient public reasoning trace to support the architecture's evidence requirements (private deliberations are out of scope by design); (2) trace types applied to public actors are conservatively narrow at launch (decision-making consistency, public reasoning quality, deviation from stated commitments) rather than aspirationally broad; (3) the calibration metrics published per actor make explicit the long-horizon nature of most public-actor decisions and the consequent calibration uncertainty.

§39 Preconditions

For the R&D directions in this part to become tractable, several preconditions must hold. These are not commitments; they are conditions under which the directions stop being aspirational and become work.

The architecture must run for long enough to accumulate corpus volume. §35 estimates 2 to 3 years for the launch universe to reach research-tractable corpus size. Premature pursuit of R&D directions before this volume exists produces preliminary work, not foundational work.

Calibration must remain credible. If the canonical calibration discipline drifts (Evaluator over-confirming mechanisms, prompt versions silently re-attributed, retrospective passes contaminating live records), the corpus loses its scientific value before research can use it. The v5 commitment to autonomous-only scoring with deterministic gates (§42) is in part a precondition for the corpus to be useful.

Tenant opt-in for the abstracted contribution path must materialise. (v4 specific) The canonical universe alone may not produce enough diversity of entity types, market regimes, and reasoning patterns for some R&D questions. The opt-in abstracted contribution path of §29 is the architecture's way of growing diversity without breaking the seal. If no tenants opt in, the corpus is bounded by canonical-only growth, which still supports many research questions but not all.

Compute economics must hold. The §37 cost analysis assumes vendor licences amortise, sweep compute scales sub-linearly, and inline compute per delta does not balloon as the trace layer matures. If any of these assumptions fails, coverage economics constrain the corpus and the directions in this chapter become more expensive to pursue.

Operational discipline must compound. The §17 runtime contracts, the §22 access matrix, the §29 seal, and the §42 phase plan are operational commitments that have to hold continuously. Slippage on any of them does not just cost a feature; it costs corpus value, because the corpus is only as good as the discipline that produced it.

What the architecture does not commit to It does not commit to any of the R&D directions becoming products. It does not commit to causal modelling, agent simulation, future-state forecasting, prompt evolution, or Public Actor Trace as named features in any specific release. It commits to building an architecture where these directions are structurally possible later, and to operating that architecture with enough discipline that the data they would require is genuinely there when the time comes.

The R&D roadmap is conditional. The architecture's commitments are not. The separation is what keeps the commitments meaningful.

Part VII · The trace algebra The four sections that follow specify how traces work in detail. §40 is the operational definition of magnitude and persistence — the two dimensions every trace carries beyond probability — and the parent-updater function that composes them up the trace graph. §41 is the eight-state activation lifecycle that lets traces sleep without being deleted. §42 is the phase plan that takes the architecture from v4 substrate to full v5 operation. §43 is the decision memo: what is being committed, what is at risk, what remains open.

The trace algebra

Magnitude, persistence, the parent updater, and the trace lifecycle — plus, in v6, a note on honest non-scoring in the deep-dive layer (§40a). The phase plan and decision memo close the document.

§40 Magnitude, persistence, and the parent updater

Every trace carries three numbers that move over time: posterior probability, magnitude, and persistence. Probability is what the system believes about the hypothesis. Magnitude is how big the effect is if the hypothesis holds. Persistence is how long the effect lasts. The three are independent: a trace can be high-probability with low magnitude (a confirmed but minor effect), low-probability with high magnitude (a possible regime shift), high-magnitude with low persistence (a one-quarter dislocation). Reducing the trace to a single confidence number throws away two dimensions of information and makes calibration uninterpretable.

A trace also rarely lives alone. It composes into a graph: child traces track mechanisms (data migration friction, workflow lock-in, audit risk); parent traces track investment-relevant conclusions (switching-cost moat strength, moat durability); chapter projections render the canonical parents. The composition is governed by the parent updater, a defined function that consumes child posteriors through typed edges with weights and substitute / complement / bottleneck semantics. The parent updater is where the graph stops being a list of independent traces and becomes a Bayesian engine that computes investment-relevant conclusions from mechanism-level evidence.

Why three numbers, not one A v4-era moat update would express “the moat weakened” as a posterior shift on a binary belief. The customer reading the output sees confidence dropped from 0.78 to 0.46 and infers something like “the moat is half as strong as we thought.” But this conflates two questions the operator actually needs separately: did the system change its mind about whether the moat exists at all (probability), and did the system change its mind about how much the moat is worth (magnitude)? A regulatory enforcement action might leave the probability of the moat existing roughly intact while sharply downgrading the magnitude (the moat still exists, but the supplier-specific component of it no longer compounds). A different evidence pattern might do the opposite. Without separating the two, the calibration record cannot distinguish “the system was wrong about whether” from “the system was right about whether but wrong about how much,” and the customer cannot tell either.

Persistence is the third dimension because effects in the real world have natural durations and the system’s claim about duration is testable. A trace whose magnitude is correct but whose persistence is overstated produces a prediction that resolves correctly in direction and magnitude but fades faster than the system claimed. That is a different failure than “wrong direction” or “wrong magnitude” and deserves its own calibration track.

Child trace	Weight	Role
Data migration friction	25%	technical switching barrier

Workflow lock-in	20%	process switching barrier
Integration depth	20%	ecosystem switching barrier
Audit and compliance risk	15%	risk switching barrier
User training burden	10%	behavioral switching barrier
Contractual lock-in	10%	legal switching barrier

Magnitude and persistence interpretation by `trace_type` The arithmetic is identical across trace types; the units and the operator semantics differ. The `trace_type` field disambiguates what the numbers mean.

Moat traces carry magnitude as strength (a dimensionless 0..1 scale) and persistence as duration (years). A widening moat increases strength and may extend duration. A regulatory enforcement that compresses a supplier-specific advantage decreases strength and shortens duration. Strength is what shows up in the chapter's moat narrative as "high / medium-high / medium / medium-low / low" per the §15 confidence-band convention. Duration is what shows up as years-to-erosion.

Ad-hoc traces carry magnitude as expected business impact (typically expressed in basis points of margin, percentage points of growth, or dollars where a unit applies) and persistence as expected relevance horizon (days). An ad-hoc trace about a one-time cost item has high magnitude and low persistence (90-day relevance). An ad-hoc trace about an emerging adjacency might have moderate magnitude and high persistence (730-day horizon). The `expected_persistence` on the trace's emitted predictions inherits this number.

Structural-future traces carry magnitude as weight on this future (probability mass on this branch of the reachable-state tree) and persistence as time-to-realisation (years until the structural claim is testable).

Required-condition traces (spawned by Threadweave's reverse traversal) carry magnitude as load-bearingness (how critical this condition is to the target future) and persistence as how long the condition window remains open (years). A required condition with high load-bearingness and short persistence is a near-term go/no-go for the target future.

Capital-allocation, demand-durability, supply-resilience, regulatory-exposure, technology-position, management-integrity traces follow the moat pattern: magnitude as a 0..1 strength of the current operating reality, persistence as duration in years.

The interpretation is documented in the trace's metadata when it is authored, and the chapter projection (`§13 / §21 chapter_projections`) renders the right units in the right chapter.

How the arithmetic moves the numbers Each `trace_snapshot` writes prior values, posterior values, and the `likelihood_ratio` vector that connects them. For probability, the move is standard Bayes: $\text{posterior odds} = \text{prior odds} \times \text{LR}$ over the counter-hypothesis set, normalised. For magnitude, the move is evidence-weighted: the agent's claim about magnitude in the new evidence packet is combined with the prior magnitude using a precision-weighted update, where the precision of the new evidence is $\text{source_reliability} \times \text{diagnosticity_for_magnitude}$. For persistence, the same precision-weighted update applies, with the agent's claim about persistence from the evidence and the prior persistence on the trace.

The three updates run together within one `trace_snapshot` commit. The substrate preserves the LR vector, the magnitude evidence weight, and the persistence evidence weight on every snapshot, so the update is fully replayable from the recorded inputs (rule 2 of §06: no hand-scored Bayesian journey).

Magnitude and persistence calibration tracks Track 2 (magnitude) and Track 3 (persistence) of §15 score these dimensions independently of Track 1 (posterior). When a `prediction_outcome` resolves a `trace_snapshot`, the Evaluator compares predicted magnitude to realised magnitude, and predicted persistence to the resolution-window evidence. A `trace_snapshot` whose probability was well-calibrated but whose magnitude was overstated by a factor of two earns a positive Track 1 verdict and a negative Track 2 verdict. Calibration aggregates carry the three tracks separately by source, by agent, by `trace_type`, and by horizon tier. A source that is reliable for direction but unreliable for magnitude is a different operational category from a source that is reliable for both, and the calibration record makes that distinguishable.

Why this replaces the v4 four-layer posterior v4's four-layer posterior decomposed every belief into business, valuation, knowability, and tail-risk components, with separate posteriors maintained on each. The model captured something real (a price move and a guidance revision shift different posteriors) but at substantial cost: every belief carried four numbers, every update touched up to four of them, the substrate maintained four sets of FK columns, and the customer surface had to reconcile four moves into one rendering. Most beliefs in practice moved on at most two of the four layers at any one update.

Magnitude and persistence capture the architecturally important content the four-layer posterior was after, with three numbers instead of four and with a substrate that does not need to know in advance which of the four layers a given evidence packet should route to. A guidance revision moves probability and magnitude on the relevant trace; the system does not need to ask “is this a business-layer move or a valuation-layer move” because the trace already has a `chapter_anchor` that tells it where the move shows up. The simplification is real, not a renaming.

The trace graph as a Bayesian engine A company is represented as a nested graph of traces, not a flat list. The example tree:

The leaf traces track mechanisms; the parent traces track investment-relevant conclusions; the company thesis at the root is what shows up as the entity's headline narrative. Evidence updates the most specific relevant trace first; the parent updater consumes child posteriors and recomputes the parent. Information flows up.

The graph structure lives in two substrate tables. `traces` holds the trace objects themselves, flat and addressable. `trace_hierarchy` holds the parent-child edges, with the measurement model attached to each edge: weight, role, semantic. `trace_relationships` (a separate table, §21) holds the typed cross-graph relationships that are not parent-child composition (`supports`, `attacks`, `causes`, `explains`, `contradicts`, `correlates_with`, `same_source_as`, `evidence_for`). The two tables are kept separate because they answer different questions: `trace_hierarchy` is read during parent-updater evaluation; `trace_relationships` is read by the Dot Connector during cross-trace synthesis.

A trace can have multiple parents. The “Data migration friction” trace might be `component_of` the switching- cost moat parent and also `evidence_for` an ad-hoc trace about implementation-services revenue compression. Multi-parent edges are the norm, not the exception, and the substrate represents them naturally because edges live in their own table.

Belief flows up through `trace_hierarchy` edges. Attention flows down through `monitoring_priority`. The two paths are separate substrate writes; rule 9 of §06 prevents parent concerns from leaking into child posteriors.

CANONICAL · ROOT

Company thesis trace

CANONICAL

Moat trace

CANONICAL · PARENT UPDATER ATTACHED

Switching-cost moat trace strength · duration · 6 component children

attacks CANDIDATE · AD HOC · ATTACKING

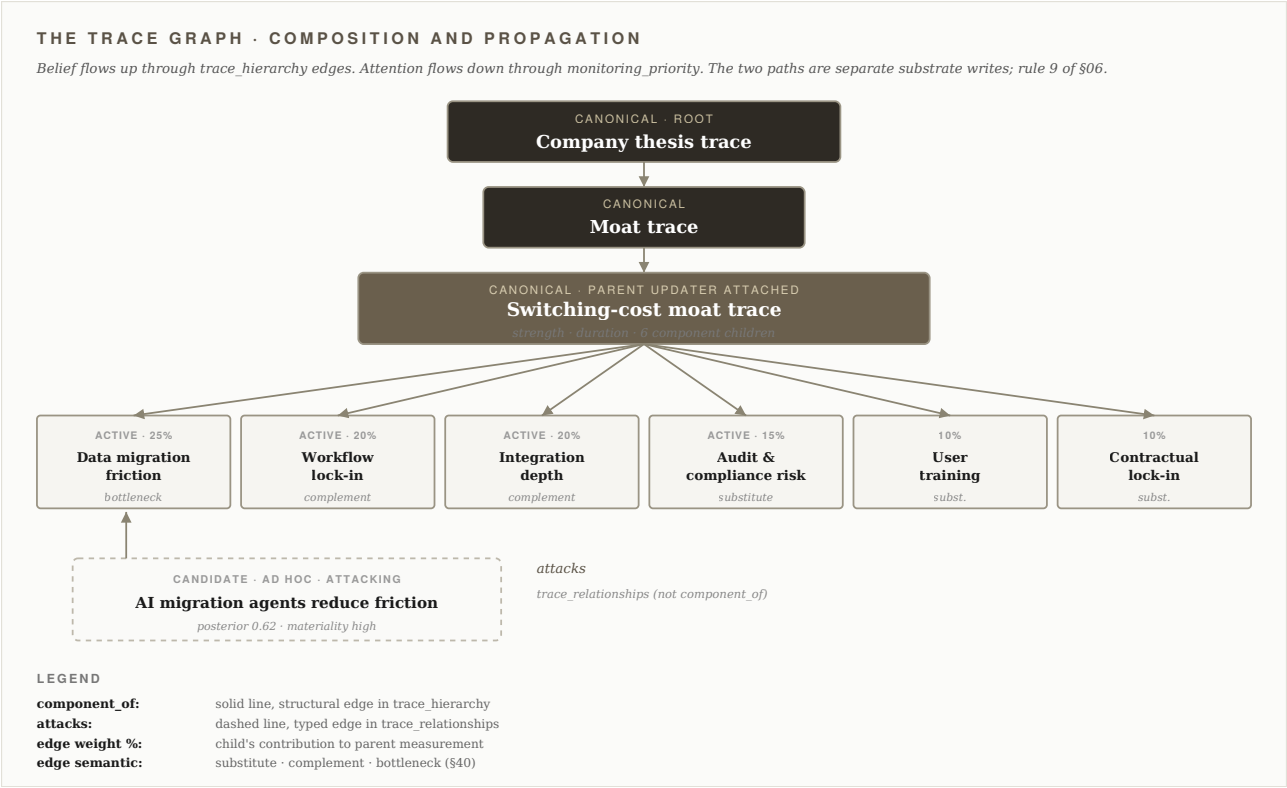


FIGURE 12 · THE TRACE GRAPH FOR ONE ENTITY, THREE LEVELS OF COMPOSITION. CANONICAL PARENT TRACES (COMPANY THESIS → MOAT → SWITCHING-COST MOAT) AT TOP COMPOSE FROM ACTIVE COMPONENT CHILDREN WITH WEIGHTS AND EDGE SEMANTICS. BELIEF FLOWS UP THROUGH THE SOLID 'COMPONENT_OF' EDGES. A CANDIDATE AD-HOC TRACE ("AI MIGRATION AGENTS REDUCE FRICTION") ATTACKS ONE OF THE CHILDREN VIA A TYPED 'TRACE_RELATIONSHIPS' EDGE, NOT A MEASUREMENT EDGE: THE ATTACK DOES NOT PROPAGATE AS A PARENT-UPDATE BELIEF MESSAGE, BUT IT DOES PROPAGATE AS A POSTERIOR SHIFT ON THE ATTACKED CHILD AFTER INDEPENDENT EVIDENCE ARRIVES.

How the parent updater works The parent updater is a typed function attached to each parent-child edge. For a parent trace P with children {C₁, C₂, ..., C_n}, when any child's posterior moves, the Orchestrator enqueues P for parent-update evaluation. The parent updater reads the latest snapshot of each child, applies the edge metadata (weight, role, semantic), and computes:

- Parent posterior probability. A precision-weighted aggregate of child posteriors, with edge weights and child confidence determining the precision of each input. The aggregation respects the semantic of each edge.
- Parent magnitude. Computed via the measurement model defined for this parent. For switching-cost moat strength, the model is roughly: strength = w₁ · data_migration_strength + w₂ · workflow_strength +

$w_3 \cdot \text{integration_strength} + \dots$ with the weights reflecting each child's contribution to the parent's measurement. • Parent persistence. Not a simple weighted average. The parent updater considers whether children are substitutes, complements, or critical bottlenecks. If one child is a critical bottleneck (the switching-cost moat collapses entirely if AI migration agents work), then the parent's persistence is dominated by that child's persistence regardless of the others. If children are substitutes (the moat survives if any one of several barriers holds), then the parent's persistence is the maximum of the children's persistences. If they are complements (the moat needs all barriers together), then the persistence is closer to the minimum. • Parent confidence. A meta-measure: how much evidence and how consistent is the chain of reasoning. A parent computed from three confident children with high diagnosticity is more confident than a parent computed from one confident child and two children whose evidence is sparse.

The substitute / complement / bottleneck semantic is stored on the `trace_hierarchy` edge as a typed enum. The semantic is set at edge creation by the trace author (typically a Deep Insight specialist or the Dot Connector when synthesising hierarchy), reviewed by CrossValidator's structural check, and revised only with full audit trail.

Switching-cost moat, worked example For the switching-cost moat parent, the brief specifies a measurement model:

The parent updater consumes each child's latest posterior, magnitude, and persistence; applies the weights; and computes parent strength and parent duration. The model is configurable per parent trace, stored in `trace_hierarchy` rows whose `parent_trace_id` points to the switching-cost moat trace. When the AI- migration-agents child's magnitude drops sharply, the switching-cost moat's strength drops proportionally — but its duration drops more sharply if AI migration is treated as a critical bottleneck (the semantic on that edge is set to `bottleneck`), which it should be: if migration becomes near-frictionless, no other switching barrier compensates.

Bottom-up belief, top-down attention Belief flows up. Attention flows down. The two are kept on separate substrate writes. The brief makes this rule explicit and the architecture honors it as rule 9 of §06: parent traces never overwrite child posteriors directly.

Bottom-up belief propagation. Evidence $\rightarrow$ child `trace_snapshot` $\rightarrow$ parent updater $\rightarrow$ parent `trace_snapshot` $\rightarrow$ `chapter_projection`. This is what changes beliefs. The parent updater reads child posteriors but writes only the parent's snapshot. Children are unaware of the parent update.

Top-down attention propagation. When a parent's posterior shifts (especially toward higher risk), the system increases monitoring of related child traces. The mechanism is a separate field on each trace: `monitoring_priority`. When a parent moves, the Orchestrator writes elevated `monitoring_priority` to the children, which lowers the materiality threshold at which their incoming observations trigger a snapshot, raises their position in the Dot Connector's sweep priority, and increases the diagnosticity budget assigned to their evidence-event scoring. The children's posteriors do not change. Their watch priority does.

The separation is what prevents confirmation-bias cascades. Without rule 9 and the attention-vs-belief separation, a parent that becomes more concerned about the moat would silently lift child posteriors, which would feed back into the parent updater, which would lift the parent further. Belief becomes self-confirming, calibration becomes circular, and the audit trail becomes meaningless. The architecture's commitment is that a parent's elevated concern increases what the system looks for in the children, not what it believes about them. Belief moves only when fresh evidence arrives.

Belief messages, computed not stored The parent updater needs to read child posteriors. The brief frames this as “child traces emit belief messages upward.” Architecturally, belief messages are computed at parent-update time

by reading each child's latest committed snapshot, not stored as a separate substrate object. The information content of a belief message is fully recoverable from the child's snapshot plus the edge metadata; storing it would duplicate state.

What is stored is the parent's snapshot row, which carries `input_provenance` jsonb capturing exactly which child snapshots the parent updater consumed and the values it read. This makes the parent update fully replayable from the recorded inputs, the same audit discipline rule 2 of §06 enforces for evidence-driven updates. A reader of the parent's snapshot history can reconstruct the entire reasoning chain without any separate belief-message ledger.

Magnitude and persistence calibration tracks Track 2 (magnitude) and Track 3 (persistence) of §15 score these dimensions independently of Track 1 (posterior). When a `prediction_outcome` resolves a `trace_snapshot`, the Evaluator compares predicted magnitude to realised magnitude, and predicted persistence to the resolution-window evidence. A `trace_snapshot` whose probability was well-calibrated but whose magnitude was overstated by a factor of two earns a positive Track 1 verdict and a negative Track 2 verdict. Calibration aggregates carry the three tracks separately by source, by agent, by `trace_type`, and by horizon tier. A source that is reliable for direction but unreliable for magnitude is a different operational category from a source that is reliable for both, and the calibration record makes that distinguishable.

Parent-trace calibration is a separate aggregate track. A parent updater whose composition consistently produces well-calibrated parent magnitudes despite noisy child inputs is better than one that overstates. The parent-updater calibration compounds at the level of the function itself, so a measurement model that aggregates poorly is identifiable and revisable.

§40a Honest non-scoring in the deep-dive layer NEW

This is a note about the deep-dive agentic layer, not a change to core trace scoring. Core scoring — the posterior, the counter-hypothesis distribution, magnitude, persistence, and the §06 enforcement rules — is v5.1's end-state design and v6 does not touch it. What v6 records here is a discipline the v4.1 deep-dive trial demonstrated and that the canonical-chapter layer (§06a–§06e) should inherit: the ability to say **"we cannot honestly answer this, because the sources do not cover it."**

v4.1's deep-dive schema carried a `confidence` enum whose values included `INSUFFICIENT`, distinct from high / medium / low. It was used when the source bundle did not cover a checklist question. In the live trial, the ST-12 deep-dive returned six `INSUFFICIENT` items — the reverse-DCF-dependent questions, when free cash flow was absent from the bundle — and zero elsewhere. That is honest non-scoring at the brief level: the specialist declined to guess on the questions it could not answer and answered the rest, rather than fabricating a confidence to fill the gap.

v6 commits to carrying `INSUFFICIENT` as a first-class value in the **deep-dive brief and structured-outline layer** (§06d): a checklist item or an outline entry may be marked insufficient, with a required `evidence_gap` naming what would be needed to answer it. This keeps the canonical chapter honest — a chapter that cannot support a claim says so in prose and in its outline, rather than seeding a trace on thin air.

Whether this should propagate *into core trace scoring* — for instance, a `scoring_status` field on `trace_snapshots` that lets a trace itself hold an explicit "insufficient evidence" state distinct from a wide confidence band — is a genuine question, but it is a change to core machinery, and core machinery is the

architecture's end state, not v4.1's to rewrite. v6 therefore **flags it as open** (D/11, §25) rather than committing it. The deep-dive layer gets the discipline now because that is the layer v4.1 actually exercised it in; the core gets it only if and when the end-state design decides it belongs there.

§41 The trace lifecycle, eight states

Traces have lifecycles. A trace's activation field carries one of eight states at any moment: candidate , active , promoted , canonical , dormant , reactivated , resolved , falsified . Formal company-chapter traces typically begin as canonical . Free-form traces typically begin as candidate or active . All eight states are real states in the substrate, not transitions; the eight-state space is what gives the lifecycle its expressiveness without proliferating.

The architecture's commitment to System B over System A (§03) requires that lifecycle is non-destructive. A moat trace that goes dormant because the entity has changed industry segment may become relevant again three years later if the entity reverses course. Deleting it would destroy the audit trail; renaming it would mask the regime change. The eight states give every move a name and an audit position.

The eight activation states candidate. A newly authored trace that has not yet committed its first scored snapshot. Candidates live in the trace graph for review (the counter-hypothesis gate of step 05 plus the layer-agent ensemble agreement check). Candidates that fail review are rejected before any snapshot commits. Candidates that pass review transition to active on first snapshot commit.

active. The default operating state. The trace is receiving updates, emitting predictions when its snapshots cross the prediction threshold, and contributing `trace_relationships` to the Dot Connector's sweep. Active traces appear in the Trace Feed but only enter chapter projections after promotion.

promoted. An active trace whose materiality and posterior strength have cleared the chapter promotion thresholds (rules 5 and 8 of §06). Promotion is gated on more than just current state: posterior probability, materiality, source reliability of contributing evidence, evidence independence, link to a core thesis assumption, time sensitivity, and prediction quality together determine whether an active trace can promote. Promoted traces appear in the entity's chapter document as named items but are still understood by the system as freshly elevated, with their full snapshot history visible alongside their canonical-elevated status.

canonical. A long-standing promoted trace that has accumulated enough scored history to be treated as a load-bearing chapter element. Formal company-chapter traces (the entity's thesis trace, moat trace, capital-allocation trace, and so on) are canonical from inception, authored as part of the entity's bootstrap. Promoted traces become canonical after sustained materiality and at least one full prediction-resolution cycle that confirms the trace's mechanism. The transition is one-way under normal operation; canonical traces can only fall back via explicit `superseded_by` retirement, never silently.

dormant. The trace has not received an update in N days (N defaults are per `trace_type` : 90 days for `ad_hoc` , 270 days for `moat-family` traces, 540 days for `structural_future` , 365 days for `capital_allocation`). The trace's hypothesis remains relevant but has not been challenged by recent evidence. Dormant traces do not enter chapter projections (rule 8 of §06) but remain matchable in step 04 for possible reactivation. Both active and promoted traces can transition to dormant ; canonical traces can also dormant under sustained absence of evidence, though this is rare and itself a signal worth logging.

reactivated. A dormant trace that has been woken by an observation passing the reactivation gate (rule 4 of §06). The state is distinct from active because the system treats reactivated traces differently: the first post- reactivation snapshot is logged with explicit reactivation provenance (which observation, which gate-check, what

diagnosticity), the prediction emission threshold is held one tier higher than for native-active traces (a guard against shallow reactivations), and the reactivated trace's posterior history is rendered with an explicit dormancy gap in customer-facing surfaces. After three subsequent confirmed snapshots, a reactivated trace transitions back to active .

resolved. The trace's predictions have resolved confirmed at high confidence, and the trace's hypothesis is now historical fact rather than open hypothesis. A resolved trace does not emit new predictions, but its history remains as evidence for future trace authoring (a confirmed moat that erodes ten years later starts a new trace whose `trace_hierarchy` edges or `trace_relationships` cite the resolved one as evidentiary precedent).

falsified. The trace's primary falsifier has triggered, or the dominant counter-hypothesis has accumulated overwhelming probability mass. Falsified traces are retained for the same reason as resolved traces — the audit trail — but explicitly marked so that the chapter projection logic excludes them and the user-facing surface marks them as historical-falsified rather than open.

State transitions The transitions between states are typed and gated:

- candidate → active : first scored snapshot commits cleanly (counter-hypothesis gate passes, ensemble agreement clears).
- active → promoted : materiality, posterior, and the seven promotion gates clear.
- promoted → canonical : sustained materiality plus at least one confirmed prediction cycle.
- active / promoted / canonical → dormant : per- `trace_type` touch-event window expires.
- dormant → reactivated : reactivation gate passes (the three typed checks of rule 4).
- reactivated → active : three confirmed snapshots after reactivation.
- active / promoted / canonical → resolved : prediction confirmed at high confidence, no contradicting evidence.
- active / promoted / canonical → falsified : primary falsifier triggers, or counter-hypothesis dominates.

Transitions write to a `trace_lifecycle_event` provenance log (an append-only table joined by `trace_id`) so the full history of state changes for any trace is auditable. The Orchestrator is the sole writer of activation transitions; layer agents propose transitions but never commit them.

Reactivation, gated against keyword matching Reactivation is the move from dormant to reactivated . The simplest reactivation rule (any new evidence whose dimensions overlap a dormant trace's dimensions wakes the trace) fails because most evidence has dimensions that overlap most traces, and the chapter projections would fill with reactivated noise. Rule 4 of §06 is the architectural answer: no keyword-only reactivation. The reactivation gate, run by the Orchestrator's step-14 activation review, requires that the new evidence materially address the dormant trace's hypothesis or its counter- hypothesis set.

The gate is implemented as a typed check, not an LLM judgment:

1. The new `atomic_observation` 's `claim_payload` is compared against the dormant trace's `expected_evidence` (registered when the trace was authored) using structured field overlap and semantic similarity bounded above a threshold. Pure semantic similarity is insufficient, the structured fields must overlap.
2. The observation's `diagnosticity_claim` for the dormant trace must clear a floor (default 0.4). Evidence that is barely diagnostic for the trace cannot wake it.
3. The observation's source must have a `current_reliability` above floor for the trace's `trace_type` . A low- reliability source on a high-stakes `trace_type` cannot wake a dormant trace by itself.

Observations that pass all three gates wake the trace into reactivated ; observations that fail enter the trace's evidence_ids list as touch-events but do not flip activation. A trace that accumulates many touch-events without any passing the gates is also a signal: it suggests either the gates are too strict for this `trace_type` or the evidence is genuinely peripheral. Both diagnostics surface as operator-facing telemetry.

Why dormant rather than retired The naive design retires traces that go quiet. Retirement is destructive: a retired trace that becomes relevant again must be re-authored, losing the prior reasoning trail. Dormancy is non-destructive: the trace remains, the prior reasoning remains, and reactivation reconnects to the original line of reasoning rather than starting fresh. The cost is substrate growth (dormant traces accumulate without bound), but `trace_snapshot` history is partitioned by month and dormant traces are cheap to keep.

The retirement that does happen is a different concept: when a trace's hypothesis has been definitively superseded by a better-formulated trace (the original was poorly worded, the `trace_type` was wrong, the `chapter_anchor` was misjudged), the original trace is marked `superseded_by` pointing to the new trace, with `retired_reason='replaced_by_better'`. The original's history remains for audit but the new trace is the operational locus. This is rare and is treated as architectural debt: a high rate of `replaced_by_better` traces signals a problem with trace authoring quality.

§42 Phase plan, migration, and the autonomy commitment

The architecture's transition from v4 to v5 is a substrate change. The phases below describe both the migration sequence and the operating gates that govern it. No phase advances on schedule; each advances when its measured gates pass. The autonomy commitment runs through every phase: at no point does a human-in-the-loop become the gating mechanism for a phase advance. Gates are measurable substrate metrics, audited continuously.

Phase 1 · Substrate migration Build the v5 ten-table substrate alongside the existing v4 substrate. Backfill v4 deltas and belief_updates as v5 `trace_snapshots`, with one `trace_snapshot` per (delta + belief_update) pair on the same transaction. Backfill v4 beliefs as v5 traces. The mapping is mechanical for the columns that overlap; magnitude and persistence are computed from v4 metadata where possible and marked as backfill-derived where not. Run both substrates in parallel for the duration of the phase; v4 remains authoritative.

Pass conditions: 100% of v4 deltas have a corresponding v5 `trace_snapshot` row; 100% of v4 beliefs have a corresponding v5 trace row; all v4 access-matrix invariants hold on the v5 substrate; replay of any v4 baseline reconstructs the same baseline from the v5 `trace_snapshot` history within tolerance.

Phase 2 · Live dual-write The Orchestrator writes to both v4 and v5 substrates on every commit. The v5 substrate adds the magnitude and persistence columns that v4 did not maintain natively; the layer agents are updated to emit magnitude and persistence on every `trace_snapshot` proposal, with backfill-derived values on traces inherited from v4 marked accordingly. Forecaster and Evaluator continue to read from v4. CrossValidator runs Check/05 against both substrates and flags any divergence.

Pass conditions: zero CrossValidator divergence over 30 consecutive days; magnitude calibration error < 0.30 on Track 2 over the launch universe; persistence calibration error < 0.30 on Track 3.

Phase 3 · Read-cutover The customer-facing read paths (Trace Feed, chapter projections, deep-dive document, sealed client surfaces) cut over to read from the v5 substrate. The v4 substrate continues to receive

writes for safety. Forecaster cuts over to reading v5; Evaluator continues to read v4 for a measured overlap window.

Pass conditions: Trace Feed and chapter projection reads from v5 produce equivalent customer-facing rendering to v4 reads (manual sample audit on the launch universe; no automated surface-comparison test substitutes for human verification at this gate); customer escalations from confused trace-vs-belief language fall to zero; the nine enforcement rules of §06 are checked in code on every commit path.

Phase 4 · Write-cutover The Orchestrator stops writing to v4. The v5 substrate is now sole authoritative. The v4 tables remain as a frozen historical record, queryable via `retrospective_view` but not authoritative. Forecaster and Evaluator both run from v5.

Pass conditions: zero unhandled write paths on v4 over 14 consecutive days (any code path that still writes v4 is a bug, not a configuration); replay-reproduction rate on v5 above 0.95; all calibration aggregates on v5 cover at least 90 days of in-system data.

Phase 5 · v4 retirement The v4 tables are archived. v5 is canonical. The phase is complete when all of the following hold continuously for 30 days: the nine enforcement rules pass at every commit, the Track 1/2/3 calibration aggregates are within bounds, no read path queries v4, and the operator-facing metrics dashboard shows the trace graph as the system's primary substrate.

The autonomy commitment is preserved across the phase plan: at no phase does a human review gate the phase advance. Gates are substrate metrics. Phase regression, moving back to a prior phase when a gate breaches, is the architecture's safety mechanism, not human override. A sustained breach of any phase's pass conditions returns the relevant subsystem to the prior phase's gate set and requires the gates to repass before progress resumes.

§43 Decision memo AMENDED

What is being committed, what is at risk, what remains open — the briefing for any reader deciding whether the v6 architecture is buildable, financeable, and operable.

What v6 commits to

Everything v5.1 committed to, unchanged: the trace as primary object; the 14-step flow as operational and epistemic spine; the nine enforcement rules as substrate-level guarantees; the eight activation states; magnitude and persistence as first-class dimensions; the twelve-table substrate; Threadweave; scoring and orchestration; canonical chapters with the structured outline and Check/06; self-scoring without human-in-the-loop at every gate. **v6 leaves all of that core machinery as the end-state design.** What v6 adds is confined to the two layers the v4.1 trial actually exercised: the deep-dive agentic decomposition (§08), the Orchestrator's per-chapter supervisor steer (§14), the cold-start phase as the canonical-chapter seeding path (§17a), the deep-dive-brief ↔ structured-outline binding (§06d), the four-layer enforcement discipline (§06.10), honest non-scoring as a deep-dive-layer convention (§40a), and the PM-facing per-run deliverable (§06e).

The throughline of every v6 addition is the same: **v4.1 is a trail, not the end state.** The reference implementation was a single-shot, ephemeral run that scored a ticker once with no longitudinal record — it deliberately lacked the calibration loop that is the architecture's moat. So v6 does not let v4.1 rewrite core machinery (scoring, orchestration, Threadweave, the substrate). It takes the insight v4.1 genuinely produced — how the deep-dive

agentic layer decomposes, how a canonical chapter seeds from a brief, how enforcement lives at four layers, how an honest "insufficient" answer beats a fabricated one — and folds exactly that into the canonical-chapter and deep-dive layers, where it belongs.

What is at risk

The v5.1 risks stand: trace-authoring quality (a poorly-chosen counter-hypothesis set is structurally weaker than calibration reveals); magnitude and persistence calibration (Track 2 and Track 3 have no in-system data at launch); reactivation-gate strictness; and promotion-threshold calibration. v6 adds one risk of its own. **Cold-start quality variance:** the cold-start phase is the most expensive single operation and the one a customer sees first, so a weak cold-start on a high-profile entity is a reputational risk before any calibration data exists to catch it; the mitigation is the weeks-3–4 hand-review of the first ten entities and the rejection-and-reroute telemetry.

What remains open

The v5.1 open items (counter-hypothesis curation, cross-tenant trace linking, materiality interpretation across trace types, persistence for short-horizon ad-hoc traces) carry forward. v6 adds the two from §25 (Sentiment's home in the dragnet; structured-outline patterns for the four meta-chapters), plus the deferred items in §A4.

The v6 architecture continues the discipline of v4, v5.0, and v5.1: typed contracts, autonomous operation, separable layers, gate-enforced commitments, calibration that compounds. v6 is not a paradigm shift over v5.1, and it is not a port of v4.1. It is the disciplined act of folding the v4 / v4.1 trial's hard-won insight — the deep-dive agentic decomposition, the canonical-chapter seeding path, four-layer enforcement, and honest non-scoring — into the canonical-chapter and deep-dive layers, while leaving the calibration loop, the trace primary, scoring, orchestration, Threadweave, and the twelve-table substrate exactly as load-bearing as v5.1 made them. The architecture's commitment is unchanged: we know what would have to be true for this to be useful, we have built the mechanisms to keep those things true, and we will pause anything that drifts.

APPENDIX

v6 amendment notes

What the amendment did and did not do, the deep-dive-brief working-artifact shape, what v4.1 validates without change, the deferred questions, and the full traceability matrix back to the v4.1 source files.

§A1 What the v6 amendment did, and did not do

v5.1 specified the canonical chapter object class and the bidirectional chapter ↔ trace relationship, but it was written before the v4 / v4.1 reference implementation had been consolidated into a single hand-off document, and it left several operational questions either unspecified or under-reconciled. v6 closes those gaps using learnings that v4.1 actually shipped and exercised.

What v6 added

- Four new sections: §06e (PM-facing deliverable), §17a (the cold-start phase as the canonical-chapter seeding path), §17b (degrade-safe input handling — an operational discipline on agent input contracts, not a runtime change), and §40a (honest non-scoring as a deep-dive-layer convention).
- One new meta-rule: §06.10 (the four-layer enforcement discipline).
- Amendments to the deep-dive agentic layer (§08, chapter-keyed decomposition), the Orchestrator's supervisor steer (§14), and the structured-outline binding (§06d).
- Cross-reference and prose edits to §01, §06, §09, §20, §24, §25, §43.

What v6 deliberately did not do

- It did not touch the core machinery: the calibration loop, the trace primary, scoring, orchestration, Threadweave (§13), the nine enforcement rules, the parent-updater function, or the twelve-table substrate (§20). These are v5.1's end-state design and v6 leaves them as-is. v4.1 is a trail that yields insight, not the end state.
- It did not add any source-of-truth tables. The substrate stays at twelve. The cold-start trial's filesystem artifacts (stats block, top-level node, deep-dive briefs, reverse-DCF surface) flow into the existing tables as canonical chapters, atomic observations, and audit-log entries — they do not become new ledgers (§20).
- It did not change core trace scoring. The honest-non-scoring discipline of §40a is a deep-dive-brief convention; whether it propagates into a `trace_snapshot` field is flagged as open (D/11, §25), not committed.
- It resolved the Deep Insight decomposition (D/07): the base layer is the ten company-facing chapter agents, the four meta-chapters sit in the priors/validation tier (§07, §08, §06c). It did not resolve the chapter-level scoring method (D/08, core Threadweave), which remains open in §25.

- It did not propose an MVP that defers predictions, outcomes, or calibration. The cold-start phase is the seeding path for the loop, not a substitute for it (§24).
- It did not adopt v4.1's file-based, single-shot, ephemeral execution model. v4.1 is the reference for *what the agents, schemas, and prompts look like in working code*; it is not the target architecture.

§A2 The deep-dive-brief working artifact

v6 adds **no** migration to the substrate. The twelve source-of-truth tables of v5.1 are unchanged; there is no DDL to run against the trace substrate. What v6 specifies is the *shape of the deep-dive brief* — the canonical-chapter / deep-dive-layer working artifact (§06d, §17a) — as an application-layer contract (Pydantic / JSON schema), not a database table. The brief is a transient seed-mode input: it materialises into a `canonical_chapters` row and its candidate traces, and the canonical chapter is its persistent form. The shape, derived from what the v4.1 trial produced:

```
-- v6: deep-dive brief as an application-layer schema (NOT a DB table)
-- A transient working artifact; persisted form is a canonical_chapters row.

DeepDiveBrief = {
  entity_id:      str
  chapter_type:   str          # one of the dragnet chapter types (§06c)
  run_id:         str
  checklist_section: str      # P16 Ch.11 mapping (10 company-facing chapters)
  checklist_items: [ {
    finding:      str
    evidence_citations: [SourceRef] # cite the bundle; never invent
    confidence:   "H" | "M" | "L" | "INSUFFICIENT" # §40a
    evidence_gap: str | null      # required iff INSUFFICIENT
    load_bearing_for: [TraceRef]  # outline -> trace mapping (§06d)
  } ]
  narrative:      { summary: str, interpretive_call: str }
  explicit_handoff: HandoffSpec      # typed claims -> trace updates
  built_from:     [SourceRef]        # source bundle provenance
}

-- On seed-mode commit (§06b), the brief's items become a structured_outline;
-- Check/06 (§06d) validates outline->snapshot consistency; the harness
-- materialises a canonical_chapters row + candidate traces in one transaction.
-- No new source-of-truth table is created.
```

Structured-outline validation is enforced at the application layer, because the outline references `trace_snapshot_id` values whose existence and consistency are verified by Check/06 at commit time (§06d). The four-layer enforcement discipline of §06.10 applies: the brief's contract lives in the agent prompt, the schema, the commit-path gate, and a regression test — not in a substrate table.

§A3 What v4.1 confirms about the architecture, no change needed

Not everything v4.1 surfaced was a gap. Several parts of the architecture were *validated* by the v4.1 trials and need no change — recording them here prevents a future revision from "fixing" what already works.

1. **Typed schemas at every boundary work.** The v4.1 schemas caught dozens of agent output errors during stub testing and produced zero false-positives in the live run. The "structured before fluent" principle is correct.
 2. **Several narrow specialists beat one monolithic agent.** The chapter-keyed decomposition ran ten deep-dives in parallel in roughly ten minutes wall-clock with consistent quality. The right shape is several narrow specialists, which is exactly the six-DI / FM / SF decomposition v5.1 already specified.
 3. **The same architectural discipline works on filesystem first.** Versioned snapshots, append-only commits, schema-validated boundaries — v4.1 demonstrated these on a filesystem artifact tree. The Postgres substrate is the right target, but the migration from v4.1's discipline to v6's substrate is conceptually a `pg_dump`-like import, not a re-architecture.
 4. **The corpus model (cite-only, no invention) holds under real LLM dispatch.** v4.1's "cite the bundle; briefs are not evidence" discipline was respected across all ten deep-dives in the live run, with zero hallucinated facts in post-hoc review. The prompt-level prohibitions plus schema requirements work.
 5. **Convergence with budgeted re-spawn is the right epistemic loop.** v4.1's three-iteration re-spawn budget was sufficient (the live entity converged on the first iteration of both passes). The parent-updater and reactivation gate are the longitudinal analogues — the same shape, applied over time rather than within a single run.
-
-

§A4 Open questions deferred to v6.1 or later

Three items were flagged but not specified in v6:

Granularity of chapter_types. Whether `moat` is one chapter or splits into `switching_costs`, `pricing_power`, `distribution_advantage`, `brand_advantage`. v6 commits to the single-chapter form per `chapter_type` at launch; splitting is empirical and deferred until calibration data justifies it. (Note that the fourteen-chapter dragnet already splits some of this — ST-4 switchability is distinct from ST-7 moat — so the question is really about sub-chapter granularity within an ST chapter.)

Threadweave's consumption of canonical chapters. Threadweave consumes the trace graph and `trace_relationships`. Whether it should also consume canonical-chapter prose, and how, is open. Conservative position: Threadweave reads typed substrate, not prose; it operates on the trace graph, not on the canonical chapter text. (§13 is unchanged from v5.1.)

Structured-outline patterns for the four meta-chapters. ST-1, ST-2, ST-11, and ST-13 score properties of the run rather than the company, and v4.1 provided no structured-outline pattern for them (it has the P16 Ch.11 mapping only for the ten company-facing chapters). These patterns must be authored before the meta-chapters can be seed-authored rather than hand-built. Carried as D/10 in §25.

§A5 The v4.1 → v6 traceability matrix

Every v6 amendment, mapped to the v5.1 section it updates and the v4.1 reference-implementation source that justifies it. Citations are bidirectional, in the spirit of the original deltas analysis: each row points at a concrete artifact, not a general impression.

V6 SECTION	WHAT CHANGED	V4.1 SOURCE
§01	Structural / numerical PIE separation stated in the one-page summary	<code>reverse_dcf.py</code> ; SKILL.md numerical-PIE policy
§06.10	Four-layer enforcement meta-rule (substrate + schema + prompt + test)	<code>convergence_flags.schema.json</code> ; <code>convergence_checker.py</code> sentinel; <code>test_convergence_checker.py</code>
§06c	Chapter set reconciled to the 14-chapter dragnet; per-chapter specialist ownership pinned	<code>chapter_definitions.yaml</code> (996 nodes); <code>orchestrator_v4.py::MANDATORY_CHAPTERS</code>
§06d	Check/06 grounded in the v4.1 brief field mapping; P16 Ch.11 as seed template	<code>deep_dive_checklists.yaml</code> ; <code>deep_dive.schema.json</code>
§06e	PM-facing 8-part branded PDF deliverable	SKILL.md output contract; <code>test_pdf_minimum.py</code>
§07 / §08	DI specialists committed as chapter-keyed agents (D/07): 10 company-facing base agents + 4 meta-chapters in the priors/validation tier	<code>deep_dive_agent_template.md</code> ; 10 live deep-dive briefs
§09	Stats-block primitive; two-surface (structural / numerical) PIE	<code>stats_block.schema.json</code> ; <code>build_stats_block.py</code> ; <code>reverse_dcf.py</code>
§13	Threadweave unchanged (core); chapter-level scoring method flagged open (D/08)	chapter-rollup logic in <code>chapter_agent_template.md</code> noted as a candidate, not adopted
§14	Orchestrator emits a typed per-chapter supervisor steer	<code>top_level_node.schema.json</code> ; <code>top_level_node_agent.md</code>
§17a	Cold-start input phase (11-stage bootstrap before the continuing flow)	<code>orchestrator_v4.py</code> state machine; live <code>audit.json</code>
§17b	Degrade-safe input handling (required vs optional input contract)	v4.1 optional-brief boundary behaviour; <code>provenance.degraded_inputs[]</code>
§20	Substrate unchanged at twelve tables; cold-start artifacts clarified as deep-dive-layer working artifacts	<code>stats_block</code> / <code>top_level_node</code> / <code>deep_dive</code> schemas (trial artifacts, not promoted to tables)
§24	Explicit cold-start budget (~\$30/entity, ~\$3k for 100)	v4.1 per-ticker cost figures (planning update)
§40a	Honest non-scoring (<code>INSUFFICIENT</code>) as a deep-dive-	

	layer convention; core propagation flagged open (D/11)	<code>deep_dive.schema.json</code> confidence enum (INSUFFICIENT); 6 live INSUFFICIENT items
--	---	---

End of v6 architecture document.

This document represents a commitment by StahlTrace Lda to build, operate, and discipline a self-scoring trace graph, now with the cold-start operational layer made first-class. The commitments here are testable and pause-able. The scope at launch is described in §24; the open questions in §25 and the appendix are committed to be resolved on the phase plan's cadence. v6 supersedes v5.1 in full; the v4 / v4.1 reference implementation remains the working code reference — a trail, not the end state — for the agents, schemas, and prompts in the canonical-chapter and deep-dive layers that v6 specifies.

Confidential. © Ronald Moesker / StahlTrace Lda · Architecture v6.